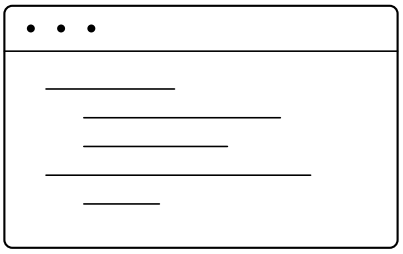
Visual Studio 2026 使用教程

（2026 年 10 月 3 日）



爱吃烧饼的大熊猫

纸上得来终觉浅，绝知此事要躬行。

目录

第一章 绪论 **2**

1.1 这是一本什么样的书 . . . 2
1.2 全书结构 . . . 2
1.3 环境要求 . . . 2
1.4 排版约定 . . . 3
1.5 关于代码示例 . . . 3
1.6 遇到问题怎么办 . . . 4

第二章 Visual Studio 2026 概览与版本选择 **5**

2.1 版本谱系：从 17.x 到 18.x . . . 5
2.1.1 版本号怎么读 . . . 5
2.1.2 关键时间线 . . . 5
2.1.3 与 Visual Studio 2022 的关系 . . . 6
2.2 选择版本（SKU） . . . 6
2.2.1 四种产品 . . . 6
2.2.2 通道：Stable 与 Insiders . . . 7
2.2.3 授权提醒 . . . 7
2.3 MSVC 工具集：本代最大的架构变化 . . . 7
2.3.1 版本解耦 . . . 7
2.3.2 平台工具集 `v145` . . . 8
2.3.3 二进制兼容性 . . . 8
2.3.4 精确锁定工具集版本 . . . 9
2.4 C++ 语言与标准库现状 . . . 9
2.4.1 一个必须澄清的坑：默认标准有两个 . . . 9
2.4.2 可用的标准档位 . . . 10
2.4.3 C++23 与 C++26 . . . 11
2.5 面向 C++ 的 IDE 新特性速览 . . . 11
2.5.1 界面与编辑 . . . 11
2.5.2 项目系统与构建 . . . 11
2.5.3 调试与诊断 . . . 12
2.5.4 AI 辅助 . . . 12

2.6 破坏性变更与迁移 . . . 12
2.6.1 已移除的功能 . . . 12
2.6.2 已弃用的功能 . . . 13
2.6.3 目标平台下限提高 . . . 13
2.6.4 值得留意的新警告与行为变化 . . . 13
2.6.5 升级的推荐路径 . . . 13
2.7 我该装哪个？决策清单 . . . 14
2.8 小结 . . . 14
2.9 练习题 . . . 14
2.10 相关笔记 . . . 15
2.11 参考链接 . . . 15

第三章 安装与环境配置 16
3.1 装之前：先确认三件事 . . . 16
3.1.1 主机系统必须是 Windows 11 . . . 16
3.1.2 硬件与磁盘 . . . 16
3.1.3 和 VS 2022 并存 . . . 17
3.2 下载与启动安装器 . . . 17
3.2.1 三种获取方式 . . . 17
3.2.2 命令行安装（适合批量部署） . . . 18
3.3 选择工作负荷：这一步决定你以后能不能干活 . . . 18
3.3.1 必选：使用 C++ 的桌面开发 . . . 19
3.3.2 可选工作负荷 . . . 19
3.3.3 值得单独勾选的”单个组件” . . . 19
3.4 安装位置与磁盘规划 . . . 20
3.5 首次启动 . . . 20
3.5.1 登录与账户 . . . 20
3.5.2 选择开发设置 . . . 21
3.5.3 挑一套主题 . . . 21
3.6 验证安装：确认工具链真的可用 . . . 21
3.6.1 从”开发者命令提示符”验证 . . . 21
3.6.2 验证 CMake 与 vcpkg . . . 21
3.6.3 用 IDE 跑一个最小程序 . . . 22
3.7 工具集版本管理 . . . 22
3.7.1 三种工具集并存 . . . 22
3.7.2 安装预览工具集 . . . 22
3.7.3 安装旧工具集 / 在升级后恢复旧工具集 . . . 22
3.7.4 锁定工具集版本 . . . 22
3.8 更新、修复与卸载 . . . 23
3.8.1 更新 . . . 23

3.8.2 修复 . . . 23
3.8.3 修改（补装/卸载组件） . . . 23
3.8.4 卸载 . . . 23
3.9 常见安装问题 . . . 23
3.10 小结 . . . 24
3.11 练习题 . . . 24
3.12 相关笔记 . . . 25

第四章 界面与工作区 **26**
4.1 起始窗口 . . . 26
4.2 主界面布局总览 . . . 26
4.3 解决方案资源管理器 . . . 27
4.4 代码编辑器 . . . 28
4.4.1 文档选项卡 . . . 28
4.4.2 编辑器左下角与右下角 . . . 28
4.4.3 滚动条上的信息 . . . 28
4.5 工具窗口体系 . . . 28
4.5.1 高频窗口 . . . 28
4.5.2 调试期才出现的窗口 . . . 29
4.5.3 查找窗口的两种形态 . . . 29
4.6 窗口布局管理 . . . 29
4.6.1 停靠与浮动 . . . 29
4.6.2 多显示器 . . . 29
4.6.3 保存与切换布局 . . . 29
4.7 主题、字体与视觉体验 . . . 30
4.7.1 主题 . . . 30
4.7.2 自定义主题颜色 . . . 30
4.7.3 字体与颜色 . . . 30
4.8 快捷键体系与自定义 . . . 30
4.8.1 为什么值得花十分钟学 . . . 30
4.8.2 必背的一批 . . . 30
4.8.3 自定义快捷键 . . . 31
4.8.4 键盘映射方案 . . . 31
4.8.5 导出与导入设置 . . . 32
4.9 快速启动：`Ctrl+Q` . . . 32
4.10 设置与选项 . . . 32
4.10.1 选项对话框 . . . 32
4.10.2 项目级设置 vs 全局设置 . . . 32
4.10.3 设置同步 . . . 33
4.11 让界面顺手的三条建议 . . . 33

4.12 小结 . . . 33
4.13 练习题 . . . 33
4.14 相关笔记 . . . 34

第五章 第一个 C++ 项目 35
5.1 创建控制台应用 . . . 35
5.1.1 走一遍流程 . . . 35
5.1.2 模板选哪个 . . . 35
5.2 项目文件结构解析 . . . 36
5.2.1 生成后磁盘上有什么 . . . 36
5.2.2 每个文件干什么 . . . 36
5.2.3 在 IDE 里看结构 . . . 37
5.3 读懂生成的代码 . . . 37
5.4 生成与运行 . . . 38
5.4.1 `Ctrl+F5` 与 `F5` 的区别 . . . 38
5.4.2 生成输出：`Ctrl+Alt+O` . . . 38
5.4.3 错误去哪看：`Ctrl+ Ctrl+E` . . . 39
5.4.4 输出窗口 vs 程序输出 . . . 39
5.5 命令行构建 . . . 39
5.5.1 用 MSBuild（推荐） . . . 39
5.5.2 用 devenv（IDE 的命令行外壳） . . . 40
5.5.3 验证生成结果 . . . 40
5.6 配置与平台：Debug/x64 是什么意思 . . . 41
5.7 动手改：加一个头文件和一个源文件 . . . 41
5.8 首次运行常见问题 . . . 43
5.9 小结 . . . 43
5.10 练习题 . . . 43
5.11 相关笔记 . . . 44

第六章 项目与解决方案管理 45
6.1 解决方案、项目、文件：三层关系 . . . 45
6.2 项目文件的解剖 . . . 46
6.2.1 四个文件各管什么 . . . 46
6.2.2 `.sln`：一份“不打算让你手改”的文本 . . . 46
6.2.3 `.vcxproj`：一份精简但真实的骨架 . . . 47
6.2.4 `.vcxproj.filters` . . . 49
6.3 SLNX：基于 XML 的新解决方案格式 . . . 49
6.3.1 从 `.sln` 迁移为 `.slnx` . . . 50
6.4 多项目解决方案 . . . 50
6.4.1 三种“跨项目用代码”的方式 . . . 50

6.4.2 项目引用到底做了什么 . . . 51
6.5 库项目：静态库与动态库 . . . 51
6.5.1 两种库的差别 . . . 51
6.5.2 导出宏：一份完整可编译的示例 . . . 52
6.6 解决方案配置与平台 . . . 53
6.6.1 配置与平台是一对 . . . 53
6.6.2 配置管理器与多项目对齐 . . . 54
6.7 属性表（Property Sheets） . . . 55
6.7.1 为什么需要它、怎么用 . . . 55
6.7.2 `Directory.Build.props`：连“添加”这一步都省掉 . . . 55
6.8 筛选器与文件夹 . . . 56
6.9 共享项目（Shared Items Project，`.shproj`） . . . 57
6.10 项目重定目标：从 `v143` 到 `v145` . . . 58
6.10.1 什么时候需要，怎么改 . . . 58
6.10.2 重定目标改写了哪些字段 . . . 58
6.11 版本控制卫生 . . . 59
6.11.1 哪些文件该进 Git . . . 59
6.11.2 `.gitignore` 片段 . . . 59
6.12 实战：一个三项目解决方案 . . . 60
6.12.1 目录树与职责划分 . . . 60
6.12.2 引用关系 . . . 61
6.12.3 一步步搭起来 . . . 61
6.13 小结 . . . 62
6.14 练习题 . . . 62
6.15 相关笔记 . . . 63

第七章 项目属性与编译配置 64
7.1 属性页的入口与结构 . . . 64
7.1.1 三种打开方式 . . . 64
7.1.2 顶部两个下拉框：新手最大的坑 . . . 64
7.1.3 属性页树形结构 . . . 65
7.2 配置属性 > 常规 . . . 66
7.2.1 配置类型：改错了会得到奇怪的产物 . . . 67
7.2.2 输出与中间目录 . . . 67
7.2.3 字符集：Unicode 还是多字节 . . . 67
7.2.4 平台工具集与 Windows SDK . . . 68
7.2.5 C++ 语言标准：属性页选项 ↔ 编译器开关 . . . 68
7.3 C/C++ > 常规 . . . 69
7.3.1 附加包含目录 . . . 69
7.3.2 调试信息格式 . . . 69

7.3.3 警告等级与 /WX . . . 69
7.3.4 多处理器编译（/MP） . . . 70
7.3.5 使用 UTF-8 编码（/utf-8） . . . 70
7.4 C/C++ > 优化 . . . 70
7.5 C/C++ > 预处理器 . . . 71
7.6 C/C++ > 代码生成 . . . 72
7.6.1 运行库：/MD 还是 /MT . . . 72
7.6.2 启用 C++ 异常 . . . 72
7.6.3 其他代码生成选项 . . . 73
7.7 C/C++ > 语言 . . . 73
7.7.1 符合模式 /permissive- . . . 74
7.7.2 关于 ___cplusplus . . . 74
7.7.3 C++ 模块 . . . 74
7.8 C/C++ > 高级 . . . 74
7.8.1 强制包含 /FI . . . 75
7.8.2 显示包含文件 /showIncludes . . . 75
7.9 链接器 . . . 75
7.9.1 常规 . . . 75
7.9.2 输入 . . . 75
7.9.3 系统 . . . 75
7.9.4 优化 . . . 76
7.9.5 高级 . . . 76
7.9.6 调试 . . . 76
7.10 生成事件 . . . 77
7.11 调试属性页 . . . 77
7.12 预编译头（PCH）实践 . . . 78
7.13 多核编译与构建速度 . . . 78
7.14 配置的三种持久化方式 . . . 79
7.15 从默认配置到生产配置：改动清单 . . . 80
7.16 小结 . . . 81
7.17 练习题 . . . 81
7.18 相关笔记 . . . 81

第八章 代码编辑与 IntelliSense 83
8.1 IntelliSense 是什么，不是什么 . . . 83
8.1.1 它是另一套引擎，不是编译器 . . . 83
8.1.2 三个必须接受的现象 . . . 83
8.2 补全与提示 . . . 84
8.2.1 五条常用命令 . . . 84
8.2.2 内联提示（Inlay Hints）与悬停信息 . . . 84

8.3 导航 . . . 85
8.3.1 跳转命令总表 . . . 85
8.4 重构 . . . 85
8.4.1 重命名：Ctrl+R, Ctrl+R . . . 85
8.4.2 创建声明 / 定义、移动定义位置与其它重构 . . . 86
8.5 C++ 专属的智能建议 . . . 86
8.5.1 建议把不修改状态的成员函数标记为 const . . . 86
8.5.2 建议把只在本文件使用的全局函数标记为 static . . . 86
8.5.3 未使用的 #include 会变灰 . . . 87
8.5.4 大小与对齐预览 . . . 87
8.6 代码片段 . . . 87
8.6.1 展开与插入 . . . 87
8.6.2 代码片段管理器与自定义片段 . . . 88
8.7 格式化与代码风格 . . . 89
8.7.1 格式化快捷键 . . . 89
8.7.2 .editorconfig：把风格写进仓库 . . . 89
8.7.3 clang-format 集成 . . . 90
8.8 编辑器效率技巧 . . . 90
8.8.1 多光标与行操作 . . . 90
8.8.2 任务列表、书签、折叠与结构参考线 . . . 91
8.9 VS 2026 的编辑体验改进 . . . 91
8.9.1 C++ 属性语法高亮与快捷键统一 . . . 91
8.9.2 右下角增强控件：查看与修改文件编码 . . . 92
8.9.3 查找排除文件、类视图子串搜索 . . . 92
8.9.4 右键一键生成预处理输出 . . . 92
8.9.5 全代码库语义索引 . . . 93
8.10 GitHub Copilot 在编辑器里的用法 . . . 93
8.10.1 行内补全与 Copilot Chat . . . 93
8.10.2 右键 Copilot Actions 与 Next Edit Suggestions . . . 93
8.10.3 隐私与关闭方法 . . . 94
8.11 常见 IntelliSense 问题与快速处置 . . . 94
8.12 编辑效率速查表 . . . 95
8.13 小结 . . . 96
8.14 练习题 . . . 96
8.15 相关笔记 . . . 96

第九章 调试技术详解 98
9.1 调试的基本流程 . . . 98
9.1.1 调试运行与“不调试运行” . . . 98
9.1.2 调试符号（.pdb）与 Debug 配置 . . . 98

9.1.3 调试器窗口布局与常用快捷键 . . . 99
9.2 断点体系 . . . 100
9.2.1 普通断点与断点图标 . . . 100
9.2.2 条件断点与命中次数 . . . 100
9.2.3 筛选器断点 . . . 101
9.2.4 函数断点 . . . 101
9.2.5 数据断点（监视地址被改写） . . . 101
9.2.6 临时断点、依赖断点与跟踪点 . . . 102
9.2.7 断点标签、导入导出与断点窗口 . . . 102
9.3 单步执行 . . . 102
9.3.1 单步命令与快捷键 . . . 102
9.3.2 设置下一条语句与“往回走” . . . 103
9.3.3 “仅我的代码”对单步的影响 . . . 103
9.4 查看数据 . . . 103
9.4.1 数据提示、自动窗口与局部变量 . . . 103
9.4.2 监视窗口与即时窗口 . . . 104
9.4.3 格式说明符与并行监视 . . . 104
9.5 调用堆栈与线程 . . . 105
9.5.1 调用堆栈窗口 . . . 105
9.5.2 并行堆栈与线程窗口 . . . 105
9.5.3 多线程死锁的排查思路 . . . 105
9.6 内存与底层调试 . . . 106
9.6.1 内存窗口 . . . 106
9.6.2 反汇编窗口与寄存器窗口 . . . 106
9.7 异常处理 . . . 106
9.7.1 异常设置窗口与“两类异常” . . . 106
9.7.2 C++ 异常、SEH 与“在 throw 处中断” . . . 107
9.8 编辑并继续 / 热重载 . . . 107
9.8.1 前提条件 . . . 107
9.8.2 能改什么，不能改什么 . . . 108
9.8.3 陈旧代码与执行点漂移 . . . 108
9.9 高级调试手段 . . . 108
9.9.1 附加到进程与远程调试 . . . 108
9.9.2 转储文件调试 . . . 109
9.9.3 OutputDebugString、TRACE 与 assert . . . 109
9.9.4 调试时的环境变量与命令行参数 . . . 110
9.10 实战示例 . . . 110
9.10.1 用数据断点抓数组越界 . . . 111
9.10.2 用断点 + 监视 + 调用堆栈抓空指针 . . . 111

9.11 小结 . . . 112
9.12 练习题 . . . 112
9.13 相关笔记 . . . 113

第十章 性能分析与诊断 114
10.1 诊断工具窗口（调试时） . . . 114
10.1.1 打开方式与面板构成 . . . 114
10.1.2 CPU 使用率：调用树与热路径 . . . 114
10.1.3 内存使用率、事件与时间线 . . . 115
10.2 性能探查器 . . . 115
10.2.1 启动与 C++ 可用的工具 . . . 115
10.2.2 采样与插桩的区别 . . . 115
10.2.3 内存、文件 I/O 与并发 . . . 116
10.3 Build Insights：编译期性能分析 . . . 116
10.3.1 它解决什么问题 . . . 116
10.3.2 两种使用方式 . . . 116
10.3.3 定位编译慢的根因 . . . 117
10.4 静态代码分析 . . . 117
10.4.1 运行代码分析 . . . 117
10.4.2 警告等级与“警告即错误” . . . 118
10.4.3 C++ Core Guidelines 检查器 . . . 118
10.4.4 SAL 注解：把契约写进声明 . . . 118
10.5 clang-tidy 集成 . . . 119
10.5.1 在 Visual Studio 中启用 . . . 119
10.5.2 `.clang-tidy` 配置文件 . . . 119
10.5.3 常用检查项 . . . 119
10.6 运行时检查 . . . 120
10.6.1 AddressSanitizer（`/fsanitize=address`） . . . 120
10.6.2 `/RTC` 运行时检查 . . . 120
10.6.3 CRT 内存泄漏检测 . . . 121
10.7 代码度量 . . . 122
10.8 实战：一次完整的性能优化 . . . 123
10.9 小结 . . . 124
10.10练习题 . . . 124
10.11相关笔记 . . . 124

第十一章 CMake 与 vcpkg 集成 126
11.1 为什么在 VS 里用 CMake . . . 126
11.1.1 `.vcxproj` 的边界与 CMake 的一等公民地位 . . . 126
11.1.2 两条路的能力对比 . . . 126

11.2 在 VS 中打开 CMake 项目 . . . 127
11.2.1 用“打开文件夹”，不是“打开项目” . . . 127
11.2.2 文件夹视图与 CMake 目标视图 . . . 127
11.2.3 首次配置与工具包（Kit）选择 . . . 128
11.3 CMakePresets.json：团队协作的推荐方式 . . . 128
11.3.1 为什么它取代了 CMakeSettings.json . . . 128
11.3.2 一份可用的 CMakePresets.json . . . 129
11.4 CMake 设置编辑器 . . . 130
11.4.1 界面构成与 Presets 的取舍 . . . 130
11.5 生成器与工具集 . . . 131
11.5.1 `Visual Studio 18 2026` 生成器、`-T` 与自带的 CMake . . . 131
11.5.2 反向：从 CMake 生成并构建 Visual Studio 项目 . . . 131
11.6 构建与调试 CMake 目标 . . . 132
11.6.1 选择启动项、断点与 F5 . . . 132
11.6.2 调试 `CMakeLists.txt` 脚本本身 . . . 132
11.6.3 CMake 模块的 IntelliSense . . . 132
11.7 CMake 项目的多配置 . . . 132
11.7.1 单配置与多配置生成器的差别 . . . 132
11.8 vcpkg 集成 . . . 134
11.8.1 清单模式与经典模式 . . . 134
11.8.2 一份 vcpkg.json 示例 . . . 134
11.8.3 vcpkg-configuration.json 与注册表 . . . 135
11.8.4 工具集对齐与 CMAKE_TOOLCHAIN_FILE . . . 135
11.9 手动管理第三方库的替代方案 . . . 136
11.9.1 直接改目录，或者用 NuGet . . . 136
11.9.2 为什么推荐 vcpkg . . . 136
11.10WSL / 容器 / 远程 Linux 开发 . . . 137
11.10.1 WSL . . . 137
11.10.2 Dev Containers（Docker 与 Podman） . . . 137
11.10.3 远程 Linux（SSH）与远程单元测试 . . . 138
11.11常见坑 . . . 138
11.12端到端实战：从零到 F5 . . . 139
11.12.1 目录结构 . . . 139
11.12.2 三个文件 . . . 139
11.12.3 在 VS 中配置、构建、调试 . . . 140
11.13小结 . . . 141
11.14练习题 . . . 141
11.15相关笔记 . . . 142

第十二章　单元测试与代码质量 **143**
12.1 为什么 C++ 项目需要单元测试 . . . 143
12.1.1 C++ 的沉默错误成本 . . . 143
12.1.2 测试资源管理器 . . . 143
12.2 Microsoft 原生单元测试框架（C++） . . . 144
12.2.1 创建本机单元测试项目 . . . 144
12.2.2 核心宏与断言 . . . 144
12.2.3 完整示例：Calculator 与被测代码 . . . 144
12.2.4 引用被测项目与配置要点 . . . 146
12.3 接入第三方框架 . . . 146
12.3.1 Google Test：vcpkg + CMake . . . 146
12.3.2 Google Test：在 VS / MSBuild 项目中使用 . . . 148
12.3.3 Boost.Test 与 Catch2 . . . 148
12.3.4 框架对比与选型 . . . 148
12.4 测试适配器、筛选与并行 . . . 149
12.4.1 适配器与发现测试 . . . 149
12.4.2 分组与筛选 . . . 149
12.4.3 测试播放列表 . . . 149
12.4.4 并行执行 . . . 150
12.5 代码覆盖率 . . . 150
12.5.1 在 IDE 与命令行中收集 . . . 150
12.5.2 版本能力差异 . . . 150
12.6 TDD 在 C++ 中的落地 . . . 151
12.7 持续集成 . . . 152
12.8 小结 . . . 152
12.9 练习题 . . . 153
12.10相关笔记 . . . 153

第十三章　发布与部署 **154**
13.1 发布前的配置切换 . . . 154
13.1.1 Debug 与 Release 的本质差异 . . . 154
13.1.2 常用优化开关的含义与风险 . . . 155
13.1.3 切换到 Release 后先做的三件事 . . . 156
13.2 运行库选择：`/MD` 还是 `/MT` . . . 156
13.2.1 四个取值 . . . 156
13.2.2 取舍表 . . . 157
13.2.3 混用 `/MT` 与 `/MD` 的后果 . . . 157
13.3 静态链接第三方库与动态链接 DLL . . . 158
13.3.1 两种方式的部署代价 . . . 158
13.3.2 用 dumpbin 查依赖 . . . 158

13.3.3 定位缺失 DLL 的排查路径 . . . 158
13.4 收集运行时依赖 . . . 159
13.4.1 Visual C++ 可再发行程序包 . . . 159
13.4.2 随安装包分发与静默安装 . . . 159
13.4.3 app-local 部署的边界 . . . 160
13.5 生成安装包 . . . 160
13.5.1 三种打包方式对比 . . . 160
13.5.2 Installer Projects 最小制作流程 . . . 161
13.5.3 WiX Toolset 简介 . . . 161
13.6 其他发布形式 . . . 161
13.6.1 绿色免安装版 . . . 161
13.6.2 单文件与自包含 . . . 162
13.6.3 分发渠道一句话说明 . . . 162
13.7 版本信息与资源 . . . 162
13.7.1 添加 .rc 资源文件 . . . 162
13.7.2 版本号字段 . . . 162
13.7.3 图标与属性页设置 . . . 164
13.8 发布检查清单 . . . 164
13.9 发布后常见故障 . . . 165
13.9.1 缺少 VCRUNTIME140.dll 一类 . . . 165
13.9.2 应用程序无法正常启动 0xc000007b . . . 165
13.9.3 DLL 地狱的标准排查步骤 . . . 165
13.10小结 . . . 166
13.11练习题 . . . 166
13.12相关笔记 . . . 167

第十四章 常见问题与效率技巧 168
14.1 编译与链接错误速查 . . . 168
14.1.1 错误号前缀对应哪个阶段 . . . 168
14.1.2 编译器高频错误（C 系列） . . . 168
14.1.3 链接器高频错误（LNK 系列） . . . 170
14.1.4 MSBuild 与 IntelliSense 错误 . . . 171
14.1.5 排错的通用流程 . . . 172
14.2 中文乱码专题 . . . 172
14.2.1 四层编码，先分清错在哪一层 . . . 172
14.2.2 推荐做法与各开关的含义 . . . 173
14.2.3 控制台输出中文的三种方案 . . . 173
14.2.4 一个“中文输出正确”的完整示例 . . . 174
14.3 IntelliSense 与实际编译不一致 . . . 175
14.3.1 典型症状与原因 . . . 175

14.3.2 按顺序修复 . . . 175
14.4 路径与工程习惯 . . . 175
14.4.1 中文与空格路径引发的怪问题 . . . 175
14.4.2 建议的目录规范 . . . 176
14.4.3 一份可用的 .gitignore . . . 176
14.5 调试符号与断点失效 . . . 177
14.6 效率技巧集 . . . 178
14.6.1 编辑与代码 . . . 178
14.6.2 导航 . . . 178
14.6.3 任务列表、代码片段与环境管理 . . . 178
14.7 常用快捷键总表 . . . 179
14.7.1 编辑 . . . 179
14.7.2 导航 . . . 179
14.7.3 调试 . . . 180
14.7.4 构建 . . . 181
14.7.5 窗口 . . . 181
14.8 学习资源 . . . 181
14.9 小结 . . . 182
14.10练习题 . . . 182
14.11相关笔记 . . . 183

附录 A 速查与资源 **184**
A.1 版本与工具集速查 . . . 184
A.1.1 IDE 与 MSVC 的对应关系 . . . 184
A.1.2 平台工具集 . . . 184
A.1.3 关键版本节点 . . . 184
A.2 C++ 语言标准档位速查 . . . 185
A.3 官方资源 . . . 185
A.4 本仓库内的相关笔记 . . . 186

前言

这份教程整理自 Visual Studio 2026 官方文档、C++ 团队博客与笔者在实际项目中的使用经验，面向**用 C++ 写代码的人**。

Visual Studio 2026 是微软于 2025 年 11 月 11 日正式发布的新一代集成开发环境，内部版本号为 18.x。与上一代相比，它有三处根本变化：**MSVC 编译器版本与 IDE 版本解耦**、**新建 C++ 项目默认使用 C++20**、**AI 辅助深度融入日常开发流程**。这些变化让不少沿用多年的经验与配置习惯失效，也是本书存在的理由。

全书共十三章，按”装好能跑 → 会用 IDE → 写出工程级代码”三段推进。每一章都尽量回答三个问题：这个功能解决什么问题、在界面上的哪个位置、以及踩坑时该怎么排查。书中的命令、开关与路径均取自官方文档，并在写作时逐条核对过版本。

本书有 Markdown 与 LaTeX 两个版本，内容一致：Markdown 版便于在 Obsidian 中检索与双链跳转，LaTeX 版便于打印与离线阅读。两者都放在 `计算机类\Visual Studio 2026 使用指南`下。

限于篇幅与版本节奏，书中涉及具体版本号之处均以写作时（2026 年 10 月）的稳定通道为准；微软每月更新一次 IDE，遇到与书中不一致的地方，请以 **帮助 > 关于 Microsoft Visual Studio** 显示的版本号与官方发行说明为准。

第一章　绪论

1.1　这是一本什么样的书

这不是一本 C++ 语言教材。语法、STL、面向对象这些内容属于语言本身，与用哪个编辑器无关；本书只讲一件事：**在 Windows 上用 Visual Studio 2026 把 C++ 代码写出来、调通、测好、发出去**。

如果你符合下面任意一条，本书就是为你写的：

- 已经会写 C++，但一直用命令行或轻量编辑器，想转到完整的 IDE；
- 用过 Visual Studio 2022，现在要迁移到 2026，想知道哪些地方变了；
- 正在被某类问题反复困扰：Release 编不过、断点不生效、依赖 DLL 找不到、中文乱码；
- 需要把个人项目升级成有测试、有依赖管理、有发布流程的工程。

1.2　全书结构

全书分三部分，共十三章，建议按顺序阅读。

部分	章节	解决什么问题
第一部分	第 1–4 章：版本选择、安装配置、界面、第一个项目	把环境装对、把第一个程序跑起来
第二部分	第 5–8 章：项目与属性管理、代码编辑、调试	日常开发真正高频使用的部分
第三部分	第 9–13 章：性能诊断、CMake 与 vcpkg、单元测试、发布、常见问题	从”能跑”到”能交付”

各章末尾都有**小结**与**练习题**。练习题多为动手验证型（例如”把运行库从 `/MD` 改成 `/MT`，用 `dumpbin /dependents` 对比依赖列表”），做完能形成肌肉记忆。

1.3　环境要求

本书假定你使用下列环境。版本差异较大的地方，正文会单独说明。

项目	要求
主机操作系统	**Windows 11**（含 ARM64 版），或 Windows Server 2025 / 2022 / 2019。Windows 10 不在官方支持列表内
硬件	内存最低 4 GB、推荐 16 GB；四核以上 CPU；磁盘典型占用 20–50 GB
IDE	Visual Studio 2026，内部版本号 18.x
编译器	MSVC Build Tools 14.50（长期支持）/ 14.51 / 14.52（预览），平台工具集 `v145`
C++ 标准	新建项目默认 `/std:c++20`；可选 C++14 / C++17 / C++20 / C++23 / 最新工作草案

注意 **”主机要 Windows 11”和”程序要跑在 Windows 10 上”并不矛盾**。前者约束你运行 IDE 的机器，后者约束你编译出来的可执行文件——MSVC 14.50 起，产物最低可运行于 Windows 10 / Windows Server 2016。这两件事经常被混淆，第 1 章会详细说明。

1.4 排版约定

为便于阅读，全书统一使用下列记号。

记号	含义
项目 > 属性 > C/C++ > 语言	界面菜单路径，> 表示逐级展开
`Ctrl+K, Ctrl+D`	键盘快捷键。两段式表示先按前一段、松开再按后一段
`/std:c++20`	编译器开关，直接写 MSVC 形式
`$(OutDir)`	MSBuild 宏
`VC\Tools\MSVC`	磁盘路径
`v145`	平台工具集名称

1.5 关于代码示例

书中的代码都是可编译的最小示例，默认在 **控制台应用**模板下创建、使用 `/std:c++20`。为节省篇幅，示例省略了部分 `##include`，也省略了与当前话题无关的异常处理。

提示 **性能相关的结论一律以 Release 配置为准**。Debug 配置关闭了优化（`/Od`），在它下面测出的耗时可能比 Release 慢十倍以上，据此优化会得出完全错误的结论。详见第 9 章。

1.6　遇到问题怎么办

1. 先查第 13 章的**编译与链接错误速查表**——LNK2019、C4819、MSB8020 这类高频错误都有”原因 + 解决方案”两列。
2. 界面找不到某个功能时，按 Ctrl+Q 打开快速启动，直接搜索命令名或设置名，比翻菜单快得多。
3. 版本相关的疑问，以官方发行说明为准；本书第 1 章给出了各版本的发布时间与随附工具集。

第二章　Visual Studio 2026 概览与版本选择

装 Visual Studio 之前，先搞清楚一件事：**这一代（18.x）和上一代（17.x）的差别不只是版本号**。MSVC 编译器的版本号从这一代起和 IDE 版本号”离婚”了，新建项目的默认 C++ 标准从 C++14 提到了 C++20，主机系统要求也从 Windows 10 提到了 Windows 11。这一章把这些变化讲清楚，让你在下载安装包之前就知道自己要装什么、会得到什么。

2.1　版本谱系：从 17.x 到 18.x

2.1.1　版本号怎么读

Visual Studio 有两套版本号，很多人第一次看会懵：

名称	形式	例子	含义
产品名	年份	Visual Studio 2026	市场名称，按年发布
内部版本号	`18.<` 次版本 `>.<` 维护号 `>`	`18.10.3`	稳定通道的正式版本
Insiders 版本号	`18.<` 次版本 `> Insiders <` 构建号 `>`	`18.10 Insiders 2`	预览通道的版本
MSBuild 版本	`18.x`	MSBuild 18.x	与 IDE 主版本号绑定

对照上一代：Visual Studio 2022 的内部版本号是 `17.x`，Visual Studio 2019 是 `16.x`。所以看到 `18.x`，就是 VS 2026。

> **提示** 在 IDE 里按 **帮助 > 关于 Microsoft Visual Studio** 可以看到完整的版本号与构建号。写作本教程时，稳定通道最新为 **18.10.3**（2026 年 9 月 29 日发布，构建号 12224.181）。

2.1.2　关键时间线

时间	事件
2025 年 9 月 9 日	**Visual Studio 2026 Insiders 通道上线**，取代 VS 2022 时代的”Preview 通道”
2025 年 11 月 11 日	**正式发布（GA）**：版本 18.0.0，构建号 11205.157，随附 MSVC 14.50 与 `v145` 工具集
2026 年 5 月	版本 18.6，随附 MSVC 14.51
2026 年 9 月 29 日	版本 18.10.3（写作时的最新稳定版）
2026 年 11 月（预计）	MSVC 14.52 进入稳定通道，预计提供 `/std:c++23`

2.1.3 与 Visual Studio 2022 的关系

- **2022 仍在服务期内**：VS 2022 会一直提供安全更新到 2032 年 1 月，不必急着迁移。
- **VS 2026 的服务期**：到 2028 年 11 月。
- **安装目录独立**：VS 2026 与 VS 2022 分别装在 `C:\Program Files\Microsoft Visual Studio\18` 和 `C:\Program Files\Microsoft Visual Studio\2022` 下的版本子目录里，两者可以并存。
- **项目文件不用改**：用 VS 2022 建的 `.vcxproj` 可以直接用 VS 2026 打开，**不需要升级项目文件格式**。只有在你主动”重定目标”时，才会改写 Windows SDK 版本、平台工具集（`v143 -> v145`）和 MSVC 工具集版本。
- **扩展兼容**：VS 2026 **开箱支持 Visual Studio 2022 的扩展**，这是这一代特意做的工作，减少了迁移时的扩展断供问题。

注意 从 VS 2026 起，年度版本采用**就地更新**（in-place）而不是并排安装。也就是说将来从 2026 升到下一个年度版本时，是原地升级，不会像过去那样在”程序和功能”里躺着两个 Visual Studio。

2.2 选择版本（SKU）

2.2.1 四种产品

产品	面向对象	是否免费	说明
Community	学生、个人开发者、开源项目、小型团队	[OK]	功能最全的免费版；**只能在 Stable 通道**，且必须在 180 天内更新到较新版本
Professional	专业开发者、中小团队	[X]	增加企业级协作与部分工具能力

产品	面向对象	是否免费	说明
Enterprise	大型企业	[X]	增加高级测试、诊断与架构工具（如代码覆盖率、IntelliTrace 等）
Build Tools	CI 服务器、无界面构建机	[OK]	**只有编译器与构建工具，没有 IDE**；用于自动化构建流水线

对绝大多数读者来说，**Community 就是正确选择**——C++ 开发所需的一切（MSVC 编译器、调试器、性能探查器、CMake 支持）它都有。

提示 如果你只是想在服务器上跑 `msbuild` 编译，不要装完整 IDE，装 **Build Tools** 就够了，体积小得多。

2.2.2 通道：Stable 与 Insiders

这一代把通道名称改了，很容易和”预览功能”混淆：

通道	旧称	更新频率	适用场景
Stable	Current（正式版）	每月一次功能更新	日常开发、生产环境
Insiders	Preview（预览版）	更频繁，可提前约一个月拿到新工具集 RC	尝鲜新特性、提前验证工具集

需要注意：**”通道”和”预览功能”是两套独立机制**。即使你用的是 Stable 通道，也可以在 **工具 > 选项 > 预览功能**里逐个打开某些还没正式发布的单项功能。

注意 Community 版**不能**加入 Insiders 通道。想尝鲜 Insiders，要么用 Professional/Enterprise，要么用 Insiders 的 Build Tools。

2.2.3 授权提醒

Community 的免费授权有适用范围（个人、开源、学术，以及规模较小的组织）。团队使用前请核对官方授权条款——本教程不构成法律意见。

2.3 MSVC 工具集：本代最大的架构变化

2.3.1 版本解耦

这是理解 VS 2026 的关键。**从 VS 2026 起，MSVC 编译器的版本号不再与 Visual Studio 的版本号同步**。

过去的规律是：VS 2022 的每个小版本都对应一个 MSVC 版本（17.10 -> 14.40，17.11 -> 14.41……），一个萝卜一个坑。现在不一样了：MSVC 有了自己的节奏，**大约每 6 个月发布一个新版本**，而 IDE 仍然是每月更新一次。于是同一个 VS 版本里，你可以选择不同的工具集。

MSVC 版本	_MSC_VER	编译器版本	首次随附于	支持类型与终止
14.50	1950	19.50	VS 2026 18.0（2025 年 11 月）	**长期支持（LTS）**，2028 年 11 月
14.51	1951	19.51	VS 2026 18.6（2026 年 5 月）	标准支持，2027 年 2 月
14.52	1952	19.52	预览通道	预览，正式发布后确定

作为对比，VS 2022 最后几个版本是 14.43（_MSC_VER 1943）、14.44（1944）。**_MSC_VER 从 19xx 跳到 1950**，正是这次解耦的直接体现。

实战场景 团队里有人用 14.50、有人用 14.51，编出来的二进制能不能混用？可以——见下一节。

2.3.2 平台工具集 v145

在 MSBuild 项目（.vcxproj）里，编译器不是按版本号指定的，而是按**平台工具集**（Platform Toolset）指定的：

```
VS 2015 -> v140
VS 2017 -> v141
VS 2019 -> v142
VS 2022 -> v143
VS 2026 -> v145   <- 本代
```

所以打开一个老项目，如果属性页里写着 v143，说明它还在用 VS 2022 的工具集。VS 2026 可以**向下兼容地**安装并使用旧工具集（最早可到 VS 2015），方便你逐个迁移项目。

工具集的实际安装位置：

```
C:\Program Files\Microsoft Visual Studio\18\<版本>\VC\Tools\MSVC\
+-- 14.50.35717\     MSVC 14.50（长期支持）
+-- 14.51.36231\     MSVC 14.51

其中 14.50 / 14.51 是版本号，35717 / 36231 是具体工具集版本
```

2.3.3 二进制兼容性

MSVC 14.50 与 14.51 都保持了与 VS 2015 及以后版本编译出的代码的二进制兼容性。这条规则很重要：

- 用 14.50 编译的**静态库**，可以链接进用 14.51 编译的程序；
- 用不同 MSVC 版本编译的 DLL 与 EXE 可以互相调用；
- 但**前提是运行库（CRT）链接方式一致**（都用 `/MD` 或都用 `/MT`）——这条规矩没变，详见**第 12 章**。

注意 二进制兼容不代表行为一致。不同工具集版本在优化、警告、标准库实现细节上仍可能有差异，正式发布前应当用**固定的工具集版本**重新构建一次。

2.3.4 精确锁定工具集版本

大型项目常常需要”锁死编译器版本”以保证可复现构建。三种做法：

```
:: 1) 命令行指定工具集版本
msbuild MyApp.sln /p:Configuration=Release /p:VCToolsVersion=14.50.35717

:: 2) 开发者命令提示符里选择工具集
vcvars64.bat -vcvars_ver=14.50

:: 3) 在项目或 Directory.Build.props 里固定
```

```
<PropertyGroup>
  <VCToolsVersion>14.50.35717</VCToolsVersion>
</PropertyGroup>
```

如果这个被固定的工具集**装在另一个 Visual Studio 安装实例里**，从 VS 2026 18.8 起可以打开跨安装发现：

```
<PropertyGroup>
  <EnableVCToolsVersionDiscovery>true</EnableVCToolsVersionDiscovery>
  <VCToolsVersion>14.50.35717</VCToolsVersion>
</PropertyGroup>
```

2.4 C++ 语言与标准库现状

2.4.1 一个必须澄清的坑：默认标准有两个

初学者最容易踩的坑在这里——”**默认 C++ 标准”有两个不同的答案，而且都对**：

场景	默认标准	原因
新建控制台应用 / Windows 桌面应用 / 动态库 / 静态库项目	**C++20**	VS 2026 的项目**模板**在项目属性里写入了 `/std:c++20`
编译器在没有指定 `/std` 时的行为	**C++14**	MSVC 编译器的历史默认值，为兼容旧代码而保留

换句话说：新建项目你拿到的是 C++20；但如果你手写 `cl.exe` 命令、或者改了一个没设 `/std` 的老项目，你拿到的是 C++14。

> **提示** 不要依赖默认值。在 **项目 > 属性 > C/C++ > 语言 > C++ 语言标准**里显式选一个，团队协作时尤其如此。

2.4.2 可用的标准档位

开关	属性页选项	说明
`/std:c++14`	ISO C++14 标准	编译器的历史默认值
`/std:c++17`	ISO C++17 标准	
`/std:c++20`	ISO C++20 标准	**新建项目的默认值**；自动启用 `/permissive-`
`/std:c++23preview`	预览 - ISO C++23 标准	只启用 C++23 范围内的特性，不含实验性内容；**预览特性可能不保证 ABI 稳定**
`/std:c++latest`	预览 - 最新 C++ 工作草案	启用所有已实现的新特性，含进行中的实验特性；自动启用 `/permissive-`

两个细节：

1. **`/std:c++20` 与 `/std:c++latest` 会自动打开 `/permissive-`**（严格一致性模式），这会拒绝一些微软扩展语法，老代码可能因此报错。
2. **检测当前标准用 `_MSVC_LANG`，不要用 `__cplusplus`**——除非你显式加了 `/Zc:__cplusplus`，否则 `__cplusplus` 永远是 `199711L`。

```cpp
#include <iostream>

int main() {
    std::cout << "_MSVC_LANG = " << _MSVC_LANG << '\n';
#if _MSVC_LANG >= 202302L
    std::cout << "当前至少是 C++23 档位\n";
#elif _MSVC_LANG >= 202002L
    std::cout << "当前是 C++20 档位\n";
#else
    std::cout << "当前是 C++17 或更低档位\n";
#endif
}
```

2.4.3 C++23 与 C++26

C++23：MSVC 14.50 起显著提高了 C++23 一致性，例如 `##warning` 预处理指令、`auto(x)` 退化拷贝等；14.51 继续补齐，并新增了标准库头文件 `<flat_map>` 与 `<flat_set>`。

> **注意** 写作本教程时（2026 年 10 月），稳定通道提供的 C++23 档位仍是 `/std:c++23preview`。据 C++ 团队博客，MSVC 14.52（预计 2026 年 11 月进入稳定通道）将提供正式开关 `/std:c++23`，届时 C++23 支持达到完整。属性页里的”预览 - ISO C++23 标准”选项会在新开关落地后自动指向 `/std:c++23`。

C++26：截至写作时，官方文档**没有列出 C++26 核心语言特性的支持状态**，涉及 C++26 的公开内容主要是 constexpr `cmath`（`sin`/`cos`/`tan`/`pow`，通过 `/Zc:cmath` 启用）。想用前沿特性就挂 `/std:c++latest`，但要做好”下个版本可能变”的心理准备。

2.5 面向 C++ 的 IDE 新特性速览

以下都是 VS 2026 相对 VS 2022 的 C++ 相关变化，后续章节会逐个展开。

2.5.1 界面与编辑

- **全新 UI，内置 11 套主题**，并可在 **环境 > 视觉体验 > 主题颜色**里直接改 Fluent 颜色令牌，不用装扩展。
- **开箱支持 VS 2022 扩展**。
- **C++ 属性（attributes）语法高亮**：`[[nodiscard]]`、`[[maybe_unused]]` 这类属性终于有自己的颜色了。
- **类视图支持子串搜索**，并大幅提升了大型项目的性能。
- **编辑器右下角增强**：可以直接查看和**修改文件编码**——这解决中文乱码问题时非常有用。
- **查找与替换支持排除文件**（Quick Find / Find in Files），大型仓库里搜索终于能排除 `third_party/` 了。
- **快捷键向其他编辑器看齐**：这一代统一了一批按键约定，从 VS Code 迁过来的人会舒服很多。

2.5.2 项目系统与构建

- **右键直接生成预处理输出**：对着一个 `.cpp` 文件右键选 **预处理**，立刻得到预处理后的结果，排查宏和头文件包含问题时极其高效。
- **自带 CMake 4.1.1**，并且 CMake 提供了 `Visual Studio 18 2026` **生成器**，还支持新的 **SLNX** 解决方案格式——现在可以从 CMake 侧构建 Visual Studio 的 C++ 项目。
- **设置助手（Setup Assistant）**：打开老项目时，帮你把缺失的工具集装上，或者把项目重定目标到新工具集。

- **后端优化**：相比 17.14，在 Unreal Engine City Sample 的 RenderThread 基准上有最高约 **6%** 的运行性能提升。

2.5.3 调试与诊断

- **调试器内联显示函数返回值**：单步跳过函数调用后，直接看到返回值，不用再设监视。
- **工具栏直接设置命令行参数**：`.vcxproj`、CMake、Unreal 项目都支持，不再局限于游戏开发工作负荷。
- **AddressSanitizer 支持 ARM64**（预览）。
- **clang-tidy 配置增强**：可分配更多处理器并行分析，也可追加自定义命令行参数（**项目属性 > 代码分析 > Clang-Tidy**）。
- **容器调试支持 Podman**，与 Docker 体验一致。
- **采样式 PGO（SPGO）预览、Arm SVE** 支持、**Intel APX** 预览、PDB 体积裁剪。

2.5.4 AI 辅助

- **Copilot Chat 的 Ask / Agent 两种模式**：Ask 回答问题，Agent 直接改代码。
- **编辑器右键新增 Copilot Actions**：把选中的代码丢给 Copilot，让它解释、优化、加注释、生成单元测试。
- **C++ 补全质量提升**：Copilot 会把相关文件作为上下文，减少胡编。
- **全代码库语义索引**：导航与着色性能提升 2 倍以上，并为 Copilot 提供跨文件的符号检索能力。
- **C++ 现代化（Modernization）正式可用**：用 `@Modernize` 之类的指令，让 Copilot 把 MSBuild 或 CMake 项目升级到更新的 MSVC，并顺手修掉警告与链接错误。
- **自带密钥（BYOK）**：可以接入 Microsoft Foundry、OpenAI、Anthropic、Ollama 等模型，不必登录 GitHub。

提示 AI 功能是可选的。如果你所在的环境不允许联网或不允许上传代码，在 **工具 > 选项 > GitHub > Copilot** 里关掉即可，IDE 的其他功能不受影响。

2.6 破坏性变更与迁移

从 VS 2022 升级过来，下面这些必须知道。

2.6.1 已移除的功能

移除项	影响	替代方案
C++AMP	GPU 并行扩展库没了	改用 CUDA / SYCL / OpenMP

移除项	影响	替代方案
ARM32 工具链	不能再编译 ARM32 目标	用旧版 MSVC；新项目请转向 **ARM64** 或 ARM64EC
`/DEBUG:FASTLINK`	链接器开关失效	改用 **`/DEBUG:FULL`**

2.6.2 已弃用的功能

- **`/await` 编译开关**：将未来版本移除。C++14/17 项目想用标准协程，请用 `/await:strict`。
- **实验性协程头文件** `<experimental/coroutine>`、`<experimental/generator>`、`<experimental/resumable>`：请迁移到 C++20 的 `<coroutine>`。
- **MSVC 14.51 移除的一批老古董**：TR1（`std::tr1`）、`<hash_map>`/`<hash_set>`、`<experimental/filesystem>`、`stdext::checked_array_iterator` 等。这些大多已经弃用五年以上，但**老代码里确实还藏着**，升级时会集中爆出来。
- **工作负荷调整**：原"C++ Linux" 工作负荷改名为 **" 使用 C++ 的 Linux、Mac 和嵌入式开发"**；**使用 C++ 的移动开发（iOS/Android）与嵌入式/IoT 工具已不再受支持**，未来版本会移除。

2.6.3 目标平台下限提高

MSVC 14.50 起，编译产物最低只能运行在 Windows 10 / Windows Server 2016 上，不再支持 Windows 7、Windows 8、Windows 8.1。

请注意把两件事分开：

```
主机（你运行 IDE 的机器）
  -> 必须是 Windows 11 或 Server 2019/2022/2025

目标（你编译出的 .exe 要运行的机器）
  -> 最低 Windows 10 / Server 2016
```

如果你的软件必须支持 Windows 7，那就不能升级到 14.50，只能停留在旧工具集上（并自行承担旧工具集停止支持的风险）。

2.6.4 值得留意的新警告与行为变化

- **`/Zc:enumTypes`**：底层类型推导相关的新警告，升级后可能刷出一批。
- **P2266 更简洁的隐式移动**：可能影响重载决议，代码里可用 `__cpp_implicit_move` 做条件编译。
- **`<regex>` 多行模式行为变化**：默认改为标准行为；如需旧行为，把 `_REGEX_LEGACY_MULTILINE_MODE` 设为 1。同时 `<regex>` 在这一代做了大幅性能重写，**速度提升明显**。

2.6.5 升级的推荐路径

```
1. 保留 VS 2022（别卸），装上 VS 2026
2. 用 VS 2026 直接打开老解决方案 -> 让它按 v143 编译，先确认能编过
3. 跑一遍测试，确认行为一致
4. 用"设置助手"或手动重定目标到 v145
5. 处理新出现的警告（/Zc:enumTypes、P2266 相关）
6. 用固定的 VCToolsVersion 重新构建并回归测试
7. 全部通过后，再考虑卸载 VS 2022
```

2.7 我该装哪个？决策清单

你的情况	建议
学生 / 个人学习 C++	**Community**，Stable 通道
公司里写 Windows C++ 项目	**Professional** 或 **Enterprise**（看公司授权）
服务器上做 CI 自动构建	**Build Tools**，不要装 IDE
想提前验证新工具集	**Insiders** 通道（注意 Community 不支持）
还在维护 Windows 7 兼容的软件	暂不升级到 14.50+，或双工具集并存
主要写跨平台小工具	直接用 **VS Code**，不必装 VS

2.8 小结

- VS 2026 的内部版本号是 **18.x**，2025 年 11 月 11 日正式发布，**主机要求 Windows 11**。
- **MSVC 版本与 IDE 版本解耦**：14.50（LTS，`_MSC_VER` 1950）、14.51（1951）、14.52（1952，预览），平台工具集统一为 `v145`。
- **新建 C++ 项目默认 C++20**，而编译器在未指定 `/std` 时默认仍是 **C++14**——两者别混。
- C++23 目前用 `/std:c++23preview`；预计 14.52 起提供正式的 `/std:c++23`。C++26 核心特性尚无公开支持。
- **已移除**：C++AMP、ARM32 工具链、`/DEBUG:FASTLINK`；**目标系统下限**提高到 Windows 10 / Server 2016。
- 日常开发选 **Community + Stable** 即可；CI 选 **Build Tools**。

2.9 练习题

1. 打开 **帮助 > 关于 Microsoft Visual Studio**，记下你机器上的完整版本号与构建号，并判断它属于 Stable 还是 Insiders。
2. 写一个程序打印 `_MSC_VER`、`_MSVC_LANG`、`_MSC_FULL_VER`，分别在 `/std:c++14`、`/std:c++17`、`/std:c++20` 下编译，观察三者如何变化。

3. 把一个 VS 2022 时代的项目用 VS 2026 打开，先**不**重定目标，确认能用 `v143` 编译通过；然后重定目标到 `v145`，记录出现了哪些新警告。
4. 用 `/std:c++23preview` 写一段使用 `<flat_map>` 的代码（需要 14.51 及以上工具集），体会 C++23 标准库的新容器。
5. 用 `msbuild /p:VCToolsVersion=...` 固定工具集版本构建一次，再用不固定的方式构建一次，对比两次输出的编译器版本。

2.10 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **下一章：安装与环境配置**
- **C++ 教程：C++ 教程**
- **VS 快速入门：Visual Studio 入门**
- **CMake：CMake 使用指南**

2.11 参考链接

- Visual Studio 2026 发行说明
- MSVC Build Tools 新增功能
- MSVC 编译器版本对照表
- `/std` 语言标准开关
- Visual Studio 2026 系统要求
- C++ 二进制兼容性

第三章　安装与环境配置

安装 Visual Studio 是那种”看起来只要下一步下一步”、实际上能埋下一年隐患的事：工作负荷少勾一个，半年后才发现不能用 CMake；装到了系统盘，C 盘爆红；随手选了预览工具集，编译行为和其他同事对不上。这一章把安装这件事一次做对。

3.1　装之前：先确认三件事

3.1.1　主机系统必须是 Windows 11

这是 VS 2026 最容易被忽略的门槛：

支持	不支持
Windows 11（家庭版、专业版、专业教育版、专业工作站版、企业版、教育版）	**Windows 10**（任何版本）
Windows 11 **ARM64** 版	32 位 Windows
Windows Server 2025 / 2022 / 2019	ARM32、Windows 10 ARM64
	S 模式、IoT 版本、LTSC 版本
	Windows 容器（Build Tools 除外）、非持久化虚拟机、FSLogix 漫游配置文件

注意 如果你现在还在 Windows 10 上，**装不了 VS 2026**。可选方案有三条：升级到 Windows 11；继续用 Visual Studio 2022（服务到 2032 年 1 月）；或改用 VS Code + 独立 MSVC Build Tools（见 **VS Code C++ 配置指南**）。

提示 别把”主机要求”和”目标要求”搞混。IDE 要 Windows 11，但你编出来的 `.exe` 最低能跑在 **Windows 10 / Server 2016** 上——见 **第 1 章 6.3 节**。

3.1.2　硬件与磁盘

项目	最低	推荐
CPU	四核	16 核以上

项目	最低	推荐
内存	4 GB	16 GB（大型 C++ 项目建议 32 GB）
磁盘	官方区间 2.5–210 GB	典型占用 **20–50 GB**，建议预留 60 GB
显示器	1366×768	1920×1080 以上（双屏体验最好）
架构	x64 或 ARM64	x64

IDE 自身还需要 **.NET Framework 4.7.2 以上**（安装阶段）与 **4.8**（运行阶段），以及 **WebView2 运行时**。这些通常随系统或安装器自动就位。

> **实战场景** 如果你的 C 盘只有 30 GB 空闲，**先别点安装**。见本章第 4 节，安装位置可以在安装器里改。

3.1.3 和 VS 2022 并存

两者的安装目录不同，可以共存：

```
C:\Program Files\Microsoft Visual Studio\
+-- 2022\
|   +-- Community\          <- Visual Studio 2022
+-- 18\
    +-- Community\          <- Visual Studio 2026
```

建议先不卸载 VS 2022，等 VS 2026 下的项目全部验证通过再清理。VS 2026 打开 VS 2022 的 `.vcxproj` **不需要升级项目文件**。

3.2 下载与启动安装器

3.2.1 三种获取方式

方式	适用场景	做法
官网下载引导程序	个人开发机	访问 visualstudio.microsoft.com 下载 Community / Professional / Enterprise
winget	命令行爱好者、脚本化部署	见下方命令
直接下载引导程序	需要指定通道	`aka.ms` 短链，见下方

```
# 用 winget 安装 Community（稳定通道）
winget install Microsoft.VisualStudio.Community

# 其他产品 ID
# Microsoft.VisualStudio.Professional
```

```
# Microsoft.VisualStudio.Enterprise
# Microsoft.VisualStudio.BuildTools           <- 只要编译器，不要 IDE
# Microsoft.VisualStudio.Enterprise.Insiders <- Insiders 通道
```

引导程序短链（可直接下载）：

```
稳定通道：https://aka.ms/vs/stable/vs_community.exe
          https://aka.ms/vs/stable/vs_professional.exe
          https://aka.ms/vs/stable/vs_enterprise.exe
          https://aka.ms/vs/stable/vs_BuildTools.exe     <- CI 构建机用这个
预览通道：https://aka.ms/vs/insiders/vs_community.exe
```

注意 **Community 版只能使用稳定（Stable）通道**。想要 Insiders，需要 Professional 或 Enterprise，或使用 Insiders 版的 Build Tools。

3.2.2 命令行安装（适合批量部署）

安装器本身支持大量命令行参数，做自动化装机时很有用：

```
:: 示例：静默安装 Enterprise，指定稳定通道
vs_enterprise.exe --quiet --wait --norestart ^
  --channelId VisualStudio.18.Stable ^
  --productId Microsoft.VisualStudio.Product.Enterprise ^
  --add Microsoft.VisualStudio.Workload.NativeDesktop ^
  --includeRecommended
```

关键参数含义：

参数	说明
`--channelId`	通道 ID：`VisualStudio.18.Stable` 或 `VisualStudio.18.Insiders`
`--productId`	产品 ID，如 `Microsoft.VisualStudio.Product.Enterprise`
`--add <工作负荷或组件 ID>`	添加工作负荷/组件
`--includeRecommended`	连同该工作负荷的推荐组件一起装
`--quiet --wait --norestart`	静默、等待完成、不自动重启
`--installPath`	自定义安装目录

3.3 选择工作负荷：这一步决定你以后能不能干活

安装器里最关键的界面是”**工作负荷**”选项卡。勾错了，后面会遇到”为什么我没有 CMake””为什么我找不到 MFC”这类问题。

3.3.1 必选：使用 C++ 的桌面开发

”使用 C++ 的桌面开发”（Desktop development with C++）是写 Windows C++ 的唯一必选项。它会带来：

- MSVC 编译器与链接器（默认工具集）
- Windows SDK
- C++ 标准库与运行库
- CMake（VS 2026 自带 **CMake 4.1.1**）
- vcpkg 包管理器
- C++ 调试器与性能探查器
- Git（可选组件）

3.3.2 可选工作负荷

工作负荷	什么时候需要
使用 C++ 的桌面开发	[OK] 必选
使用 C++ 的游戏开发	做 Unreal Engine / DirectX 游戏开发
使用 C++ 的 Linux、Mac 和嵌入式开发	远程 Linux 开发、WSL、CMake 交叉编译（注意：此工作负荷原名”C++ Linux”，VS 2026 已改名）
使用 C++ 的移动开发	[X] **已不再受支持**，未来版本将移除，不要选
.NET 桌面开发	只有写 C# 时才需要
使用 C++ 的桌面开发（含 ATL/MFC）	维护老式 Windows 桌面程序

注意 使用 C++ 的移动开发（iOS/Android）以及嵌入式/IoT 工具（RTOS 查看器、串行监视器、外设查看器）在 VS 2026 中已不再受支持。如果你的项目依赖它们，请继续使用 VS 2022。

3.3.3 值得单独勾选的”单个组件”

切到”**单个组件**”选项卡，按需补充：

组件	用途
MSVC v145 生成工具 (x64/x86)	VS 2026 的默认工具集
MSVC v143 生成工具	仍要编译 VS 2022 时代的项目
Windows 11 SDK	Windows 桌面开发的标准选择
适用于 Windows 的 C++ CMake 工具	CMake 项目支持（通常已随工作负荷安装）

组件	用途
vcpkg 包管理器	依赖管理
Git for Windows	版本控制（也可用系统已装的 Git）
C++ AddressSanitizer	内存安全检测（详见第 9 章）
用于 Windows 的 C++ Clang 工具	需要 clang-cl / clang-tidy 时
测试工具（Google Test / Microsoft 原生测试框架）	单元测试（详见第 11 章）

提示 拿不准就先只勾”使用 C++ 的桌面开发 + 推荐组件”。Visual Studio 的安装器**随时可以再打开补装组件**，不必一次到位——但补装也要下载几个 GB，能一次勾好更省事。

3.4 安装位置与磁盘规划

安装器默认把 IDE 装到系统盘：

```
C:\Program Files\Microsoft Visual Studio\18\<版本>\
```

C 盘紧张的话，可以在安装器的”**安装位置**”选项卡里改三处。三处的默认位置分别是：

```
Visual Studio IDE:      C:\Program Files\Microsoft Visual Studio\18\...
下载缓存:               C:\ProgramData\Microsoft\VisualStudio\Packages
共享组件、工具和 SDK:   C:\Program Files (x86)\Microsoft Visual Studio\Shared
```

项目	说明
Visual Studio IDE	建议保留在系统盘（体积相对可控）
下载缓存	**最占空间**，可移到其他盘，装完可清理
共享组件、工具和 SDK	多个 VS 版本共用

实战场景 装完之后如果 C 盘少了几十 GB，去”**下载缓存**”目录看看。安装器里也有”**清理**”相关选项可以回收这部分空间。

3.5 首次启动

3.5.1 登录与账户

首次启动会要求登录 Microsoft 账户。登录的好处是**设置同步**（主题、快捷键、字体等跟着账户走）。不想登录可以点”以后再说”，不影响使用，**也不影响 C++ 编译**。

3.5.2 选择开发设置

安装器会问”**选择你的开发设置**”。这里的选项只影响**快捷键映射与窗口布局预设**，不改变编译器行为：

选项	适合谁
Visual C++	本教程读者的默认选择：界面最精简，快捷键贴合 C++ 开发
常规	什么语言都写
Visual Studio Code	从 VS Code 迁过来，想保留熟悉的按键
Visual Basic / C#	写.NET 的人

提示 选错了也没关系，之后可在 **工具 > 导入和导出设置**里重选。

3.5.3 挑一套主题

VS 2026 提供了**全新的界面与 11 套内置主题**。选一套顺眼的即可。更进一步，还可以在 **工具 > 选项 > 环境 > 视觉体验 > 主题颜色**里直接改界面颜色令牌，不用装任何扩展。

3.6 验证安装：确认工具链真的可用

装完别急着写代码，先花两分钟验证编译器就位。

3.6.1 从”开发者命令提示符”验证

在开始菜单搜索 **Developer Command Prompt for VS 2026**（或 **x64 Native Tools Command Prompt for VS 2026**）打开，执行：

```
cl.exe
```

正常输出应类似：

```
Microsoft (R) C/C++ Optimizing Compiler Version 19.50.xxxxx for x64
Copyright (C) Microsoft Corporation.  All rights reserved.

usage: cl [ option... ] filename... [ /link linkoption... ]
```

看到 **19.50** 说明用的是 MSVC 14.50（`_MSC_VER` 1950）。如果显示 19.51，说明当前默认工具集是 14.51。

3.6.2 验证 CMake 与 vcpkg

```
cmake --version
vcpkg version
```

CMake 版本应为 **4.1.1** 左右（VS 2026 自带）。vcpkg 若提示找不到命令，说明该组件没勾选，可在安装器里补装。

3.6.3 用 IDE 跑一个最小程序

按 `Ctrl+Shift+N` 新建项目 -> 选 **控制台应用**（C++）-> 起名 -> 创建 -> `Ctrl+F5` 运行。看到黑窗口输出，安装就算彻底成功。详细步骤见 **第 4 章**。

3.7 工具集版本管理

3.7.1 三种工具集并存

VS 2026 的安装器里可以同时提供多个 MSVC 版本：

类型	说明	什么时候用
预览工具集	最新的特性与修复，约每周更新（Insiders 更频繁）	提前验证新特性，**不要用于生产**
默认工具集	当前推荐的稳定版本	日常开发
较早但仍受支持的工具集	例如 14.50（LTS）	需要可复现构建、与团队对齐

工具集实际落在：

```
C:\Program Files\Microsoft Visual Studio\18\<版本>\VC\Tools\MSVC\
+-- 14.50.35717\     <- MSVC 14.50（长期支持）
+-- 14.51.36231\     <- MSVC 14.51
```

3.7.2 安装预览工具集

在安装器的”**单个组件**”选项卡里，找到并勾选 **MSVC 生成工具（预览）**（英文界面为 MSVC Build Tools for <arch> (Preview)）。

3.7.3 安装旧工具集 / 在升级后恢复旧工具集

同样在”**单个组件**”里勾选对应版本。典型场景：工具集从 14.50 升级到 14.51 后，你想回到 14.50 ——从”单个组件”里把它**加回来**即可。

3.7.4 锁定工具集版本

团队协作或需要可复现构建时，锁定版本：

```
:: 命令行构建时指定
msbuild MyApp.sln /p:Configuration=Release /p:VCToolsVersion=14.50.35717

:: 在开发者命令提示符里切换工具集
```

```
vcvars64.bat -vcvars_ver=14.50
vcvars64.bat -vcvars_ver=preview
```

或者在项目文件 / Directory.Build.props 里固定：

```
<PropertyGroup>
  <VCToolsVersion>14.50.35717</VCToolsVersion>
</PropertyGroup>
```

如果被锁定的工具集装在**另一个 Visual Studio 安装实例**里，从 **18.8** 起可以打开跨实例发现：

```
<PropertyGroup>
  <EnableVCToolsVersionDiscovery>true</EnableVCToolsVersionDiscovery>
  <VCToolsVersion>14.50.35717</VCToolsVersion>
</PropertyGroup>
```

注意 MSVC 14.50 与 14.51 **保持与 VS 2015 及以后版本的二进制兼容性**，所以不同工具集编出的库通常能互相链接。但**优化行为与警告可能有差异**，正式发布前请用固定工具集重新构建一次。

3.8 更新、修复与卸载

3.8.1 更新

- 打开 **Visual Studio 安装程序**（开始菜单里独立存在），点”**更新**”。
- VS 2026 的更新是**就地更新**，不需要卸载重装。
- **Community 版必须在 180 天内更新**到较新版本，否则会提示并要求更新。

3.8.2 修复

IDE 行为异常、组件莫名缺失时，在安装程序里点 **更多 > 修复**。这一步会重新校验并补回缺失文件，是解决”玄学问题”的常用手段。

3.8.3 修改（补装/卸载组件）

点 **修改**回到工作负荷与组件界面。这是**最常用**的按钮——装完才发现少勾了组件，走这里。

3.8.4 卸载

在安装程序里卸载。**注意**：卸载 VS 2026 不会影响 VS 2022，反之亦然。

3.9 常见安装问题

现象	原因	处置
安装程序卡在” 正在准备” 很久	下载缓存写盘慢 / 网络问题	换网络，或把下载缓存移到 SSD
提示需要重启但重启后仍提示	有挂起的文件操作	手动重启一次，再点” 继续”
装完找不到 `cl.exe`	只装了 IDE 没装 C++ 工作负荷	安装程序 > 修改 > 勾” 使用 C++ 的桌面开发”
`cl.exe` 版本与预期不符	默认工具集是别的版本	用 `vcvars64.bat -vcvars_ver=14.50` 切换，或补装对应工具集
无法安装 Insiders	Community 版限制	换 Professional/Enterprise，或用 Insiders Build Tools
磁盘空间不足	下载缓存 + 共享组件占用大	移动缓存目录，或用安装程序的清理功能

3.10 小结

- VS 2026 **主机必须是 Windows 11**（或 Server 2019/2022/2025）；目标程序最低可跑在 Windows 10 / Server 2016。
- 安装时**必选” 使用 C++ 的桌面开发”**；” 使用 C++ 的移动开发” 已不受支持，不要选。
- 安装位置三处可改，**下载缓存最占空间**。
- 验证安装最快的办法：开发者命令提示符里跑 `cl.exe`，看版本是否为 **19.50 / 19.51**。
- 工具集可在安装器里**并存多个版本**，并用 `VCToolsVersion` 锁定以保证可复现构建。
- Community 版**只能用稳定通道**，且需在 **180 天内**更新。

3.11 练习题

1. 用 `winget` 或官网引导程序安装 VS 2026 Community，工作负荷只勾” 使用 C++ 的桌面开发”。
2. 在开发者命令提示符里执行 `cl.exe`，记录版本号；再用 `vcvars64.bat -vcvars_ver=14.50` 切换后重新执行，对比两次输出。
3. 打开 Visual Studio 安装程序，把” 下载缓存” 位置改到非系统盘，并观察 C 盘空间变化。
4. 在安装器” 单个组件” 里找出**预览工具集**与**旧版工具集**的条目，说明它们的区别。
5. 写一个程序打印 `_MSC_VER`，分别在默认工具集与固定 `VCToolsVersion` 下编译，验证版本锁定是否生效。
6. 检查你的机器是否满足全部系统要求，列出不满足的项并给出替代方案。

3.12 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **上一章：VS 2026 概览与版本选择**
- **下一章：界面与工作区**
- **CMake：CMake 使用指南**
- **VS Code 对照：VS Code C++ 配置指南**

第四章　界面与工作区

Visual Studio 的界面元素比 VS Code 多得多，第一次打开容易“信息过载”。但只要抓住一条主线就清楚了：**左边管项目、中间写代码、下面看反馈、右边看属性**。这一章把窗口体系、布局管理和快捷键一次性讲清楚，让你能把它调教成顺手的形状。

4.1　起始窗口

打开 VS 2026 首先看到的是**起始窗口**，它做四件事：

区域	作用
克隆存储库 / 打开项目或解决方案 / 打开本地文件夹	三种入口：Git 仓库、`.sln`/`.vcxproj`、任意文件夹（CMake 项目走这里）
创建新项目	打开模板列表
最近使用的项目	快速回到上次的工作
继续但无需代码	不建项目直接开一个空编辑器

> **提示** CMake 项目必须用“**打开本地文件夹**”，不能用“打开项目或解决方案”。这是新手最常见的困惑之一，详见**第 10 章**。

起始窗口可以在 **工具 > 选项 > 环境 > 启动**里改成“打开时显示空环境”，省掉这一步。

4.2　主界面布局总览

```
+-------------------------------------------------------------------
| 菜单栏：文件 编辑 视图 项目 生成 调试 测试 分析 工具 扩展 窗口 帮助
| 工具栏：[保存] [撤销] [重做] | [Debug v] [x64 v] | [> 启动] [工具集]
+-------------------------------------------------------------------
| 左：解决方案资源管理器 (Ctrl+Alt+L)
|   解决方案
|     > 项目
|         main.cpp
|         utils.h
+-------------------------------------------------------------------
| 中：文档选项卡 [main.cpp] [utils.h]
```

```
|   代码编辑器
|     1  #include <iostream>
|     2
|     3  int main() {
|     4      std::cout << ...;
|     5  }
+-----------------------------------------------------------------
| 右: 属性窗口 (F4) / 类视图 (Ctrl+Shift+C) / 对象浏览器
+-----------------------------------------------------------------
| 下: 错误列表 | 输出 | 任务列表 | 终端 | Git 更改   <- 底部停靠区
+-----------------------------------------------------------------
```

四块区域的分工：

区域	主要窗口	你在这里做什么
左	解决方案资源管理器	管理文件、项目、引用
中	代码编辑器	写代码
右	属性 / 类视图 / 对象浏览器	查看与修改属性、浏览类型结构
下	错误列表 / 输出 / 终端 / Git 更改	看编译结果、程序输出、提交代码

4.3 解决方案资源管理器

打开方式：`Ctrl+Alt+L`，或 **视图 > 解决方案资源管理器**。

它是 VS 的“文件中枢”，能做的事比看起来多：

操作	入口
添加新建/现有项	右键项目 > **添加 > 新建项 / 现有项**
添加类	右键项目 > **添加 > 类**
添加引用（项目引用）	右键项目 > **添加 > 引用**
管理 NuGet 包	右键项目 > **管理 NuGet 包**
查看项目属性	右键项目 > **属性**
设为启动项目	右键项目 > **设为启动项目**
在文件资源管理器中打开	右键 > **在文件资源管理器中打开文件夹**
生成/重新生成/清理	右键项目或解决方案 > **生成 / 重新生成 / 清理**
运行 Build Insights	右键选中文件 > **对选定文件运行 Build Insights**
生成预处理输出	右键 `.cpp` 文件 > **预处理**（详见**第 7 章**）

工具栏上的几个开关（资源管理器顶部）：

- **同步显示当前文件**：编辑器切到哪个文件，资源管理器自动定位——强烈建议打开。
- **显示所有文件**：把磁盘上存在但未加入项目的文件也显示出来（显示为虚影图标）。

- **预览选定项**：单击文件时在右侧预览，不占用文档选项卡。

> **注意** `.vcxproj.filters` 里的“筛选器”只是 **IDE 里的显示分组**，与磁盘目录**不是一回事**。物理上文件可能都在同一目录，而筛选器里看起来分了多层文件夹。真正决定“哪些文件参与编译”的是 `.vcxproj` 里的条目，不是筛选器。详见**第 5 章**。

4.4 代码编辑器

编辑器的核心能力（快捷键、IntelliSense、重构、格式化）在**第 7 章**展开，这里只说界面层面的东西。

4.4.1 文档选项卡

- **固定选项卡**：右键选项卡 > **固定**，常用文件不会被新文件挤走。
- **选项卡分组**：把选项卡拖到编辑器另一侧可形成新的组，便于对照两份代码。
- **选项卡颜色**：右键选项卡可设置颜色，用于标记“正在改的文件”。
- **新建垂直选项卡组**：**窗口 > 新建垂直选项卡组**，左右对照。

4.4.2 编辑器左下角与右下角

- **左下角**显示当前光标位置、是否处于插入/改写模式。
- **右下角**在 VS 2026 里做了增强：可以**查看并修改当前文件的编码**。中文乱码排查时这里是第一站。

4.4.3 滚动条上的信息

垂直滚动条不只是滚动条，它还会标注：**修改过的位置**、**断点**、**错误**、**警告**、**搜索结果**。大文件里靠它快速定位问题。

4.5 工具窗口体系

VS 的工具窗口很多，不必全记住。下面按使用频率分组。

4.5.1 高频窗口

窗口	打开方式	用途
解决方案资源管理器	`Ctrl+Alt+L`	文件与项目管理
错误列表	`Ctrl+\, Ctrl+E`	编译错误与警告（双击跳转）
输出	`Ctrl+Alt+O`	生成输出、调试输出、`OutputDebugString`

窗口	打开方式	用途
属性	F4	查看/修改选中对象（项目、文件、控件）的属性
类视图	Ctrl+Shift+C	按类型而非文件浏览代码
对象浏览器	**视图 > 对象浏览器**	浏览程序集/库中公开的类型
终端	**视图 > 终端**	内置命令行
Git 更改	**视图 > Git 更改**	提交、暂存、查看差异
测试资源管理器	**测试 > 测试资源管理器**	运行与管理单元测试
任务列表	**视图 > 任务列表**	汇总 // TODO: 之类的注释

4.5.2 调试期才出现的窗口

断点、调用堆栈、监视、自动、局部变量、即时、线程、并行堆栈、内存、反汇编、寄存器、诊断工具——这些在**第 8 章**详细讲。

4.5.3 查找窗口的两种形态

- **停靠形态**：默认贴在编辑器右上角，适合小范围查找。
- **浮动形态**：点查找框上的“浮动”按钮，变成独立对话框，适合需要频繁调整选项时。

4.6 窗口布局管理

4.6.1 停靠与浮动

把工具窗口的标题栏**拖到屏幕边缘**会出现停靠提示菱形，松手即停靠；拖到屏幕中间则变为**浮动窗口**。

操作	效果
双击标题栏	在“停靠”与“浮动”之间切换
拖到编辑器中间	形成**选项卡式文档组**
右键标题栏 > **自动隐藏**	收起为侧边标签，鼠标经过才展开
右键标题栏 > **停靠为选项卡式文档**	让工具窗口像代码文件一样占一个选项卡

4.6.2 多显示器

把右侧的“属性/类视图”拖到第二块屏幕，主屏就只剩下代码——这是双屏下最实用的布局。

4.6.3 保存与切换布局

调好布局后一定要保存：

- **窗口 > 保存窗口布局**：给当前布局起名保存。

- **窗口 > 应用窗口布局**：随时切回某个布局。
- 典型用法：保存一套“**编码布局**”（只有编辑器和资源管理器）和一套“**调试布局**”（堆栈、监视、局部变量全开），按场景一键切换。

实战场景 调试时窗口乱成一团是常态。调好一次后存成“调试布局”，下次直接切回来，省掉每次重新拖窗口的几分钟。

4.7 主题、字体与视觉体验

4.7.1 主题

VS 2026 提供**全新的界面与 11 套内置主题**：**工具 > 选项 > 环境 > 常规 > 颜色主题**。

4.7.2 自定义主题颜色

不想装第三方主题扩展，可以直接改颜色：

工具 > 选项 > 环境 > 视觉体验 > 主题颜色

在这里可以修改任意 Fluent 颜色令牌（强调色、悬停状态、外壳框架等），**改动按主题分别保存**，也可以单独重置某一项。

4.7.3 字体与颜色

工具 > 选项 > 环境 > 字体和颜色是更细粒度的控制：字体、字号、以及**每一类语法元素**的前景色（关键字、注释、字符串、类型名……）。

提示 C++ 开发者建议把字体换成等宽且区分 `0/O`、`1/l/I` 的字体（如 Cascadia Code、JetBrains Mono），能省下不少看错标识符的时间。

4.8 快捷键体系与自定义

4.8.1 为什么值得花十分钟学

VS 的快捷键是**两段式**的，这是它和 VS Code 最大的差异：

```
Ctrl+K, Ctrl+C     <- 先按 Ctrl+K，松开，再按 Ctrl+C
Ctrl+K, Ctrl+D     <- 格式化文档
Ctrl+R, Ctrl+R     <- 重命名
Ctrl+\, Ctrl+E     <- 错误列表
```

习惯之后效率很高，但刚开始会觉得别扭。

4.8.2 必背的一批

快捷键	功能
Ctrl+Shift+N	新建项目
Ctrl+Shift+O	打开项目/解决方案
Ctrl+Shift+S	全部保存
Ctrl+F5	生成并运行（不调试）
F5	生成并启动调试
Ctrl+Shift+B	生成解决方案
F7	转到代码视图
Ctrl+K, Ctrl+D	格式化文档
Ctrl+K, Ctrl+C	注释选中行
Ctrl+K, Ctrl+U	取消注释
Ctrl+Q	快速启动（搜索命令、设置、文件）
Ctrl+T	转到所有（文件/类型/成员/符号）
Ctrl+Alt+L	解决方案资源管理器
Ctrl+\, Ctrl+E	错误列表
Ctrl+Alt+O	输出窗口
F4	属性窗口
Shift+Alt+Enter	全屏切换
Ctrl+M, Ctrl+O	折叠到定义
Ctrl+M, Ctrl+M	折叠/展开当前区域

更多编辑与导航类快捷键见**第 7 章**，调试类见**第 8 章**，完整速查表见**第 13 章**。

4.8.3 自定义快捷键

工具 > 选项 > 环境 > 键盘：

1. 在搜索框输入命令名（如 `Edit.FormatDocument`）；
2. 选中命令，把光标放进“按快捷键”框；
3. 按下你想要的组合；
4. 看“快捷键的当前使用对象”是否冲突，点“**分配**”。

4.8.4 键盘映射方案

如果你从别的编辑器过来，可以直接换整套方案：**工具 > 选项 > 环境 > 键盘 > 应用以下其他键盘映射方案**，可选 **Visual Studio Code**、ReSharper、Visual C++ 6 等。

提示 VS 2026 这一代特意让一批快捷键**向其他编辑器看齐**，从 VS Code 迁过来的人上手会比过去顺很多。

4.8.5 导出与导入设置

工具 > 导入和导出设置可以把快捷键、字体颜色、窗口布局、代码片段等打包导出，换机器时一键恢复。

4.9 快速启动：Ctrl+Q

Ctrl+Q 是 VS 里最被低估的功能。它一个框搜遍**命令**、**设置项**、**已打开的文件**、**最近文件**、**模板**：

```
按下 Ctrl+Q，输入：
  "字体"          -> 直接跳到字体和颜色设置
  "clang"         -> 跳到 clang-tidy 相关设置
  "附加到进程"     -> 直接执行该命令
  "编码"          -> 跳到文件编码相关选项
```

忘了某个功能藏在哪个菜单里，Ctrl+Q 比翻菜单快得多。

4.10 设置与选项

4.10.1 选项对话框

工具 > 选项是 VS 的总设置中心，常用页面：

路径	用途
环境 > 常规	主题、启动行为
环境 > 字体和颜色	编辑器字体与语法着色
环境 > 视觉体验 > 主题颜色	自定义 Fluent 颜色令牌
环境 > 键盘	快捷键
环境 > 导入和导出设置	设置迁移
文本编辑器 > C/C++	C++ 编辑行为（含 IntelliSense）
项目和解决方案	生成并运行行为、并行生成数
调试	调试器行为
源代码管理 > Git	Git 集成行为
GitHub > Copilot	AI 功能的开关与设置

4.10.2 项目级设置 vs 全局设置

这一点必须分清：

层级	存放位置	影响范围
全局	**工具 > 选项**	你机器上的所有项目
项目级	**项目 > 属性**	只有当前项目的当前配置

层级	存放位置	影响范围
团队级	`.editorconfig`、`Directory.Build.props`	提交到仓库，对所有人生效

注意 **编译相关的配置（语言标准、运行库、优化）在“项目 > 属性”里，不在“工具 > 选项”里**。新手常犯的错误就是在选项里翻半天找不到 `/std:c++20` 在哪设。见**第 6 章**。

4.10.3 设置同步

登录 Microsoft 账户后，可在 **工具 > 选项 > 环境 > 账户**里开启设置同步，把主题、快捷键、字体等带到其他机器。

4.11 让界面顺手的三条建议

1. **关掉不用的窗口**。C++ 开发者很少需要“服务器资源管理器”“数据工具”之类。**视图**菜单里取消勾选即可。
2. **保存两套布局**（编码 / 调试），用 **窗口 > 应用窗口布局**切换。
3. **把 `Ctrl+Q` 当搜索引擎用**，而不是去背菜单结构。

4.12 小结

- 界面主线：**左边项目、中间代码、下面反馈、右边属性**。
- 起始窗口的三种入口要分清：**打开项目/解决方案**（`.sln`/`.vcxproj`）、**打开本地文件夹**（CMake）、**克隆存储库**（Git）。
- **筛选器** $\neq$ **磁盘目录**；真正决定编译内容的是 `.vcxproj`。
- **窗口 > 保存窗口布局**是提升效率最快的一步，建议存“编码”与“调试”两套。
- VS 快捷键是**两段式**的（`Ctrl+K, Ctrl+C`），并可在 **工具 > 选项 > 环境 > 键盘**里换成 VS Code 方案。
- `Ctrl+Q` **快速启动**能搜命令、设置和文件，比翻菜单快。
- **编译配置在“项目 > 属性”里，不在“工具 > 选项”里**。

4.13 练习题

1. 保存一套只留“解决方案资源管理器 + 编辑器 + 错误列表”的编码布局，再保存一套含“调用堆栈 + 监视 + 局部变量”的调试布局，并在两者间切换。
2. 用 `Ctrl+Q` 分别找到：字体和颜色设置、clang-tidy 设置、“附加到进程”命令。
3. 把编辑器字体换成 Cascadia Code，并调整注释与字符串的颜色。

4. 在 **工具 > 选项 > 环境 > 视觉体验 > 主题颜色**里改一个颜色令牌，再单独重置它。
5. 把键盘映射方案临时切到 Visual Studio Code，按 `Ctrl+Shift+P` 试试，然后切回 Visual C++。
6. 用 **工具 > 导入和导出设置**导出一份设置文件，看看它包含哪些类别。

4.14 相关笔记

- **本教程**：Visual Studio 2026 使用教程
- **上一章**：安装与环境配置
- **下一章**：第一个 C++ 项目
- **编辑与 IntelliSense**：代码编辑与 IntelliSense
- **效率技巧**：常见问题与效率技巧

第五章　第一个 C++ 项目

前面三章都是准备工作。这一章真正开始写代码：建一个控制台项目，看懂 VS 在磁盘上生成了什么，弄明白 F5 和 Ctrl+F5 的区别，最后学会脱离 IDE 用命令行构建——这一步在接入 CI 时会用到。

5.1　创建控制台应用

5.1.1　走一遍流程

1. 打开 VS 2026 -> 起始窗口点 **创建新项目**（或按 Ctrl+Shift+N）。
2. 在模板列表里搜索 控制台，选择 **控制台应用**（语言筛选 **C++**，平台 **Windows**）。
3. 点 **下一步**，填写：

字段	建议值	说明
项目名称	HelloVS	会同时作为默认命名空间名与 .exe 名
位置	D:\cpp	**不要放在中文路径或含空格的路径下**（见本章第 8 节）
解决方案名称	HelloVS	可以留空，默认与项目同名
将解决方案和项目放在同一目录中	建议勾选	目录结构更扁平，便于理解

4. 点 **创建**。

5.1.2　模板选哪个

模板	生成什么	适合
控制台应用	一个 .cpp，含 main 与 ##include <iostream>	[OK] 学习、算法练习、命令行工具
空项目	什么都没有，需要自己添加文件	想完全从零开始
Windows 桌面应用程序	Win32 窗口程序骨架	图形界面入门
静态库 / 动态库	库项目骨架	写库给别人用（见**第 5 章**）

模板	生成什么	适合

提示 VS 2026 的**新建** C++ 控制台项目默认使用 **C++20** 标准（项目模板会写入 `/std:c++20`）。这和“编译器在没指定 `/std` 时默认 C++14”并不矛盾——见**第 1 章 4.1 节**。

5.2 项目文件结构解析

5.2.1 生成后磁盘上有什么

假设项目名 `HelloVS`，位置 `D:\cpp`，勾选了“同一目录”，生成后的目录大致是：

```
D:\cpp\HelloVS\
+-- HelloVS.sln              <- 解决方案文件（文本格式，记录包含哪些项目）
+-- HelloVS.vcxproj          <- MSBuild 项目文件（真正的构建定义，XML）
+-- HelloVS.vcxproj.filters  <- 筛选器：只影响 IDE 里的显示分组
+-- HelloVS.vcxproj.user     <- 你本机的个人设置（不提交到 Git）
+-- HelloVS.cpp              <- 源文件
+-- .vs\                     <- IDE 的隐藏缓存目录（不提交到 Git）
    +-- ...
```

编译一次之后，还会多出：

```
D:\cpp\HelloVS\
+-- x64\
    +-- Debug\
        +-- HelloVS.exe          <- 生成的可执行文件
        +-- HelloVS.pdb          <- 调试符号
        +-- HelloVS.obj          <- 编译产生的目标文件
```

注意 `.vs` 是**隐藏目录**，在文件资源管理器里默认看不到。它保存了 IDE 的临时状态；遇到“IntelliSense 发疯”“项目加载异常”时，**关掉 VS、删掉 `.vs`、重新打开**是最常用的急救手段（详见**第 13 章**）。

5.2.2 每个文件干什么

文件	是否提交到 Git	作用
`HelloVS.sln`	[OK]	解决方案：记录包含哪些项目、启动项目等
`HelloVS.vcxproj`	[OK]	**核心**：编译器开关、包含目录、链接库、文件清单都在这

文件	是否提交到 Git	作用
HelloVS.vcxproj.filters	[OK]	IDE 里的筛选器分组（不影响编译）
HelloVS.vcxproj.user	[X]	本机调试设置（如命令行参数、工作目录）
.vs	[X]	IDE 缓存
x64、Debug、Release	[X]	中间产物与输出
HelloVS.cpp	[OK]	你的代码

对应的 .gitignore 片段：

```
# Visual Studio
.vs/
x64/
Debug/
Release/
*.user
*.aps
*.ipch
*.opendb
*.db
```

实战场景 .vcxproj.user 里存着“调试时用什么命令行参数、工作目录是哪里”。这些是**本机**设置，同事的机器上路径可能完全不同，所以不该提交。项目级的公共调试参数应该写进 .vcxproj 的调试属性页。

5.2.3 在 IDE 里看结构

解决方案资源管理器显示的是“逻辑结构”：

```
解决方案 'HelloVS' (1 个项目)
+-- HelloVS
    +-- 头文件              <- 筛选器（可能是空的）
    +-- 源文件
    |   +-- HelloVS.cpp
    +-- 资源文件            <- 筛选器（可能是空的）
    +-- 引用                <- 引用的其他项目/库
```

注意**“头文件”“源文件”“资源文件”是筛选器**，不是磁盘目录。哪怕它们显示为空，磁盘上也没有对应的空文件夹。

5.3 读懂生成的代码

```cpp
#include <iostream>

int main()
{
    std::cout << "Hello World!\n";
    return 0;
}
```

逐行看：

代码	含义
##include <iostream>	引入标准输入输出库，std::cout 需要它
int main()	程序入口。Windows 控制台程序的入口就是 main（或宽字符版 wmain）
std::cout << ...	向标准输出写字符串；std:: 是命名空间前缀
"\n"	换行符。比 std::endl 更轻量（不强制刷新缓冲区）
return 0;	返回 0 表示正常结束。main 里其实可以省略，但写上更清晰

提示 这里的 std:: 前缀别嫌烦。虽然可以写 using namespace std;，但在头文件里绝对不要这么干，否则会把命名空间污染传给所有包含者。

5.4 生成与运行

5.4.1 Ctrl+F5 与 F5 的区别

这是新手第一个必须搞清的差别：

快捷键	行为	什么时候用
Ctrl+F5	**开始执行（不调试）**：编译并运行，程序结束后**停在“按任意键继续”**	只想看运行结果
F5	**开始调试**：编译并附加调试器运行，程序结束窗口**直接关闭**	要打断点排查问题
Ctrl+Shift+B	只生成，不运行	检查能否编译通过

注意 用 F5 跑一个“打印完就退出”的程序，黑窗口会一闪而过——因为调试器接管了生命周期，不会自动暂停。想看输出就用 Ctrl+F5，或者在 main 的 return 前打断点。

5.4.2 生成输出：Ctrl+Alt+O

输出窗口（Ctrl+Alt+O）里选“**生成**”，能看到完整的编译过程：

```
1>------ 已启动生成: 项目: HelloVS, 配置: Debug x64 ------
1>HelloVS.cpp
1>HelloVS.vcxproj -> D:\cpp\HelloVS\x64\Debug\HelloVS.exe
========== 生成: 成功 1 个, 失败 0 个, 最新 0 个, 跳过 0 个 ==========
```

关键信息在最后一行。-> 后面就是**可执行文件的完整路径**。

5.4.3 错误去哪看：Ctrl+\，Ctrl+E

编译失败时看**错误列表**（Ctrl+\，Ctrl+E）：

列	含义
代码	如 C2143、LNK2019（错误码速查见**第 13 章**）
说明	错误描述
项目 / 文件 / 行	**双击直接跳到出错位置**

提示 错误列表支持筛选（只看错误 / 只看警告），也支持按“生成 + IntelliSense”分类。**以“生成”分类为准**——IntelliSense 的红色波浪线有时是误报。

5.4.4 输出窗口 vs 程序输出

两者完全不同，别搞混：

```
输出窗口（Ctrl+Alt+O）   ->  编译器/链接器/调试器说的话
程序的黑窗口             ->  你的 std::cout 输出
```

调试时想让信息出现在**输出窗口**，可以用：

```
1 #include <windows.h>
2
3 OutputDebugStringA("这条信息会出现在 VS 的输出窗口\n");
```

这在写没有控制台的 Windows 程序时特别有用。

5.5 命令行构建

脱离 IDE 构建是接入 CI 的第一步。有两种方式。

5.5.1 用 MSBuild（推荐）

先在开始菜单打开 **Developer Command Prompt for VS 2026**（这个快捷方式会帮你配好环境变量），然后：

```
:: 在解决方案目录下
msbuild HelloVS.sln /p:Configuration=Debug /p:Platform=x64
```

```
:: 生成 Release 版本
msbuild HelloVS.sln /p:Configuration=Release /p:Platform=x64

:: 重新生成（先清理再编译）
msbuild HelloVS.sln /t:Rebuild /p:Configuration=Release /p:Platform=x64

:: 清理
msbuild HelloVS.sln /t:Clean
```

常用参数：

参数	作用
/p:Configuration=Debug|Release	指定配置
/p:Platform=x64|Win32|ARM64	指定平台（注意 32 位在 MSBuild 里叫 **Win32**，不是 x86）
/t:Build|Rebuild|Clean	指定目标
/m	多进程并行构建
/v:minimal|normal|detailed	输出详细程度
/p:VCToolsVersion=14.50.35717	锁定工具集版本
/bl	生成 msbuild.binlog，可用结构化日志查看器分析

注意 **必须用“开发者命令提示符”**，不能用普通的 cmd。因为 msbuild 依赖一堆环境变量（PATH、INCLUDE、LIB 等），普通命令行里没配，会报“找不到 windows.h”之类的错。想自己配的话，先执行 vcvars64.bat。

5.5.2 用 devenv（IDE 的命令行外壳）

```
devenv HelloVS.sln /Build "Release|x64"
devenv HelloVS.sln /Rebuild "Debug|x64"
devenv HelloVS.sln /Clean
```

devenv 会启动 IDE 的构建引擎。适合需要与 IDE 行为完全一致的场景，但**比 MSBuild 慢**，CI 里一般用 MSBuild。

5.5.3 验证生成结果

```
:: 确认 exe 存在
dir x64\Release\HelloVS.exe

:: 直接运行
x64\Release\HelloVS.exe
```

5.6 配置与平台：Debug/x64 是什么意思

工具栏上那两个下拉框（如 Debug + x64）合起来叫一个**配置**：

```
配置 = 配置(Configuration) x 平台(Platform)
       +-- Debug   : 不优化、带完整调试信息 -> 开发时用
       +-- Release : 开优化、面向发布      -> 发布时用
       +-- 平台
           +-- x64   : 64 位（现在的默认选择）
           +-- Win32 : 32 位（对应 x86，MSBuild 里就叫 Win32）
           +-- ARM64 : ARM 64 位
```

组合	输出目录
Debug + x64	x64\Debug
Release + x64	x64\Release
Debug + Win32	Debug（32 位时通常不带平台前缀）

Debug 与 Release 的关键差异：

维度	Debug	Release
优化	/Od 关闭	/O2 最大化速度
调试信息	/Zi 完整	通常仍有 /Zi（但优化会让单步错位）
运行库	/MDd（调试版 CRT）	/MD（发行版 CRT）
assert	生效	被 NDEBUG 屏蔽
速度	慢得多	快

注意 **性能测试必须在 Release 下做**。在 Debug 下测出来的耗时可能比 Release 慢十倍以上，据此优化会得出完全错误的结论。详见**第 9 章**。

实战场景 Debug 下能跑、Release 下崩溃，是最经典的一类 bug。常见原因是**未初始化变量**（Debug 下 CRT 会帮忙填 0xCC，Release 下是真正的垃圾值）或**优化暴露的越界访问**。这时该上 AddressSanitizer——见**第 9 章**。

5.7 动手改：加一个头文件和一个源文件

试着把代码拆开，体会多文件项目的工作方式。

mathutils.h（右键项目 > **添加** > **新建项** > **头文件**）：

```
1 #pragma once
2
```

```cpp
namespace mathutils {
    int add(int a, int b);
    int factorial(int n);
}
```

mathutils.cpp（右键项目 > **添加** > **新建项** > **C++ 文件**）：

```cpp
#include "mathutils.h"

namespace mathutils {

int add(int a, int b) {
    return a + b;
}

int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; ++i) {
        result *= i;
    }
    return result;
}

}  // namespace mathutils
```

HelloVS.cpp：

```cpp
#include <iostream>
#include "mathutils.h"

int main()
{
    std::cout << "3 + 4 = " << mathutils::add(3, 4) << '\n';
    std::cout << "5! = " << mathutils::factorial(5) << '\n';
    return 0;
}
```

按 Ctrl+F5，输出：

```
3 + 4 = 7
5! = 120
```

提示 新建的文件会**自动加入项目**（.vcxproj 里多了一条 <ClCompile>）。如果手动往目录里拷文件，IDE 不会自动感知——需要右键项目 > **添加** > **现有项**。

5.8 首次运行常见问题

现象	原因	处置
Ctrl+F5 后黑窗口一闪而过	用的是 F5，或程序崩溃退出	改用 Ctrl+F5；或在 return 前打断点
中文输出乱码	源码编码与控制台代码页不一致	用 /utf-8 编译选项，或设置控制台代码页。详见**第 13 章**
报错“无法打开源文件 iostream”	没装 C++ 工作负荷，或包含路径被改坏	安装器里补装“使用 C++ 的桌面开发”
报错 LNK2019 无法解析的外部符号	函数声明了但没实现，或没把 .cpp 加入项目	检查 .cpp 是否在项目里；见**第 13 章**
路径含中文/空格导致怪问题	工具链对非 ASCII 路径支持不佳	把项目移到纯英文、无空格的路径
提示找不到 msbuild	用了普通 cmd	改用“开发者命令提示符”

5.9 小结

- 创建项目：Ctrl+Shift+N -> **控制台应用**（C++）-> **不要放中文/空格路径**。
- 核心文件是 .vcxproj；.filters 只管 IDE 显示，.user 与 .vs 不该提交。
- Ctrl+F5 **不调试运行（窗口会停住）**，F5 **调试运行（窗口直接关）**。
- 看编译过程用**输出窗口**，看错误用**错误列表**，两者不同。
- 命令行构建用 **MSBuild**，且必须在**开发者命令提示符**里；32 位平台名是 Win32。
- **Debug 与 Release 差异巨大**，性能测试必须在 Release 下做。
- 新建文件会自动加入项目；手工拷进来的文件需要“添加 > 现有项”。

5.10 练习题

1. 创建一个名为 HelloVS 的控制台应用，用 Ctrl+F5 和 F5 各运行一次，说明黑窗口行为为什么不同。
2. 在磁盘上找到生成的 .exe，记录它在 Debug x64 与 Release x64 两种配置下的完整路径。
3. 用开发者命令提示符分别构建 Debug 与 Release，并用 /v:minimal 观察输出差异。
4. 用 msbuild /p:VCToolsVersion=... 锁定工具集版本构建一次，在输出里确认编译器版本。
5. 按第 7 节拆出 mathutils.h / mathutils.cpp，再故意把 mathutils.cpp 从项目里排除，观察报什么错，并说明原因。
6. 在 Release 下写一个未初始化变量的程序，观察它是否“偶发崩溃”，再用 AddressSanitizer 复查一次。

5.11 相关笔记

- **本教程**：Visual Studio 2026 使用教程
- **上一章**：界面与工作区
- **下一章**：项目与解决方案管理
- **多文件项目管理**：多文件项目管理
- **CMake 基础**：CMake 基础

第六章　项目与解决方案管理

上一章点几下鼠标，磁盘上就多出 `.sln`、`.vcxproj`、`.vcxproj.filters`、`.vcxproj.user` 四个文件，而多数人对它们的了解止步于“不要删”。本章把它们拆开，讲清楚**解决方案 ⊃ 项目 ⊃ 文件**这三层关系，再进入真实工程躲不开的部分：多项目拆分、静态库与动态库、配置映射、属性表、版本控制卫生。学完你应该能从零搭出一个三项目解决方案，并知道自己每个操作改了哪个文件的哪一行。

6.1　解决方案、项目、文件：三层关系

```
MyProduct.slnx                <- 解决方案：一次完整的"打开 / 构建 / 调试"容器
+-- App.vcxproj               <- 项目：一套编译链接设置 + 一份文件清单
|   +-- main.cpp              <- 文件：编译器真正读的东西
|   +-- App.vcxproj.filters   <- 只给 IDE 看的分组信息
+-- MathLib.vcxproj           <- 项目：产出 MathLib.lib
+-- Plugin.vcxproj            <- 项目：产出 Plugin.dll + Plugin.lib
```

层次	是什么	谁读它	产出
文件	`.cpp` / `.h` / `.rc` / 资源	编译器 `cl.exe`	`.obj`
项目	一个 MSBuild 项目文件（C++ 是 `.vcxproj`）	MSBuild	一个 `.exe` / `.lib` / `.dll`
解决方案	项目清单 + 配置映射 + IDE 状态	IDE 与 MSBuild	**没有产出**

为什么要分三层：编译器和链接器只认文件，但程序还要记住一堆与文件无关的东西——用哪个 C++ 标准、警告几级、定义哪些宏、链接哪些库、产出是 exe 还是 dll，这些存在**项目**里；一个项目只产出一个东西，而软件由多个东西组成，于是需要**解决方案**来一次打开全部、按依赖顺序一次构建全部、并描述项目之间的关系。

注意 解决方案**本身不产出任何二进制**。`msbuild MyProduct.slnx` 做的事只是“按依赖顺序把里面每个项目分别构建一遍”，所以“生成解决方案失败”从来不是解决方案的错，而是里面某个项目的错——看报错里的项目名。只想构建一个项目时用 **生成 > 生成 App**。

6.2 项目文件的解剖

6.2.1 四个文件各管什么

文件	作用	该不该手改	进 Git 吗
.sln / .slnx	解决方案：项目清单 + 配置映射	官方不建议手改 .sln	[OK]
.vcxproj	MSBuild 项目：编译链接设置 + 文件清单	可以，但优先用属性页	[OK]
.vcxproj.filters	解决方案资源管理器里的**筛选器**（显示分组）	几乎不用	[OK]
.vcxproj.user	个人设置：调试命令行参数、工作目录、环境变量	不手改	[X]
.suo（在 .vs 下）	用户级状态：断点、窗口布局	不手改	[X]

一句话：**.filters 只影响 IDE 显示，.user 和 .suo 只属于你个人**。

6.2.2 .sln：一份“不打算让你手改”的文本

官方文档的原话是：.sln 是一个有自有格式的文本文件，**不打算让人手工编辑**（真实文件用制表符缩进）：

```
Microsoft Visual Studio Solution File, Format Version 12.00
# Visual Studio Version 18
Project("{8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942}") = "App", "src\App\App.vcxproj", "{1C1D2E3F
    -0001-4A5B-8C7D-000000000001}"
EndProject
Global
    GlobalSection(SolutionConfigurationPlatforms) = preSolution
        Debug|x64 = Debug|x64
    EndGlobalSection
EndGlobal
```

- **Format Version 12.00 长期没有变过**，所以它不代表文件新旧；标注 IDE 版本的是第二行注释。
- Project(...) 行里有两个 GUID：**第一个是项目类型 GUID**，8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942 表示 C++ 项目（解决方案文件夹用 2150E333-8FDC-42A3-9474-1A3956D46DE8）；第二个才是该项目自己的 ProjectGuid，项目引用与配置映射全靠它认人。
- 两个分支同时往里加项目，合并冲突很难看：制表符 + GUID + 无序行。

6.2.3 .vcxproj：一份精简但真实的骨架

省掉了 ExtensionSettings、UserMacros 等本教程暂时用不到的段落，其余一字未改（Release|x64 与 Debug|x64 同构，仅取值不同，此处省略）：

```
<?xml version="1.0" encoding="utf-8"?>
<Project DefaultTargets="Build" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <!-- (1) ProjectConfigurations: 本项目支持哪些"配置 + 平台"组合 -->
  <ItemGroup Label="ProjectConfigurations">
    <ProjectConfiguration Include="Debug|x64">
      <Configuration>Debug</Configuration>
      <Platform>x64</Platform>
    </ProjectConfiguration>
  </ItemGroup>
  <!-- (2) Globals: 整个项目只出现一次的全局信息 -->
  <PropertyGroup Label="Globals">
    <VCProjectVersion>18.0</VCProjectVersion>
    <ProjectGuid>{1C1D2E3F-0001-4A5B-8C7D-000000000001}</ProjectGuid>
    <WindowsTargetPlatformVersion>10.0</WindowsTargetPlatformVersion>
  </PropertyGroup>
  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.Default.props" />
  <!-- (3) Configuration: 每种配置各自的"身份" -->
  <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'" Label="Configuration">
    <ConfigurationType>Application</ConfigurationType>
    <UseDebugLibraries>true</UseDebugLibraries>
    <PlatformToolset>v145</PlatformToolset>
  </PropertyGroup>
  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.props" />
  <!-- (4) 属性表在此导入，顺序即优先级（第 7 节） -->
  <ImportGroup Label="PropertySheets" Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
    <Import Project="..\Common.props" />
  </ImportGroup>
  <!-- (5) ItemDefinitionGroup: 编译与链接的具体开关 -->
  <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
    <ClCompile>
      <WarningLevel>Level3</WarningLevel>
      <ConformanceMode>true</ConformanceMode>
      <LanguageStandard>stdcpp20</LanguageStandard>
      <PreprocessorDefinitions>_DEBUG;_CONSOLE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
    </ClCompile>
    <Link>
      <SubSystem>Console</SubSystem>
```

```
    </Link>
  </ItemDefinitionGroup>
  <!-- (6) ItemGroup：参与构建的文件清单 -->
  <ItemGroup>
    <ClCompile Include="main.cpp" />
    <ClInclude Include="math_utils.h" />
  </ItemGroup>
  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.targets" />
</Project>
```

段落与关键元素	含义
ProjectConfigurations 中的 ProjectConfiguration (Include="Debug|x64")	声明本项目**支持**哪些配置与平台组合。属性页里能选到的配置，源头就在这里；缺少的组合在配置管理器里显示为空
Globals 中的 ProjectGuid	项目唯一标识，**项目引用与解决方案都靠它认人**；复制项目目录后必须换新 GUID，否则会出现“两个项目同一个 ID”
Globals 中的 WindowsTargetPlatformVersion	目标 Windows SDK 版本。写 10.0 表示“用本机已安装的最新一个”，写具体版本号则是锁死
Globals 中的 VCProjectVersion	记录生成这份文件的 VS 版本（18.0），重定目标时更新
Label="Configuration" 中的 ConfigurationType	Application / DynamicLibrary / StaticLibrary / Utility，决定产出什么
Label="Configuration" 中的 PlatformToolset	平台工具集，VS 2026 是 v145（VS 2022 是 v143）
Label="Configuration" 中的 UseDebugLibraries	是否用调试版运行库，就是 /MDd 与 /MD 的开关
ItemDefinitionGroup 中的 ClCompile / Link	编译与链接开关的集中地，对应属性页里绝大多数选项
ItemGroup 中的 ClCompile / ClInclude	源文件与头文件。**ClInclude 不参与编译**，只是让头文件出现在解决方案资源管理器里

两个容易踩的细节：%(PreprocessorDefinitions) 这种“继承已有值并追加”的写法**不能省**——属性表、项目引用、命令行都可能往同一个属性里塞值，少了 %(...) 就会把别人挤掉；**导入顺序决定优先级**（Microsoft.Cpp.Default.props → Microsoft.Cpp.props → 你的属性表 → Microsoft.Cpp.targets，后导入的覆盖先导入的）。另外，ClCompile / ClInclude 里的路径**相对于项目文件所在目录**，与筛选器无关。

6.2.4 .vcxproj.filters

```xml
<?xml version="1.0" encoding="utf-8"?>
<Project ToolsVersion="4.0" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <ItemGroup>
    <Filter Include="源文件">
      <UniqueIdentifier>{4FC737F1-C7A5-4376-A066-2A32D752A2FF}</UniqueIdentifier>
      <Extensions>cpp;c;cc;cxx</Extensions>
    </Filter>
  </ItemGroup>
  <ItemGroup>
    <ClCompile Include="main.cpp">
      <Filter>源文件</Filter>
    </ClCompile>
  </ItemGroup>
</Project>
```

.filters 只影响 IDE 里的显示分组，不参与构建：删掉它编译照样成功，只是所有文件会堆在项目根节点下。<Filter> 的值是筛选器的**名字**（不是路径），与磁盘目录没有任何绑定；<Extensions> 决定“新建这类文件时默认进哪个筛选器”。

6.3 SLNX：基于 XML 的新解决方案格式

.sln 的问题不是不能用，而是**不友好**：自有格式、制表符缩进、行序无意义、全靠 GUID 认人。.slnx 换成标准 XML，同一个解决方案长这样（<Folder> 就是解决方案文件夹——只存在于解决方案资源管理器里的虚拟分组，磁盘上不会因此生成目录）：

```xml
<Solution>
  <Project Path="src\App\App.vcxproj" />
  <Folder Name="/库/">
    <Project Path="src\MathLib\MathLib.vcxproj" />
    <Project Path="src\Plugin\Plugin.vcxproj" />
  </Folder>
  <Folder Name="/解决方案项/">
    <File Path="README.md" />
  </Folder>
</Solution>
```

维度	.sln	.slnx
格式	自有文本格式	标准 XML
可读性	差：GUID + 制表符	好：写的就是项目路径
手工编辑	官方不建议	可以，也适合脚本生成

维度	.sln	.slnx
合并冲突	难解，常靠人肉数 GUID	通常是清晰的行级冲突
项目类型	由类型 GUID 区分	由文件扩展名推断
CMake 侧	支持	VS 2026 自带的 **CMake 4.1.1** 的 `Visual Studio 18 2026` 生成器支持 SLNX

6.3.1 从 .sln 迁移为 .slnx

三条路：**命令行**最省事，`dotnet sln MyProduct.sln migrate` 会从同名 `.sln` 生成 `.slnx`（同名 `.slnx` 已存在时命令直接失败）；**IDE 内转换**是右键解决方案节点，使用与 SLNX 相关的“转换 / 另存”入口（不同小版本菜单措辞略有差异，找不到就用命令行）；**手写**则只需照着 `.sln` 里的 `Project(...)` 行把路径抄进 `.slnx`。官方文档的示例都是.NET 项目，纯 C++ 解决方案建议先在副本上试。

- **`.vcxproj` 完全不动**：换格式只换了项目清单的写法。
- **迁移可逆**：转换不会删除 `.sln`；建议先提交一次，转换后立刻全量构建，确认产物一致再删旧文件。
- **团队混用要确认版本**：还有人用旧版 Visual Studio 时先确认它认识 `.slnx`，不确定就两份清单暂时并存（长期维护两份迟早不同步）。

实战场景 CI 上跑的是 `msbuild MyProduct.sln`。迁移后 CI 脚本、构建文档、同事的快捷方式全都要跟着改——改完先手工跑一遍 CI 命令，再提交。

6.4 多项目解决方案

6.4.1 三种“跨项目用代码”的方式

方式	怎么做
项目引用	右键引用方项目下的 **引用**节点 > **添加引用**，在 **项目**页勾选
添加为链接	**添加 > 现有项**，选中文件后从“添加”按钮的下拉里选 **添加为链接**
直接改包含目录	**项目 > 属性 > C/C++ > 常规 > 附加包含目录**，再手工补 **链接器 > 输入 > 附加依赖项**

方式	头文件路径	链接依赖	构建顺序
项目引用	自动传递	自动传递	自动排序
添加为链接	无	无	无
直接改包含目录	手工	手工	手工

方式	头文件路径	链接依赖	构建顺序

方式	什么时候用
项目引用	同一解决方案内的自有代码，**默认就该选它**
添加为链接	一个几百行的小工具文件被两个项目共享，不值得为它建库
直接改包含目录	第三方预编译库（只有头文件和 `.lib`，没有项目文件）

“添加为链接”的代价：文件不会被复制，项目文件里写的是外部相对路径（可用 `<Link>` 元数据控制它在 IDE 里的显示位置），但**同一份源码会被编译进多个项目**——如果它同时被两个静态库包含、而这两个静态库又被同一个 exe 链接，就会得到 `LNK2005` 重复定义。

6.4.2 项目引用到底做了什么

加上项目引用后，MSBuild 会替你完成四件事：**排构建顺序**（先构建被引用项目）、**传递链接依赖**（把被引用项目的 `.lib` 或 DLL 导入库自动加进引用方的链接器输入）、**传递头文件搜索路径**（把被引用项目对外公开的包含目录加进编译命令）、**复制运行期文件**（产出 DLL 时把 DLL 复制到引用方输出目录）。它在项目文件里只是一个 `<ProjectReference Include="..\MathLib\MathLib.vcxproj">`，里面记着被引用项目的 `ProjectGuid`。

```
        App.vcxproj --ProjectReference--+
                                        v
                                MathLib.vcxproj ----> MathLib.lib
                                        ^
      Tests.vcxproj --ProjectReference--+

    MSBuild 自动完成的三件事：
      1. 先构建 MathLib，再构建引用它的项目
      2. 把 MathLib.lib 塞进引用方的链接器输入
      3. 把 MathLib 对外公开的包含目录加进引用方的搜索路径
```

注意 包含路径的传递有前提——头文件要放在被引用项目对外公开的目录里。库项目模板会把项目目录本身作为包含路径，所以头文件放在库项目根目录时通常能直接被引用方 `##include`；一旦你把公共头文件挪进子目录（例如 `include`），就要把这个目录显式加进**提供方**的附加包含目录，别指望它自动找到。另外，**项目 > 项目依赖项**只影响构建顺序，**不会**传递头文件与库——为了修“构建顺序不对”而加依赖项的人，最后往往发现链接依然不通过。

6.5 库项目：静态库与动态库

6.5.1 两种库的差别

	静态库	动态库
项目模板	静态库（.lib）	动态库（.dll）
ConfigurationType	StaticLibrary	DynamicLibrary
产出	MathLib.lib（里面是**真正的代码**）	Plugin.dll + 导入库 Plugin.lib（只有符号表）
链接行为	代码在链接时被**复制进** exe	链接的是导入库，运行时才加载 DLL
需要导出符号吗	不需要	需要（除非用 .def 文件列出导出名）
更新库之后	必须重新链接 exe	接口不变时直接换掉 DLL 即可

创建方式：**文件 > 新建 > 项目** → 搜“静态库”或“动态库” → 选 C++ 模板。新建库项目默认 C++20；动态库模板会**自动**在预处理器定义里写入“项目名大写 + _EXPORTS”形式的一个导出开关。

6.5.2 导出宏：一份完整可编译的示例

静态库不需要任何导出声明——链接器会把 .lib 里的符号直接搬进最终镜像。动态库相反：DLL 里的符号默认对外不可见，必须显式导出。__declspec(dllexport) 在编译 DLL 自己时用，告诉链接器“这个符号放进导出表”；__declspec(dllimport) 在编译调用方时用，让编译器生成走导入表的代码（不用它也能链接成功，但会多一层跳转，访问数据符号时还可能出错）。两者由同一个宏按“谁在编译”切换。

```
// ---------- mylib.h ----------
#pragma once
//   MYLIB_STATIC  -> 静态库或直接编进 exe：什么都不加
//   MYLIB_EXPORTS -> 正在编译 DLL 自己：导出
//   都没有        -> 正在编译调用方：导入
#if defined(MYLIB_STATIC)
#  define MYLIB_API
#elif defined(MYLIB_EXPORTS)
#  define MYLIB_API __declspec(dllexport)
#else
#  define MYLIB_API __declspec(dllimport)
#endif
// 函数：加 extern "C" 可避免 C++ 名字修饰，方便 GetProcAddress 按名字取
extern "C" MYLIB_API int mylib_add(int a, int b);
// 类：导出的是整个类的成员，接口里不要暴露 STL 容器的实现细节
class MYLIB_API Calculator {
public:
    int add(int a, int b) const;
};
```

```
20
21 // ---------- mylib.cpp (编译 DLL 时) ----------
22 #include "mylib.h"
23 extern "C" int mylib_add(int a, int b) { return a + b; }
24 int Calculator::add(int a, int b) const { return a + b; }
25
26 // ---------- main.cpp (调用方: 没有定义 MYLIB_EXPORTS) ----------
27 #include "mylib.h"
28 #include <iostream>
29 int main() {
30     Calculator calc;
31     std::cout << calc.add(2, 3) << '\n';   // 走导入表
32     std::cout << mylib_add(4, 5) << '\n';
33 }
```

用命令行验证一遍，比在 IDE 里点半天更直观（在项目里，这个开关就是预处理器定义里的一行 `MYLIB_EXPORTS;%(PreprocessorDefinitions)`）：

```
cl /nologo /LD /std:c++20 /EHsc /DMYLIB_EXPORTS mylib.cpp   :: 生成 mylib.dll + 导
    入库 mylib.lib
cl /nologo /std:c++20 /EHsc main.cpp /I. mylib.lib          :: 编译调用方
main.exe                                                    :: mylib.dll 需与 main
    .exe 同目录
dumpbin /exports mylib.dll                                  :: 核对导出表
```

注意 `MYLIB_EXPORTS` 必须**只**在编译 DLL 本身时定义。调用方也定义它的话，头文件会把函数声明成 `dllexport`，编译器于是认为“这个函数就在本模块里”，链接时得到 `LNK2019` 无法解析的外部符号。看到这个错误先检查预处理器定义，再怀疑代码。

提示 宏名要和项目名对上——VS 是按**建项目时**的项目名写入 `<项目名大写>_EXPORTS` 的，项目一改名这个宏不会跟着改，导出会静默失效：DLL 编得出来，调用方却链接不上，这是重命名项目后最常见的“灵异现象”。同一份源码既要能编静态库又要能编 DLL 时，给静态库配置加一条 `MYLIB_STATIC` 定义，头文件里的三段分支会自动选对身份。

6.6 解决方案配置与平台

6.6.1 配置与平台是一对

配置	平台（项目文件里的名字）	用途	典型开关
Debug	x64	日常开发与调试	关优化、带调试信息、调试版运行库（/MDd）
Release	x64	交付给用户	开优化、定义 NDEBUG、发布版运行库（/MD）、全程序优化
Debug / Release	Win32（IDE 显示为 x86）	32 位目标	指针 4 字节；只有 32 位版本的第三方库只能这样链
Release	ARM64	Windows on ARM 设备	在 x64 主机上交叉编译 ARM64 目标

注意 32 位平台在项目文件里的名字是 **Win32**，IDE 属性页里却显示成 **x86**。这是历史遗留的不一致，写 MSBuild 条件时别搞错——Condition="'$(Platform)'=='x86'" 永远不成立。

6.6.2 配置管理器与多项目对齐

入口是 **生成 > 配置管理器**。它是一张“解决方案配置 × 项目”的表：顶部下拉选当前解决方案配置，每个项目有 **配置**、**平台**、**生成**三列；**生成**复选框决定它是否参与“生成解决方案”（取消勾选只是不主动构建，被别的项目引用时通常仍会被编出来）；**配置**列就是**配置映射**——解决方案的 Debug|x64 完全可以映射到某个项目的 Debug|Win32。

- 让**所有项目拥有同一套配置与平台**（例如 Debug|x64、Release|x64），最省事的办法是新建项目时就用同一套；某个项目缺了解决方案里用到的配置时，配置管理器里那一格显示为空，这正是“生成解决方案时这个项目被静默跳过”的最常见原因。
- **运行库必须一致**：/MD（Release）与 /MDd（Debug）混链是经典崩溃来源，症状是堆内存跨模块分配/释放时出错；Debug 与 Release 的 .lib 同样不能混用。
- **ARM64**：VS 2026 提供 ARM64 工具集，平台名就是 ARM64，在 x64 主机上可交叉编译（安装时勾选 ARM64 生成工具）；**ARM32 工具链已移除**，要 32 位就只剩 x86；**ARM64EC** 让 ARM64 进程里能混入 x64 代码，适合依赖只有 x64 版的第三方库、又想上 ARM64 的场景。

提示 新增解决方案配置时，VS 会问每个项目“从哪个已有配置复制设置”。**这一步选错，新配置里会什么编译开关都没有**——典型症状是能编过但跑不起来。另外，每增加一个平台，项目文件里就要多一份 ProjectConfiguration、属性页里就多一整套设置要维护，平台不是免费的——这正是下一节属性表存在的理由。

6.7 属性表（Property Sheets）

6.7.1 为什么需要它、怎么用

算一笔账：3 个项目 × 2 个配置 = 6 份几乎相同的 `ItemDefinitionGroup`，改一次警告等级要开 6 次属性页，而且总有人漏改。属性表把“一组设置”抽出来变成独立文件（`.props`），项目通过 `<Import>` 引用它：**改一处，所有引用它的项目一起生效**；它也顺带解决了团队规范问题——统一规范做成一个 `.props` 纳入版本控制，新人建项目时挂上就合规。做法：**视图 > 其他窗口 > 属性管理器** → 展开项目，右键 **Debug | x64** 这样的配置节点（或右键项目）> **添加新项目属性表** → 双击它，出现的属性页与项目属性页**长得一模一样**，但设置只写进 `.props` → 别的项目用 **添加现有属性表**选中同一个文件。

挂在**具体配置节点**下只对该配置生效，挂在**项目节点**下对所有配置生效；多种属性表的优先级由顺序决定：**列表里靠下的覆盖靠上的**，而项目自己的设置又覆盖所有属性表。

6.7.2 `Directory.Build.props`：连“添加”这一步都省掉

MSBuild 构建每个项目时会从项目目录开始**向上查找** `Directory.Build.props` 并自动导入。把它放在解决方案根目录，整个目录树下所有项目就都继承了，不需要任何手工添加；同名的 `Directory.Build.targets` 效果相同，区别是它在项目末尾才导入。

```
<?xml version="1.0" encoding="utf-8"?>
<Project>
  <!-- (1) 固定工具集：整棵树用同一个编译器（详见第 1 章） -->
  <PropertyGroup>
    <VCToolsVersion>14.50.35717</VCToolsVersion>
    <!-- VS 2026 18.8+：允许跨 VS 安装实例查找被固定的工具集 -->
    <EnableVCToolsVersionDiscovery>true</EnableVCToolsVersionDiscovery>
  </PropertyGroup>
  <!-- (2) 统一输出位置：所有产出集中到 build\ 下，便于打包与清理 -->
  <PropertyGroup>
    <OutDir>$(SolutionDir)build\$(Platform)\$(Configuration)\</OutDir>
    <IntDir>$(SolutionDir)build\obj\$(ProjectName)\$(Platform)\$(Configuration)\</
    IntDir>
  </PropertyGroup>
  <!-- (3) 统一编译开关：所有项目、所有配置 -->
  <ItemDefinitionGroup>
    <ClCompile>
      <LanguageStandard>stdcpp20</LanguageStandard>
      <ConformanceMode>true</ConformanceMode>
      <WarningLevel>Level4</WarningLevel>
      <SDLCheck>true</SDLCheck>
      <MultiProcessorCompilation>true</MultiProcessorCompilation>
      <PreprocessorDefinitions>WIN32_LEAN_AND_MEAN;NOMINMAX;%(
    PreprocessorDefinitions)</PreprocessorDefinitions>
    </ClCompile>
```

```
  </ItemDefinitionGroup>
  <!-- (4) Release 特有：交付物也带 PDB，警告当错误 -->
  <ItemDefinitionGroup Condition="'$(Configuration)'=='Release'">
    <ClCompile>
      <DebugInformationFormat>ProgramDatabase</DebugInformationFormat>
      <TreatWarningAsError>true</TreatWarningAsError>
    </ClCompile>
    <Link>
      <GenerateDebugInformation>true</GenerateDebugInformation>
    </Link>
  </ItemDefinitionGroup>
</Project>
```

- **`$(SolutionDir)` 只在通过解决方案构建时才有定义**。如果你习惯直接对单个项目跑 `msbuild App.vcxproj`，得用 `/p:SolutionDir=D:\work\MyProduct` 补进去，否则输出目录会变成一串奇怪的字面量；某个项目的输出没落到 `build` 下，多半是它在项目文件里自己定义了 `OutDir`。
- **`Level4` + `TreatWarningAsError` 威力很大，但别在迁移期打开**：打开当天你会收到几百个“错误”。建议先在 CI 上开，本地保持只报警告。

注意 导入得**早**就意味着容易被覆盖。`Directory.Build.props` 里的值在项目自身内容之前求值，所以项目里显式写过的同名设置会赢。想“强制统一”，要么把这些重复设置从项目里删掉，要么改用 `Directory.Build.targets`（后导入者胜）。

6.8 筛选器与文件夹

名字里带“文件夹”的东西有三个，含义完全不同：

名字	存在于	影响什么	与磁盘目录的关系
磁盘目录	文件系统	编译器按路径找文件	事实本身
筛选器	项目的 IDE 元数据（`.vcxproj.filters`）	只影响解决方案资源管理器里的分组	**无**，可以完全不一致
解决方案文件夹	解决方案文件（`.sln` / `.slnx`）	只影响项目的分组	**无**，不能放文件，也不会被编译器看见

```
磁盘（真实）：
src\App\
+-- main.cpp
+-- app.h
```

```
+-- impl\
|   +-- engine.cpp
+-- engine.h

IDE 筛选器（虚拟）：
App
+-- 源文件
|   +-- main.cpp
+-- 头文件
|   +-- app.h
|   +-- engine.h
+-- 实现
    +-- engine.cpp

<- engine.cpp 在磁盘的 impl\ 下，在 IDE 里却和 engine.h 摆在一起
```

- 在 IDE 里新建筛选器**不会**在磁盘上建目录；反过来，在磁盘上建目录，IDE 也不会自动显示。
- **显示所有文件**：解决方案资源管理器工具栏上的按钮。打开后，项目目录下**没有被项目引用**的文件会以虚线图标显示；右键 → **包括在项目中**即可加入项目清单，用它批量纳管一整个新目录比逐个“添加现有项”快得多。反过来，**从项目中排除**只是把文件移出清单，磁盘上的文件还在。
- 想让显示位置与物理位置解耦，可以用 `<Link>` 元数据：物理上在 `..\shared\util.cpp` 的文件，可以显示成自己项目里的 `公共\util.cpp`。
- 两类常见困惑：**复制项目时漏拷 `.filters`**，于是所有文件堆在根节点下（编译一切正常，删掉 `.filters` 让 VS 重建即可）；**合并两个分支的 `.filters` 时 GUID 打架**，处理办法是选一边的结果，再让 IDE 重新整理。

6.9 共享项目（Shared Items Project，`.shproj`）

用途很明确：**一份源码要被多个项目共享，而你不想为它单独建库**。文件只存在一份（不像“添加为链接”要在每个项目里各加一遍），加入一次所有引用它的项目都能看到，典型场景是同一套核心代码既编进主程序、又编进测试程序。限制同样明确：

- **不能单独构建**，没有自己的产出，也没有自己的 `ConfigurationType`；**没有自己的配置与平台**，完全跟随引用它的项目，所以不能在里面放“只给某个项目用”的设置。
- 同一份源码会被每个引用它的项目各编译一遍，项目多了编译时间会明显变长。
- 在 C++ 里它比在.NET 里更“薄”：**只用来放文件**，不管引用与包，依赖仍要各项目自己配（或用第 7 节的属性表统一）。
- `.shproj` 只是一个很薄的外壳，真正列出文件的是同目录下的共享项目文件；引用方通过一个 `<Import ... Label="Shared" />` 把它导进来——VS 生成的 `.vcxproj` 里本来就留着一个空的 `<ImportGroup Label="Shared" />` 就是给它用的。

提示 在“共享项目”和“静态库”之间犹豫时，选**静态库**的次数应该更多：它有明确的产出、能单独跑单元测试、能控制导出、也能被 CMake 这类非 MSBuild 的构建系统使用。共享项目是“我就是要源码级复用，且坚决不想多一个二进制产物”时才动用的手段。

6.10 项目重定目标：从 v143 到 v145

6.10.1 什么时候需要，怎么改

- **不需要**：你只是想在 VS 2026 里打开、编辑、调试 VS 2022 的项目。**VS 2026 打开 VS 2022 的 .vcxproj 不需要升级项目文件**，它会照旧按 v143 编译（前提是装了 v143 工具集，安装时勾选）。
- **需要**：想用 14.50 / 14.51 的新特性（新的标准库设施、新警告、新优化），或者想在 CI 上只维护一套工具集。
- **怎么做**：打开老解决方案时如果缺工具集或缺 Windows SDK，VS 2026 会提示使用**设置助手（Setup Assistant）**，它既能帮你把缺失的工具集装上，也能把项目重定目标到新工具集；手动入口是右键解决方案节点 > **重定目标解决方案**。只想换编译器、不想动 SDK，就单独改一项：**项目 > 属性 > 配置属性 > 常规 > 平台工具集**选 v145。

6.10.2 重定目标改写了哪些字段

各字段所在的位置：

字段	位置
PlatformToolset	PropertyGroup，Label="Configuration"
WindowsTargetPlatformVersion	PropertyGroup，Label="Globals"
VCProjectVersion	PropertyGroup，Label="Globals"
MSVC 工具集版本	**不写在项目文件里**

各字段在重定目标前后的变化：

字段	变化
PlatformToolset	v143 → v145
WindowsTargetPlatformVersion	老 SDK 版本 → 新 SDK 版本（或写成 10.0 表示“用已安装的最新一个”）
VCProjectVersion	17.0 → 18.0
MSVC 工具集版本	由 PlatformToolset 决定实际选中的工具集（例如 14.50.35717），除非你用 VCToolsVersion 固定住

注意最后一行：**“MSVC 工具集版本变化”是选 v145 的结果，而不是项目文件里多了一个版本号**；工具集的真实路径在 C:\Program Files\...\VC\Tools\MSVC\14.50.35717 这类位置（第 1 章）。后遗症三条：**警告和错误会变多**（先做一次“只看警告”的构建评估工作量）；**二进制兼容性没问题**（14.50 / 14.51 与 VS 2015 及以后编译的代码保持二进制兼容，前提是 CRT 链接方式一致，所以链接第三方老 .lib 通常可行，但正式发布前应当用固定的工具集版本重新构建一次）；**重定目标后 VS 2022 就打不开这个项目了**（它不认识 v145），团队里还有人在用 VS 2022 时，最省事的过渡办法就是先别重定目标——VS 2026 编 v143 毫无问题。

6.11 版本控制卫生

6.11.1 哪些文件该进 Git

文件	进 Git 吗	原因
.sln / .slnx	[OK]	项目清单与配置映射，团队必须一致
.vcxproj	[OK]	编译链接设置的唯一来源
.vcxproj.filters	[OK]	只是显示分组，但缺了它每个人的解决方案资源管理器都不一样
.props / Directory.Build.props	[OK]	团队规范本体
.vcxproj.user	[X]	个人设置：调试命令行参数、工作目录、启动环境变量。你提交了它，同事的调试参数就被你覆盖
.vs	[X]	IDE 本地状态与 IntelliSense 数据库，体积大且因机器而异
.suo	[X]	用户级解决方案状态（断点、窗口布局），现在躺在 .vs 下
.aps / .ipch	[X]	资源编辑器缓存、IntelliSense 预编译头缓存，都是自动生成的
x64、Debug、Release、*.pdb、*.ilk、*.exp	[X]	构建产物（发布时另行归档）

6.11.2 .gitignore 片段

```
# ---------- Visual Studio 本地状态 ----------
.vs/
*.suo
*.user
*.aps
*.ipch
```

```
*.sdf
*.opendb
*.VC.db

# ---------- 构建产物 ----------
[Bb]in/
[Oo]bj/
x64/
x86/
ARM64/
Debug/
Release/
*.obj
*.exe
*.dll
*.lib
*.exp
*.pdb
*.ilk
*.tlog

# ---------- 例外：团队决定提交的第三方预编译库 ----------
!third_party/**/*.lib
```

- `*.lib` 一刀切会把**导入库**一起忽略——那是构建产物，忽略是对的；但团队若把第三方预编译库提交进仓库，必须用 `!` 例外捞回来，否则同事 clone 下来直接链接失败。
- `Debug/`、`Release/`、`[Bb]in/` 这类目录名规则会**误伤同名源码目录**：如果你真有一个叫 `bin` 的源码目录，把规则改成锚定仓库根的写法（例如 `/x64/`），或者只忽略真正的输出目录。输出目录被第 7 节的 `Directory.Build.props` 统一到 `build` 之后，记得把 `build/` 也加进来。
- `.vcxproj.user` 一旦被提交过，`git rm --cached <文件>` 之后还要提醒同事：VS 每次修改调试设置都会重写这个文件，别“顺手”再加回去。

6.12 实战：一个三项目解决方案

6.12.1 目录树与职责划分

```
D:\work\MyProduct\
+-- MyProduct.slnx              <- 解决方案清单（第 3 节）
+-- Directory.Build.props       <- 全树统一的编译开关（第 7 节）
+-- .gitignore                  <- 第 11 节
+-- src\
|   +-- App\                    <- 主程序：App.vcxproj、main.cpp
```

```
|   +-- MathLib\                <- 静态库：MathLib.vcxproj、math.cpp
|   |   +-- include\math_api.h  <- 对外头文件集中放这里
|   +-- Plugin\                 <- 动态库：Plugin.vcxproj、plugin.cpp
+-- build\                      <- 构建产物（不进 Git）
```

项目	类型	产出	引用谁	职责
App	Application	App.exe	MathLib、Plugin	命令行解析、调度、把结果打印出来
MathLib	StaticLibrary	MathLib.lib	无	纯计算，不碰 Windows API，方便测试与移植
Plugin	DynamicLibrary	Plugin.dll + Plugin.lib	MathLib	扩展点实现，由 App 在运行期加载

划分原则很简单：**依赖方向必须是一条线，不能成环**。MathLib 谁都不认识，Plugin 只认识 MathLib，App 认识上面两个。以后要写单元测试，再加一个 MathLibTests 项目引用 MathLib 即可（见 **第 11 章**）。

6.12.2 引用关系

```
App.exe（唯一的可执行文件）
  |
  +-- 项目引用 --> MathLib.lib（纯计算、零外部依赖）
  |
  +-- 项目引用 --> Plugin.dll + Plugin.lib（扩展点实现，自己也引用 MathLib）

MathLib <-- Plugin <-- App      这是 MSBuild 的构建顺序
MSBuild 把 MathLib.lib、Plugin.lib 送进 App 的链接器输入
```

6.12.3 一步步搭起来

```
1. 文件 > 新建 > 项目 > 空白解决方案，命名 MyProduct
2. 右键解决方案 > 添加 > 新建项目 > 静态库，命名 MathLib，位置 src\MathLib
3. 右键解决方案 > 添加 > 新建项目 > 动态库，命名 Plugin，位置 src\Plugin
4. 右键解决方案 > 添加 > 新建项目 > 控制台应用，命名 App，位置 src\App
5. 在 Plugin 下的"引用"节点右键 > 添加引用 > 项目 > 勾选 MathLib
6. 在 App 下的"引用"节点右键 > 添加引用 > 项目 > 勾选 MathLib 与 Plugin
7. 生成 > 配置管理器，把三个项目的平台统一成 x64
8. 在解决方案根目录放一份 Directory.Build.props（第 7 节那份）
9. 生成 > 生成解决方案，检查 build\x64\Debug\ 下的产物
```

搭完之后至少验证三件事：**产物是否齐全**（App.exe、MathLib.lib、Plugin.dll 应出现在同一处——项目引用会把 DLL 复制到 exe 旁边）；**头文件能不能找到**（在 Plugin 里写 ##include

"math_api.h"，报“无法打开源文件”就把 MathLib\include 加进 MathLib 的附加包含目录，或在 Directory.Build.props 里统一补一条，后者更省事也更容易在换机器时保持一致）；**别把本地状态提交上去**（第一次 git status 就核对，.vs、x64、*.user 都不该出现在待提交列表里）。

6.13 小结

- 三层关系：**解决方案** ⊃ **项目** ⊃ **文件**。解决方案只是容器，**不产出任何二进制**；项目才是承载编译链接设置的地方。
- .vcxproj 记住四段：ProjectConfigurations（支持哪些配置）、Globals（ProjectGuid、WindowsTargetPlatformVersion）、Label="Configuration" 的 PropertyGroup（ConfigurationType、PlatformToolset）、ItemDefinitionGroup + ItemGroup（开关与文件清单）；追加值别忘了 %(...)。
- **.vcxproj.filters 只影响 IDE 显示，不影响编译**；.vcxproj.user 与 .suo 是个人的，绝不提交。
- **SLNX** 是 VS 2026 支持的 XML 解决方案格式：可读、可手写、冲突好解；可用 dotnet sln < 文件 >.sln migrate 生成，迁移不影响 .vcxproj，而且可逆。
- 跨项目用代码优先用**项目引用**：它自动传递链接依赖与头文件搜索路径并排好构建顺序；**项目依赖项**只管顺序，不管传递。
- 静态库直接搬代码进 exe，不需要导出；动态库必须用 __declspec(dllexport) / dllimport，靠 < 项目名 >_EXPORTS 宏切换，项目改名后记得同步宏名。
- 多项目要**配置与平台对齐**，运行库链接方式（/MD 与 /MDd）绝不能混；32 位平台在项目文件里叫 Win32，IDE 里显示为 x86。
- 用 .props 消除重复设置；Directory.Build.props 放在解决方案根目录就能自动作用于整个目录树，但它导入得早、容易被项目自身设置覆盖。
- VS 2026 打开 VS 2022 项目**不需要升级项目文件**；只有主动重定目标时才改写工具集与 Windows SDK 字段，从 v143 到 v145。

6.14 练习题

1. 新建一个控制台项目，用文本编辑器打开 .vcxproj，找出 LanguageStandard、PlatformToolset、WindowsTargetPlatformVersion 三个值；再只用属性页各改一次，回到编辑器看是哪几行变了。
2. 把 .vcxproj.filters 删掉，重新打开项目并用 **显示所有文件**加回一个源文件，观察解决方案资源管理器的变化，并确认项目仍能编译。
3. 用 dotnet sln < 你的解决方案 >.sln migrate 生成 .slnx，对比两个文件描述同一个解决方案的方式，构建一次确认产物没有变化。

4. 做一个两项目方案：一个静态库 + 一个控制台应用，用项目引用连起来；然后把同一个头文件移到 include 子目录，记录项目引用是否还能找到它，以及你需要补哪一步设置。
5. 把练习题 4 的静态库改成动态库，写出带 MYLIB_EXPORTS 切换的导出宏，并用 dumpbin /exports 确认导出表里确实有你写的符号。
6. 在解决方案根目录加一份 Directory.Build.props，把 WarningLevel 统一成 Level4 并打开 TreatWarningAsError，统计整个解决方案多出多少警告；再把它挪到 Directory.Build.targets，对比生效范围有什么不同。

6.15 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **上一章：第一个 C++ 项目**
- **下一章：项目属性与编译配置**
- **CMake：CMake 使用指南**
- **Git 命令：常用 Git 命令**

第七章　项目属性与编译配置

前面几章里，`/std:c++20`、`/MD`、`/O2` 这些开关都是一句带过。这一章把它们全部展开——**项目属性页是 Visual Studio 里信息密度最高的地方**，也是绝大多数“玄学编译错误”的真正源头。读完这一章，你应该能做到：看到一个编译开关就知道它在哪个页面，看到一类报错就知道该去改哪一项。

7.1　属性页的入口与结构

7.1.1　三种打开方式

方式	路径	适用
右键项目	解决方案资源管理器 > 右键项目 > **属性**	最常用
菜单	**项目 > 属性**	键盘流
快捷键	选中项目后按 `Alt+Enter`	最快

注意 **属性页是“项目”级别的，不是“解决方案”级别，也不是全局设置**。解决方案右键没有“属性”这一项。如果你在找 `/std:c++20` 却翻到了 **工具 > 选项**，那永远找不到——那里管的是 IDE 自己的行为。

7.1.2　顶部两个下拉框：新手最大的坑

属性页**左上角**有两个下拉框：

```
[ 配置(C):  Debug    v ]      [ 平台(P):  x64    v ]      <- 看这里!
------------------------------------------------------------------
  v 配置属性
    v 常规
    v C/C++
    v 链接器
    v 清单工具
    v 生成事件
    v 调试
------------------------------------------------------------------
右侧是当前"配置 × 平台"组合下这一项的取值
```

你在属性页里改的任何一项，默认只作用于当前选中的“配置 × 平台”组合。

举个真实的翻车场景：

```
1. 打开项目属性，看到配置是 Debug|x64
2. 把"附加包含目录"改成 D:\libs\include
3. 编译 Debug -> 通过 [OK]
4. 切到 Release 编译 -> 报"无法打开源文件 xxx.h" [X]
原因：Release|x64 那一份配置里，包含目录根本没改
```

两种正确做法：

做法	操作	何时用
一次改所有配置	把“配置”下拉切成 **所有配置**，“平台”切成 **所有平台**，再改	绝大多数情况
逐个配置改	分别切到 Debug/Release 各改一遍	确实需要差异时（如 Debug 用 `/Od`）

> **提示** 改完之后，用 **配置**下拉逐个切过去检查一遍。养成“改完扫一眼其他配置”的习惯，能省掉大量“Release 编不过”的排查时间。

7.1.3 属性页树形结构

```
配置属性
+-- 常规                    <- 输出目录、配置类型、工具集、语言标准、字符集
+-- 高级                    <- 极少用（如 C++/CLI 属性）
+-- 调试                    <- 命令行参数、工作目录、环境变量
+-- VC++ 目录               <- 包含/库/可执行文件目录（建议改用 C/C++ 与链接器页）
+-- C/C++                   <- 编译器开关（cl.exe）
|   +-- 常规                <- /I /Zi /W4 /WX /MP /utf-8
|   +-- 优化                <- /Od /O1 /O2 /Ox /GL
|   +-- 预处理器            <- /D /U /X
|   +-- 代码生成            <- /MD /MT /EHsc /GR /Gy /GS /arch
|   +-- 语言                <- /std: /permissive- /openmp /Zc
|   +-- 预编译头            <- /Yc /Yu /Fp
|   +-- 输出文件            <- /Fa /Fd /Fo
|   +-- 浏览信息            <- /FR
|   +-- 高级                <- /FI /TC /TP /showIncludes
|   +-- 所有选项            <- 纯命令行视角
+-- 链接器                  <- 链接开关（link.exe）
|   +-- 常规                <- /LIBPATH /INCREMENTAL
|   +-- 输入                <- 附加依赖项 /DEFAULTLIB /NODEFAULTLIB
|   +-- 清单文件            <- UAC 清单
|   +-- 系统                <- /SUBSYSTEM /STACK
|   +-- 优化                <- /LTCG /OPT:REF /OPT:ICF
```

```
|   +-- 嵌入的 IDL          <- 组件开发
|   +-- Windows 元数据      <- WinRT
|   +-- 高级                <- /ENTRY /DYNAMICBASE /NXCOMPAT
|   +-- 命令行              <- 最终命令行预览
|   +-- 所有选项            <- 纯命令行视角
+-- 清单工具                <- 清单生成
+-- XML 文档生成器          <- /doc
+-- 浏览信息                <- 代码浏览数据库
+-- 生成事件                <- 生成前/链接前/生成后
+-- 自定义生成步骤          <- 单文件自定义命令
+-- 代码分析                <- /analyze、clang-tidy（见第 9 章）
+-- 调试                    <- 同"配置属性 > 调试"
```

提示 **每个页面的“命令行”子页（或“所有选项”页）会实时显示当前配置对应的完整命令行**。想确认某个图形选项到底生成了什么开关，去那里看一眼最快——这是学习属性页最好的方法。

7.2 配置属性 > 常规

各字段对应的开关或变量：

字段	对应开关/变量
输出目录	`$(OutDir)`
中间目录	`$(IntDir)`
目标名称	`$(TargetName)`
目标扩展名	`$(TargetExt)`
配置类型	`ConfigurationType`
C++ 语言标准	`/std:`
Windows SDK 版本	`WindowsTargetPlatformVersion`
平台工具集	`PlatformToolset`
字符集	`CharacterSet`

各字段的说明：

字段	说明
输出目录	默认 `$(SolutionDir)$(Platform)\$(Configuration)`
中间目录	`.obj`、`.pdb` 等中间产物
目标名称	默认等于项目名
目标扩展名	`.exe` / `.dll` / `.lib`

字段	说明
配置类型	**应用程序 / 动态库 / 静态库 / 生成文件**
C++ 语言标准	见 2.5 节
Windows SDK 版本	通常选“最新的已安装版本”
平台工具集	VS 2026 是 `v145`
字符集	使用 Unicode 字符集 / 使用多字节字符集

7.2.1 配置类型：改错了会得到奇怪的产物

配置类型	产物	何时选
应用程序 (.exe)	可执行文件	有 `main` 的项目
动态库 (.dll)	`.dll` + 导入库 `.lib`	运行时加载、跨语言调用（见 **第 5 章**）
静态库 (.lib)	`.lib`	编译期链接进调用方
生成文件	不调用 MSVC	交给 NMake 之类的自定义构建

注意 **“没有 `main` 却选了应用程序”会报 `LNK1561`：** 必须定义入口点；反过来“有 `main` 却选了静态库”则会把 `main` 一起打进库里，链接时可能出现重复定义。

7.2.2 输出与中间目录

默认值依赖两个宏：

```
$(SolutionDir)          -> .sln 所在目录
$(Platform)             -> x64 / Win32 / ARM64
$(Configuration)        -> Debug / Release
$(ProjectDir)           -> .vcxproj 所在目录
```

想统一把产物收到一个目录：

```
输出目录:
  $(SolutionDir)build\$(Platform)\$(Configuration)\
中间目录:
  $(SolutionDir)build\intermediate\$(ProjectName)\$(Platform)\$(Configuration)\
```

实战场景 中间目录**必须按项目分开**。如果多个项目共用同一个中间目录，`.obj` 与 `.pdb` 会互相覆盖，症状是“改了代码没生效”“断点跳到别的文件”，极难排查。

7.2.3 字符集：Unicode 还是多字节

选项	效果	建议
使用 Unicode 字符集	定义 _UNICODE 与 UNICODE，TCHAR → wchar_t，MessageBox → MessageBoxW	[OK] 新项目一律选这个
使用多字节字符集	不定义上述宏，走 char 版本 API	只在维护极老的代码时用

注意 多字节字符集**早已被微软标记为弃用**。它和“中文乱码”关系密切，但**换字符集不是解决乱码的正确手段**——乱码的根因是源码编码与控制台代码页，见 **第 13 章**。

7.2.4 平台工具集与 Windows SDK

- **平台工具集**：v145 是 VS 2026 的默认值。改成 v143 就用 VS 2022 的编译器（需要已安装该工具集）。
- **Windows SDK 版本**：决定能用哪些 Windows API。选“最新的已安装版本”最省事；要兼容老系统时才需要降级。

7.2.5 C++ 语言标准：属性页选项 ↔ 编译器开关

属性页下拉项	编译器开关	说明
ISO C++14 标准	/std:c++14	编译器的历史默认值
ISO C++17 标准	/std:c++17	
ISO C++20 标准	/std:c++20	**VS 2026 新建项目的默认值**；隐含 /permissive-
预览 - ISO C++23 标准	/std:c++23preview	只启用 C++23 范围内的特性
预览 - 最新 C++ 工作草案	/std:c++latest	含进行中的实验特性；隐含 /permissive-

在 .vcxproj 里对应的属性是 <LanguageStandard>，常用取值：stdcpp14、stdcpp17、stdcpp20、stdcpplatest。

```
<ClCompile>
  <LanguageStandard>stdcpp20</LanguageStandard>
  <ConformanceMode>true</ConformanceMode>
</ClCompile>
```

注意 **没有 /std:c++26 这个开关**。想用前沿特性只能挂 /std:c++latest，并接受“下个版本可能变”的风险。关于 C++23 与 /std:c++23 的时间线，见 **第 1 章 4.3 节**。

7.3 C/C++ > 常规

字段	开关	说明
附加包含目录	/I	头文件搜索路径
调试信息格式	/Zi /Z7 /ZI	见 3.2
取消显示启动版权标志	/nologo	让输出干净一点
警告等级	/W0–/W4、/Wall	见 3.3
将警告视为错误	/WX	见 3.3
多处理器编译	/MP	见 3.4
使用 UTF-8 编码	/utf-8	见 3.5
SDLCheck	/sdl	额外的安全检查警告，建议开

7.3.1 附加包含目录

多个路径用**分号**分隔：

```
D:\libs\fmt\include;$(ProjectDir)include;$(SolutionDir)third_party\eigen
```

提示 $(ProjectDir) 这类宏在属性页里是**可用的**，用它写相对路径比写死绝对路径更适合团队协作。

7.3.2 调试信息格式

选项	开关	特点
程序数据库 (/Zi)	/Zi	调试信息写进 .pdb，**推荐**；支持“编辑并继续”
C7 兼容 (/Z7)	/Z7	调试信息嵌进 .obj，文件大，但不用管 .pdb 丢失
用于“编辑并继续”的程序数据库 (/ZI)	/ZI	支持编辑并继续，但会禁用部分优化

注意 VS 2026 中 **/DEBUG:FASTLINK 已被移除**，链接器调试信息请用 **/DEBUG:FULL**。老项目如果手工指定过 FASTLINK，升级后会直接报错。

7.3.3 警告等级与 /WX

等级	开关	含义
关闭所有警告	/W0	[X] 不建议
等级 1	/W1	严重警告
等级 2	/W2	
等级 3	/W3	**MSVC 默认**
等级 4	/W4	[OK] 新项目建议
启用所有警告	/Wall	极其冗长，需要大量 /wd 屏蔽，谨慎使用

将警告视为错误（/WX）：把警告升级为错误。这是保证代码质量最有效的一招，但**迁移老项目时不要一上来就开**——先看 /W4 会刷出多少条，逐个处理完再开 /WX。

```
// 只想在本地屏蔽某一条时，用代码级抑制（比全局 /wd 更精准）
#pragma warning(push)
#pragma warning(disable: 4996)
    std::strcpy(dst, src);   // 明知不安全但暂时保留
#pragma warning(pop)
```

7.3.4 多处理器编译（/MP）

/MP 让编译器**并行编译同一个项目里的多个 .cpp**。在 8 核机器上，十几个源文件的项目构建时间通常能减半。

限制：

- 与 /Gm（已弃用）不兼容；
- 与预编译头配合时需要注意；
- **只并行同一项目内的文件**——多项目并行由 MSBuild 的 /m 参数负责（**工具 > 选项 > 项目和解决方案 > 生成并运行 > 最大并行项目生成数**）。

7.3.5 使用 UTF-8 编码（/utf-8）

/utf-8 等价于同时指定：

```
/source-charset:utf-8      源码文件按 UTF-8 解读
/execution-charset:utf-8   生成的窄字符串字面量用 UTF-8
```

这是现代 C++ 项目处理中文最省事的方案：源码存 UTF-8（不带 BOM 也行），加 /utf-8，代码里的中文字符串字面量就不会在编译期被按 GBK 误解。控制台输出乱码是另一层问题，见**第 13 章**。

7.4 C/C++ > 优化

属性页选项	开关	含义	何时用
已禁用	/Od	不做优化	Debug
最小化大小	/O1	体积优先	体积敏感的发布版
最大化速度	/O2	速度优先	**[OK]** Release 默认
完全优化	/Ox	折中	较少用
自定义	逐项设置		精细控制

细项：

细项	开关	说明
内联函数展开	/Ob0 /Ob1 /Ob2	/Ob2 是 /O2 的默认
启用内部函数	/Oi	用内建指令替换 memcpy 之类
优选大小或速度	/Ot /Os	默认 /Ot（速度）
启用纤维安全优化	/GT	使用纤程时开启
整个程序优化	/GL	跨编译单元优化，**必须配合链接器的 /LTCG**

注意 **/GL 与 /LTCG 必须成对出现**。只开 /GL 不开 /LTCG，链接器会给出警告并退化为普通链接，白折腾一场。Release 配置模板通常已经成对开好了。

实战场景 /O2 会暴露未定义行为。代码在 Debug 下跑得好好的，Release 下崩溃或结果不对——先怀疑**未初始化变量**、**越界访问**、**有符号溢出**，而不是怀疑编译器。用 Address-Sanitizer 复查，见 **第 9 章**。

7.5 C/C++ > 预处理器

字段	开关	示例
预处理器定义	/D	WIN32;_DEBUG;_CONSOLE;MYLIB_EXPORTS
取消预处理器定义	/U	取消某个宏
忽略标准包含路径	/X	忽略 INCLUDE 环境变量
保留注释	/C	保留注释到预处理输出

注意保留 %(PreprocessorDefinitions)：

```
<PreprocessorDefinitions>MY_FEATURE=1;%(PreprocessorDefinitions)</
    PreprocessorDefinitions>
```

%(...) 是“继承已有的定义”。**漏掉它会把 Debug/Release 模板自带的 _DEBUG、NDEBUG 等宏全部覆盖掉**，导致 assert 行为异常——这是手改 .vcxproj 时最常见的错误。

7.6 C/C++ > 代码生成

7.6.1 运行库：/MD 还是 /MT

这是 C++ 在 Windows 上最重要的一条配置。

属性页选项	开关	CRT 链接方式	部署要求
多线程调试 DLL (/MDd)	/MDd	动态链接调试版 CRT	Debug 用
多线程 DLL (/MD)	/MD	**动态**链接发行版 CRT	目标机需装 **VC++ 可再发行程序包**
多线程调试 (/MTd)	/MTd	静态链接调试版 CRT	Debug 用
多线程 (/MT)	/MT	**静态**链接发行版 CRT	免依赖，但体积大

维度	/MD	/MT
体积	小	大（每个 exe/dll 都带一份 CRT）
部署	需要 vc_redist	拷过去就能跑
跨 DLL 传 CRT 对象	[OK] 可以	[X] 危险
多个模块共享堆	[OK] 同一份堆	[X] 各自的堆
与第三方库混用	通常没问题	**必须完全一致**

注意（铁律） **同一个程序里的所有编译单元与所有静态库，必须使用同一种运行库选项。**
混用 /MT 与 /MD 会让同一个进程里出现**两份 CRT、两个堆**——在 A 模块里 new、在 B 模块里 delete，就是**堆损坏**，症状是随机崩溃且极难定位。
静态库混用通常会在链接期被 LNK2038（RuntimeLibrary 不匹配）拦下；但**通过 DLL 边界混用时链接器发现不了**，只能靠发布前的人工检查。

7.6.2 启用 C++ 异常

选项	开关	说明
是	/EHsc	[OK] 推荐：只捕获 C++ 异常，假定 extern "C" 函数不抛
是，但有 extern "C" 函数	/EHs	假定 extern "C" 可能抛
是，但有 SEH 异常	/EHa	同时捕获结构化异常，会拖慢性能

选项	开关	说明

注意 /EHsc 不只是“打开异常”，它还告诉编译器**析构函数需要参与栈展开**。用 /EHs 或 /EHa 代替 /EHsc 可能带来性能损失，而完全不开异常会让 try/catch 报错。

7.6.3 其他代码生成选项

字段	开关	建议
启用运行时类型信息	/GR（关闭为 /GR-）	用 dynamic_cast / typeid 就必须开
启用函数级链接	/Gy	**[OK]** 开，配合 /OPT:REF 能显著减小体积
缓冲区安全检查	/GS	**[OK]** 开（默认开）
控制流保护	/guard:cf	**[OK]** 建议开，缓解劫持攻击
启用最小重新生成	/Gm	**[X]** 已弃用，别开
启用增强指令集	/arch:	见下

/arch: 的取值与风险：

选项	含义	风险
/arch:SSE2	x86 默认	安全
/arch:AVX	使用 AVX	老 CPU 会崩
/arch:AVX2	使用 AVX2	**在不支持的机器上直接 0xC000001D（非法指令）**
/arch:AVX512	使用 AVX-512	同上，且支持面更窄

实战场景 为了那点性能开了 /arch:AVX2，结果在客户的旧机器上一启动就崩，报 0xC000001D。**要么用运行时分发（同一份代码编多个版本 + CPU 特性检测），要么老实留在 /arch:SSE2**。详见 **第 12 章**。

7.7 C/C++ > 语言

字段	开关	说明
C++ 语言标准	/std:	见 2.5 节
符合模式	/permissive-	严格标准一致性，拒绝微软扩展
禁用特定警告	/wd########	如 /wd4996

字段	开关	说明
将警告视为错误	/WX	
C++ 异常	/EHsc	见 6.2
OpenMP 支持	/openmp	使用 ##pragma omp
C++ 模块支持	见下	

7.7.1 符合模式 /permissive-

/permissive- 关闭一堆历史遗留的宽松行为（两阶段名称查找、模板中的 typename 宽容处理等）。**/std:c++20 与 /std:c++latest 会自动启用它**，所以选了 C++20 之后，老代码可能突然报一堆错——这不是新标准的问题，而是**编译器终于按标准严格检查了**。

7.7.2 关于 ___cplusplus

MSVC 为了兼容老代码，__cplusplus **默认永远是** 199711L。想让它反映真实标准，需要显式开启：

C/C++ > 命令行 > 其他选项里加 /Zc:__cplusplus。

或者干脆用 _MSVC_LANG 判断（推荐）：

```
#if _MSVC_LANG >= 202002L
    // C++20 及以上
#endif
```

7.7.3 C++ 模块

C++20 模块在 MSVC 上已可用，但需要显式开启。在属性页里把相关选项打开（或在命令行加 /std:c++20 /experimental:module 之类），并把模块接口单元（.ixx）的文件项类型设为“**C/C++ 编译器**”。模块的构建配置比传统头文件复杂，**团队项目里建议先用 CMake 统一管理**，见 **第 10 章**。

7.8 C/C++ > 高级

字段	开关	用途
强制包含文件	/FI	每个源文件隐式包含某头文件
编译为	/TC /TP	强制按 C 或 C++ 编译
禁用特定警告	/wd	
显示包含文件	/showIncludes	排查“这个头文件从哪来的”
预编译头	/Yc /Yu /Fp	见第 12 节
调用约定	/Gd /Gr /Gz	一般不改

7.8.1 强制包含 /FI

```
/FI"pch.h"
```

等价于在每个 .cpp 顶部自动写一行 ##include "pch.h"。配合预编译头使用，或者用来统一注入版本宏。

7.8.2 显示包含文件 /showIncludes

排查头文件依赖的神器。开启后，**输出窗口**会列出每个源文件实际包含的全部头文件（带层级缩进）：

```
1>注意: 包含文件:  D:\cpp\App\pch.h
1>注意: 包含文件:  C:\Program Files\...\include\vector
1>注意: 包含文件:  D:\cpp\App\config.h
```

配合第 9 章的 Build Insights，可以精确定位“哪个头文件拖慢了编译”。

7.9 链接器

7.9.1 常规

字段	开关	说明
附加库目录	/LIBPATH	.lib 搜索路径
启用增量链接	/INCREMENTAL	Debug 默认开；Release 建议关
忽略导入库	/IGNORE	

7.9.2 输入

字段	开关	说明
附加依赖项	直接写 .lib 名	如 fmt.lib;ws2_32.lib
忽略特定默认库	/NODEFAULTLIB	谨慎使用
强制符号引用	/INCLUDE	强制拉入某个符号（常用于静态库注册）

提示 **“附加库目录”负责去哪找，“附加依赖项”负责找什么**——两者要配套设置，只设一个必然报 LNK2019 或 LNK1104。

7.9.3 系统

字段	开关	说明
子系统	/SUBSYSTEM:CONSOLE	控制台程序：有黑窗口，入口 main
	/SUBSYSTEM:WINDOWS	GUI 程序：无黑窗口，入口 WinMain
堆栈保留大小	/STACK	默认 1 MB；递归很深的算法需要加大

注意 **子系统选错会导致 LNK2019：** *无法解析的外部符号* **main (或 WinMain)**。控制台程序误设成 WINDOWS，就会出现“找不到 main”这种看似荒谬的报错——因为链接器在找 WinMain。

7.9.4 优化

字段	开关	说明
链接时间代码生成	/LTCG	**必须与 /GL 成对**
引用	/OPT:REF	删除未引用代码，Release 默认开
启用 COMDAT 折叠	/OPT:ICF	合并相同函数，Release 默认开

7.9.5 高级

字段	开关	建议
入口点	/ENTRY	一般不用手改
随机基址	/DYNAMICBASE	**[OK]** 保持开启（安全缓解）
数据执行保护	/NXCOMPAT	**[OK]** 保持开启
目标计算机	/MACHINE	通常自动

7.9.6 调试

字段	开关	说明
生成调试信息	/DEBUG	配合“生成完整程序数据库” = /DEBUG:FULL
生成程序数据库文件	/PDB	PDB 路径与文件名
~~生成快速链接调试信息~~	~~/DEBUG:FASTLINK~~	**VS 2026 已移除**，老项目需改掉

7.10 生成事件

三个入口，按执行时机排列：

```
生成前事件 -> 编译 -> 链接前事件 -> 链接 -> 生成后事件
```

常用于：拷贝依赖 DLL 到输出目录、生成版本头文件、调用代码生成器。

可用宏：

宏	含义
`$(SolutionDir)`	解决方案目录（**含结尾反斜杠**）
`$(ProjectDir)`	项目目录（含结尾反斜杠）
`$(ProjectName)` / `$(SolutionName)`	名称
`$(OutDir)`	输出目录（含结尾反斜杠）
`$(IntDir)`	中间目录
`$(TargetPath)`	输出文件的完整路径
`$(TargetName)` / `$(TargetExt)`	文件名与扩展名
`$(Configuration)` / `$(Platform)`	当前配置与平台

一个典型的生成后事件（把第三方 DLL 拷到输出目录）：

```
xcopy /Y /I "$(SolutionDir)third_party\bin\$(Platform)\*.dll" "$(OutDir)"
```

注意 `$(SolutionDir)` 只在**通过解决方案构建**时才有定义。如果直接 `msbuild MyProject.vcxproj`（不经过 `.sln`），它是空的——生成事件会静默失败。稳妥写法是用 `$(ProjectDir)` 加相对路径。

7.11 调试属性页

配置属性 > 调试决定了按 F5 时发生什么：

字段	说明
命令	要启动的可执行文件（默认 `$(TargetPath)`）
命令参数	传给程序的命令行参数
工作目录	程序的当前目录（**读相对路径文件时很关键**）
环境	临时环境变量，如 `PATH=$(OutDir);%PATH%`
调试器类型	自动 / 仅限本机 / 混合等
合并环境	是否继承系统环境变量

提示 **VS 2026 也可以直接从工具栏设置命令行参数**，不必每次都进属性页。这一能力现在对 `.vcxproj`、CMake 与 Unreal 项目都可用，且不再依赖“使用 C++ 的游戏开发”工作负荷。

注意：这些设置存在 `.vcxproj.user` 里，是**本机**配置，不要提交到 Git。团队共用的参数应写进 `.vcxproj`。

实战场景 程序在工作目录不对时读不到配置文件。把“工作目录”设成 `$(ProjectDir)`，并在代码里用相对路径 `"config/settings.ini"`，本地调试就稳定了。

7.12 预编译头（PCH）实践

目的：把那些“又大又几乎不变”的头文件（`<vector>`、`<string>`、Windows SDK 头）**只解析一次**，后续编译直接复用，能显著加快构建。

配置步骤：

1. 建 `pch.h`，把常用头文件放进去；
2. 建 `pch.cpp`，内容只有一行 `##include "pch.h"`；
3. 选中 `pch.cpp` → **属性** > **C/C++** > **预编译头** > **预编译头**设为 **创建 (/Yc)**；
4. 选中其余所有 `.cpp` → 同一项设为 **使用 (/Yu)**；
5. 可选：在 **C/C++** > **高级** > **强制包含文件**里填 `pch.h`。

常见报错：

报错	原因	处置
`C1010`：查找预编译头时遇到意外的文件结尾	某个 `.cpp` 没包含 `pch.h`	在文件首行加 `##include "pch.h"`
`C1853`：预编译头文件来自早期版本的编译器	工具集换了但 PCH 没重建	**重新生成**解决方案
`C1083`：无法打开预编译头文件	`pch.cpp` 没设为“创建”	检查第 3 步

注意 `/Yc` 只能有一**个**文件。把它误设到多个 `.cpp` 上会报重复定义。

7.13 多核编译与构建速度

按性价比排序：

手段	做法	收益
`/MP`	**C/C++ > 常规 > 多处理器编译** = 是	单项目内并行，收益最大
MSBuild 并行项目	**工具 > 选项 > 项目和解决方案 > 最大并行项目生成数**	多项目解决方案
预编译头	见第 12 节	大型项目显著
`/GL` + `/LTCG`	Release	提升运行性能，但**会拖慢链接**
关闭增量链接	Release 下	避免增量链接的额外开销
减少头文件依赖	前置声明、PIMPL	治本
`/showIncludes` **或 Build Insights**	找出“谁最慢”	定位问题

提示 MSVC 14.51 起启用了**调试信息裁剪**来减小 PDB 体积，大型项目的 `.pdb` 会明显变小。如果 PDB 曾经大到难以管理，升级工具集本身就有收益。

7.14 配置的三种持久化方式

方式	载体	优点
图形界面	`.vcxproj`	直观、即时生效
属性表	`.props`	一处修改、多项目共享
`Directory.Build.props`	目录级 XML	**自动应用**到该目录及子目录所有项目

方式	缺点	适用
图形界面	逐个项目改，易漏配置	单人、少量项目
属性表	需要理解导入顺序	团队统一规范
`Directory.Build.props`	隐式生效，新人容易“找不到配置在哪”	整个仓库统一基线

`Directory.Build.props` 示例：

```
<Project>
  <PropertyGroup>
    <LanguageStandard>stdcpp20</LanguageStandard>
    <WarningLevel>Level4</WarningLevel>
    <MultiProcessorCompilation>true</MultiProcessorCompilation>
    <CharacterSet>Unicode</CharacterSet>
```

```
  </PropertyGroup>
  <ItemDefinitionGroup>
    <ClCompile>
      <AdditionalOptions>/utf-8 %(AdditionalOptions)</AdditionalOptions>
    </ClCompile>
  </ItemDefinitionGroup>
</Project>
```

注意 Directory.Build.props 是**从项目目录向上查找**并自动导入的，**不需要在项目里写 <Import>**。好处是统一，坏处是“这个 /W4 到底是哪来的”——排查时记得往上看目录树。属性表的导入顺序见 **第 5 章**。

7.15 从默认配置到生产配置：改动清单

配置项	Debug	Release	理由
优化	/Od	/O2	调试要准，发布要快
调试信息	/Zi	/Zi（保留 PDB 并归档）	线上崩溃要靠它分析
运行库	/MDd	/MD	与可再发行包配套
警告等级	/W4	/W4 + /WX	发布前零警告
多处理器编译	/MP	/MP	都受益
字符集	Unicode	Unicode	保持一致
C++ 标准	/std:c++20	/std:c++20	**两套必须一致**，否则行为不同
/GL + /LTCG	关	开	跨模块优化
/arch:	SSE2	SSE2（除非确定目标机支持）	避免非法指令崩溃
增量链接	开	关	发布构建要干净
调试属性页参数	本地路径	**清空**	不要把本机路径带进发布
/utf-8	开	开	中文一致

注意 **Debug 与 Release 的“语言标准”必须一致**。Debug 用 C++20、Release 用 C++17，会导致某些代码只在一种配置下能编译，或产生不同的重载决议——这是非常隐蔽的一类 bug。

7.16 小结

- **属性页改的是“当前配置 × 平台”**，改完记得切到 **所有配置**或逐个检查——这是最高频的踩坑点。
- **C++ 语言标准**：新建项目默认 `/std:c++20`；没有 `/std:c++26`；`/std:c++20` 与 `/std:c++latest` 会隐含 `/permissive-`。
- **运行库 `/MD` 与 `/MT` 必须全局一致**，混用会导致堆损坏；DLL 边界上的混用链接器发现不了。
- **`/GL` 与 `/LTCG` 必须成对**；`/Gy` 配合 `/OPT:REF` 能减小体积。
- `/utf-8` 是现代项目处理中文源码的推荐做法。
- **`/arch:AVX2` 在旧 CPU 上会直接崩溃**（`0xC000001D`），非必要不要开。
- **子系统选错**会报“找不到 `main`”；**配置类型选错**会报“必须定义入口点”。
- **`/DEBUG:FASTLINK` 已在 VS 2026 移除**，用 `/DEBUG:FULL`。
- 改 `.vcxproj` 的 `PreprocessorDefinitions` 时，**别漏掉 `%(PreprocessorDefinitions)`**。
- 团队统一配置优先用 `Directory.Build.props` 或**属性表**，而不是逐个项目点属性页。

7.17 练习题

1. 打开一个控制台项目，把“配置”切成 **所有配置**，统一把语言标准设为 `/std:c++20`、警告等级设为 `/W4`；然后逐个配置切过去确认都生效了。
2. 分别在 `/std:c++14` 与 `/std:c++20` 下编译同一段使用了 C++17 结构化绑定的代码，记录两次报错信息，并解释为什么。
3. 把项目的运行库从 `/MD` 改成 `/MT`，用 `dumpbin /dependents` 对比改前改后的依赖列表，说明差异。
4. 故意把“子系统”从 `CONSOLE` 改成 `WINDOWS`，观察报错；再改回来。解释链接器在找什么。
5. 为一个含 10 个以上 `.cpp` 的项目配置预编译头，记录开启前后的构建耗时；再故意让一个文件漏掉 `##include "pch.h"`，记录报错码。
6. 用 `/showIncludes` 找出某个源文件包含的头文件总数，再指出其中最“重”的两个并说明理由。
7. 写一份 `Directory.Build.props`，为某目录下所有项目统一开启 `/W4`、`/MP`、`/utf-8`，并验证新建的项目自动继承了这些设置。
8. 用“命令行”子页（或“所有选项”页）抄下 Debug|x64 与 Release|x64 两份完整命令行，逐项对比差异并解释每一项的作用。

7.18 相关笔记

- **本教程**：**Visual Studio 2026 使用教程**

- **上一章：项目与解决方案管理**
- **下一章：代码编辑与 IntelliSense**
- **发布与部署：发布与部署**
- **CMake：CMake 使用指南**

第八章　代码编辑与 IntelliSense

写 C++ 的时间里，真正在“想算法”的部分可能不到一半，剩下的都花在找符号、对名字、改调用点这些琐事上。Visual Studio 的编辑器就是冲着这些琐事来的：IntelliSense 负责读懂你的代码，导航与重构负责带你在几万个符号之间穿梭，格式化、代码片段与 AI 补全负责让你少敲几百次键盘。这一章把这些能力一次讲清楚，并把 VS 2026 在编辑体验上的新改进点出来。

8.1　IntelliSense 是什么，不是什么

8.1.1　它是另一套引擎，不是编译器

本章最重要的一句话：**IntelliSense 与 MSVC 编译器是两套完全独立的代码理解引擎**。补全、波浪线、快速信息、导航全部来自 IntelliSense（内部使用 **EDG 前端**解析），与你按下“生成”时跑的 `cl.exe` 不是一回事。

```
你的 .cpp / .h 源码
   |
   +-- IntelliSense (EDG 前端，容错解析)
   |      补全 / 波浪线 / 快速信息 / 导航
   |      只提示，不生成代码
   |
   +-- MSVC 编译器 cl.exe (严格，一次编译到底)
   |      真报错，真代码生成
   |
   +-- 两条路线给出的结论常常不一致：
         "波浪线报错，但能编译通过"
         "编译通过，波浪线还在"
```

8.1.2　三个必须接受的现象

现象	原因	怎么办
红波浪线报错，但生成成功	EDG 在模板、宏、`constexpr` 上比 MSVC 保守	编译通过就先用着；反复误报见第 13 章
编译报错，编辑器却一片干净	IntelliSense 还没跟上编辑，或解析路径与编译器不同	先看**错误列表**，必要时重新扫描解决方案

现象	原因	怎么办
刚打开大项目时提示“发虚”	全代码库语义索引还在建立，符号表不完整	等索引跑完，或手动重新扫描（见 9.5）

注意 **以编译器为准**。IntelliSense 的报错**不会**阻断“生成”，编译器的报错会。所以“能跑起来”不能反推“代码没问题”，反过来也一样——别把红波浪线当成编译失败的证据。

8.2 补全与提示

8.2.1 五条常用命令

命令	快捷键	菜单路径	作用
列出成员	`Ctrl+J`	**编辑 > IntelliSense > 列出成员**	弹出当前作用域可见的成员与变量
完成单词	`Ctrl+Space`	**编辑 > IntelliSense > 完成单词**	补全正在输入的标识符
参数信息	`Ctrl+Shift+Space`	**编辑 > IntelliSense > 参数信息**	显示重载列表并高亮当前参数
快速信息	`Ctrl+K, Ctrl+I`	**编辑 > IntelliSense > 快速信息**	鼠标悬停提示的键盘版本
快速操作	`Ctrl+.`	**编辑 > 快速操作和重构**	打开灯泡 / 螺丝刀建议列表

提示 `Ctrl+K, Ctrl+I` 这类**组合键**分两段按：先按 `Ctrl+K`，全部松开，再按 `Ctrl+I`。`Ctrl+Space` 是“把正在敲的词补完”，`Ctrl+J` 是“把能用的东西都列出来看看”，记不住名字时用后者。

8.2.2 内联提示（Inlay Hints）与悬停信息

内联提示把**本来不写在代码里、读代码时却需要知道的信息**直接渲染在标识符旁边，默认开启。按 `Alt+F1` 随时切换显示与隐藏；设置位于 **工具 > 选项 > 文本编辑器 > C/C++ > IntelliSense**（想换按键就到 **工具 > 选项 > 环境 > 键盘**里搜“内联提示”）。它主要显示**参数名提示**与**类型提示**（`auto`、结构化绑定推导出的真实类型）；这些提示只是“画上去的”，不会写进文件，也不影响编译。

```
auto count = items.size();              // 内联类型提示: size_t
for (auto& [key, value] : table) { }    // 内联类型提示: const std::string / int
draw(1920, 1080, true);                 // 内联参数名提示: width / height / antialias
```

8.3 导航

8.3.1 跳转命令总表

操作	快捷键	说明
转到定义	F12	跳到符号的定义处
转到声明	Ctrl+F12	跳到头文件里的声明处
速览定义	Alt+F12	就地弹出浮层查看定义，不离开当前文件
查找所有引用	Shift+F12	列出全解决方案的引用，可筛选读 / 写
转到所有	Ctrl+T 或 Ctrl+,	统一的搜索框式跳转入口
导航后退 / 前进	Ctrl+- / Ctrl+Shift+-	跳转后回原位，成对使用
类视图 / 书签窗口	Ctrl+Shift+C / Ctrl+K, Ctrl+W	按类组织代码（见 9.3）；打开书签列表（见 8.2）

Ctrl+T / Ctrl+, 的价值在于“我大概记得有这么个东西”——输入时实时过滤出**文件**、**类型**、**成员**、**符号**；只想要某一类结果时，**编辑 > 转到**菜单下有专门的**转到文件 / 转到类型 / 转到成员 / 转到符号**命令。

速览定义与 CodeLens：Alt+F12 打开的**速览定义**是一个**可以编辑的真编辑器**（只是嵌在浮层里），不是只读预览——读实现细节时就地看、顺手改，按 Esc 关掉，人还在原处，浮层里继续按 F12 还能再钻一层。改函数签名前的必做动作是 Shift+F12（**查找所有引用**），结果按项目 / 文件分组。**CodeLens** 则在每个函数、类上方显示**引用计数**与作者信息，点计数就地展开引用列表；VS 2026 的 C++ CodeLens 引用依赖全代码库语义索引（见 9.5），索引跑完计数才准，想关掉它去 **工具 > 选项 > 文本编辑器 > 所有语言 > CodeLens**。编辑器顶部由三个下拉框组成的**导航栏**（左选类型 / 命名空间，中选成员，右切重载）适合在长文件里定位成员，开关在 **工具 > 选项 > 文本编辑器 > 所有语言 > 常规 > 导航栏**。

> **实战场景** 读陌生函数时用 Alt+F12 一层层往下“扒”，比反复开文件、跳来跳去再按十几次后退快得多，而且不污染导航历史。

8.4 重构

8.4.1 重命名：Ctrl+R, Ctrl+R

Ctrl+R, Ctrl+R（或 Ctrl+. 里选重命名）打开重命名面板，输入新名字后回车：

- **改之前会预览**，可以展开确认每一处改动；
- **可以带上注释与字符串**里的同名内容一起改；
- **重命名类型时可以同步改文件名**，把 Foo 改成 Bar 时顺手把 Foo.h / Foo.cpp 一起改名。

8.4.2 创建声明 / 定义、移动定义位置与其它重构

在头文件里写了声明还没写实现时，把光标放在函数名上按 Ctrl+. 会出现**创建定义**；反过来在 .cpp 里对着实现按 Ctrl+. 可以**创建声明**；已有实现想挪进头文件，用**移动定义位置**。

> **提示** 执行“创建声明 / 定义”后编辑器要不要跳过去是可以配置的——选项里提供**速览**、**打开文档**、**不导航**三种行为（配置入口在 **工具 > 选项 > 文本编辑器 > C/C++** 下）。写代码快的人通常选“速览”，批量补实现时选“不导航”。

操作	入口	说明
提取函数	Ctrl+. > 提取函数	选中一段代码抽成新函数，自动处理参数与返回值
转为原始字符串字面量	Ctrl+. > 转为原始字符串	把满是 \\ 的路径变成 R"(C:\data\out)"
添加 / 移除 ##include	Ctrl+.	补上缺失的直接包含，或清理未使用的包含，见 5.3

按 Ctrl+. 弹出的图标有两种含义：**灯泡**是“快速操作”，会改变代码结构或行为（创建定义、提取函数、添加 ##include）；**螺丝刀**是“建议”，只影响代码**风格 / 质量**、不改变语义（成员函数加 const、全局函数加 static）。第 5 节讲的就是 C++ 特有的螺丝刀建议。

8.5 C++ 专属的智能建议

8.5.1 建议把不修改状态的成员函数标记为 const

```
1 class Counter {
2 public:
3     int Value() { return value_; }   // 螺丝刀：没改状态，可以加 const
4     void Add(int n) { value_ += n; }
5 private:
6     int value_ = 0;
7 };
```

改成 int Value() const 之后它就能被 const Counter& 调用了。这不是洁癖：**漏写 const 是“能编译但用不了”的典型原因**——某个函数接 const 引用，你却在里面调了非 const 成员函数，报错信息还很难看。

8.5.2 建议把只在本文件使用的全局函数标记为 static

在 .cpp 里定义、又只被本文件调用的全局函数（例如 static int clamp01(int v) { ... }），编辑器会建议加 static 改为内部链接。好处有两个：**避免符号名冲突**（两个 .cpp 里各写

一个同名函数也不会撞车）与给编译器更多优化空间。同样的建议也适用于只在本翻译单元使用的全局常量与变量，更彻底的做法是放进匿名命名空间。

8.5.3 未使用的 #include 会变灰

一个 ##include 如果在**当前翻译单元里没被用到**，它的文字就会变**灰色**——这是排查“编译时间为什么这么长”的第一现场。

```
#include <vector>    // 用到了，正常颜色
#include <string>    // 没用到，显示为灰色
#include <map>       // 没用到，显示为灰色

void f(const std::vector<int>& v) { (void)v; }
```

把光标放到灰行上按 Ctrl+.，可以**移除这一个**，也可以**一次移除当前文件中所有未使用的 ##include**。反方向的能力同样有：使用了只被间接包含进来的符号时（例如没直接包含 <algorithm> 就调用了 std::sort），编辑器会建议**添加对应的 ##include**。

> **注意** 别盲目相信“灰掉就没用”。宏、模板特化、static_assert 里用到的符号可能不被 IntelliSense 统计到；头文件里删 ##include 尤其要谨慎——它可能一直是某个 .cpp 赖以工作的**传递依赖**。**删完立刻生成一次**，让编译器兜底。

8.5.4 大小与对齐预览

把鼠标悬停在一个类 / 结构体 / 联合体 / 枚举上，快速信息里会给出**大小与对齐**：

```
struct Packet {
    char   tag;      // 偏移 0
    int    seq;      // 偏移 4（tag 之后有 3 字节填充）
    double payload;  // 偏移 8
};                   // 大小 16 字节，对齐 8 字节
```

它在三种场合非常值钱：给结构体加字段前评估体积变化、按二进制格式收发数据时核对布局、多线程代码里判断两个对象会不会落在同一个缓存行上。要更严格地钉死布局，可配合 ##pragma pack 或 alignas，再回来悬停复查。

8.6 代码片段

8.6.1 展开与插入

方式	快捷键 / 操作	说明
输入片段名 + Tab	例如输入 for 后按 Tab	最常用，也可在补全列表里挑带“代码片段”图标的候选

方式	快捷键 / 操作	说明
插入代码片段	Ctrl+K, Ctrl+X	从列表里挑一个片段插入
外侧代码	Ctrl+K, Ctrl+S	用片段把选中代码包裹起来，如包进 ##if

C++ 内置片段常用的有 ##if（条件编译骨架）、for、while、switch、class、struct、enum、main。展开后用 Tab 在可替换字段之间跳，Enter 或 Esc 结束编辑。

8.6.2 代码片段管理器与自定义片段

工具 > 代码片段管理器是片段的总入口：左侧选语言（选 **Visual C++**），能看到每个片段的位置路径，右下角有**导入**按钮。自定义片段有两条路：在管理器里用**导入**按钮选择 .snippet 文件，或直接把它拷进用户片段目录（管理器里会显示具体路径，通常是“文档”下的 Visual Studio 2026\Code Snippets\Visual C++\My Code Snippets）。下面这个片段叫 cwout，展开后打印“变量名 = 值”：

```
<?xml version="1.0" encoding="utf-8"?>
<CodeSnippets xmlns="http://schemas.microsoft.com/VisualStudio/2005/CodeSnippet">
  <CodeSnippet Format="1.0.0">
    <Header>
      <Title>cwout</Title>
      <Shortcut>cwout</Shortcut>
      <Description>把变量名与值一起输出到 std::cout</Description>
      <SnippetTypes><SnippetType>Expansion</SnippetType></SnippetTypes>
    </Header>
    <Snippet>
      <Declarations>
        <Literal><ID>var</ID><ToolTip>要打印的变量</ToolTip><Default>x</Default></Literal>
      </Declarations>
      <Code Language="cpp" Delimiter="$"><![CDATA[std::cout << "$var = " << $var << '\n';$end$]]></Code>
    </Snippet>
  </CodeSnippet>
</CodeSnippets>
```

几个必须注意的点：扩展名是 .snippet，用 **UTF-8** 保存（否则中文描述乱码）；<Shortcut> 是展开时输入的名字，<Title> 是列表里显示的名字；var 是可替换字段，end 是展开后光标停留的位置；<Code Language="cpp"> 指明这是 C++ 片段，XML 结构少一层就不会被识别。

8.7 格式化与代码风格

8.7.1 格式化快捷键

操作	快捷键	说明
格式化文档	`Ctrl+K, Ctrl+D`	按当前风格设置重排整个文件
格式化选区	`Ctrl+K, Ctrl+F`	只排选中部分；输入 `;`、`}` 时编辑器也会自动调整缩进

8.7.2 `.editorconfig`：把风格写进仓库

`.editorconfig` 是与编辑器无关的纯文本约定文件，放在仓库里跟着 Git 走。Visual Studio 会读取它并让它**覆盖个人的 IDE 设置**，团队里每个人的缩进、换行、编码就都一致了，代码评审里也不会再出现“整篇都是改动”的假 diff。规则是**就近生效**：子目录里的配置覆盖上层，`root = true` 表示到此为止不再往上找。

键	取值	作用
`indent_style`	`space` | `tab`	用空格还是制表符缩进
`indent_size`	整数	一级缩进的宽度
`end_of_line`	`lf` | `crlf` | `cr`	换行符类型，跨平台协作的关键
`charset`	`utf-8` | `utf-8-bom` | `gbk` …	文件编码，**解决乱码的第一道防线**
`trim_trailing_whitespace`	`true` | `false`	保存时删除行尾空白
`insert_final_newline`	`true` | `false`	文件末尾补一个空行

一份实用的 C++ 仓库配置：

```
# 仓库根目录，不再向上级目录查找
root = true

[*]
charset = utf-8
end_of_line = crlf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 4

[*.{cpp,cc,cxx,h,hpp,inl,ipp}]
indent_size = 4
max_line_length = 100
```

```
# Markdown 里行尾两个空格是有意义的换行，别删
[*.md]
trim_trailing_whitespace = false
```

提示 `charset = utf-8` 只约束**新建文件与格式化时的编码**，已经在磁盘上存成 GBK 的老文件不会自动转换，还得靠 9.2 的编码控件手工转一次。

8.7.3 clang-format 集成

团队里已有 `.clang-format` 时不必维护两套风格。在 **工具 > 选项 > 文本编辑器 > C/C++ > 格式设置**里启用 ClangFormat 支持后，只要项目目录中存在 `.clang-format`，`Ctrl+K, Ctrl+D` 就会按它的规则排版，并不再使用 VS 自己的缩进 / 空格设置。

```
# .clang-format
BasedOnStyle: LLVM
Language: Cpp
Standard: c++20
IndentWidth: 4
ColumnLimit: 100
PointerAlignment: Left
SortIncludes: true
```

注意 `BasedOnStyle` 决定基线（`LLVM` / `Google` / `Microsoft` 等），`ColumnLimit` 决定折行宽度，`PointerAlignment` 决定 `int* p` 还是 `int *p`；`SortIncludes` 会重排 `#include`，而**头文件之间的顺序有时是有语义的**，顺序敏感的文件可以用 `// clang-format off` / `// clang-format on` 圈起来。

提示 格式化用 clang-format，**静态检查**用 clang-tidy，两者的配置是分开的：clang-tidy 的开关与检查项在 **项目属性 > 代码分析 > Clang-Tidy** 里配置。

8.8 编辑器效率技巧

8.8.1 多光标与行操作

操作	快捷键 / 方式	说明
加一个光标	`Ctrl+Alt+` 单击	在点击处再插一个插入点，可加多个，`Esc` 取消

操作	快捷键 / 方式	说明
框选（列选择）	Alt+ 拖动	按矩形区域选择，改对齐的数据列极快
复制行 / 移动行	Ctrl+D / Alt+↑、Alt+↓	复制当前行到下方；上下挪动当前行或选中块
删除行 / 剪贴板环	Ctrl+Shift+L / Ctrl+Shift+V	删除当前行；在历史剪贴内容之间循环粘贴
注释 / 取消注释 / 大小写	Ctrl+K，Ctrl+C、Ctrl+K，Ctrl+U / Ctrl+Shift+U、Ctrl+U	对选中块整块加 // 或去掉；改宏名、常量名时顺手
跳到匹配括号	Ctrl+]	光标放在 {、(上跳到配对的另一半，也能在 ##if / ##endif 间跳

典型用法：把一组 int x1, x2, x3; 里的 int 全改成 long long——用 Alt+ 拖动框住那一列直接输入替换。

8.8.2 任务列表、书签、折叠与结构参考线

在注释里写下约定令牌，它就会出现在 **视图 > 任务列表**里，默认令牌包括 TODO、HACK、UNDONE、UnresolvedMergeConflict：

```
// TODO: 把这里换成 std::string_view，避免一次拷贝
// HACK: 依赖了未定义行为，等上游修复后删掉
```

在 **工具 > 选项 > 环境 > 任务列表**里可以添加自己的令牌（比如 FIXME）并设置**优先级**；双击条目直接跳到代码位置。它按令牌解析，比满仓库搜 TODO 靠谱得多，不会误命中字符串里的普通文本。

书签适合“这几个地方等会儿都要回来改”：Ctrl+K，Ctrl+K 切换书签，Ctrl+K，Ctrl+N / Ctrl+K，Ctrl+P 前后跳，Ctrl+K，Ctrl+W 打开书签窗口（可改名、分组）。**折叠**适合收拾长文件：Ctrl+M，Ctrl+O 折叠到定义（只留函数签名，一屏看完文件结构），Ctrl+M，Ctrl+M 折叠 / 展开当前块，Ctrl+M，Ctrl+L 全部展开或折叠，Ctrl+M，Ctrl+P 停止大纲显示。**结构参考线**是编辑器左侧那些淡色竖线，用来指示代码块的嵌套层级，开关在 **工具 > 选项 > 文本编辑器 > 常规 > 显示结构参考线**。

8.9 VS 2026 的编辑体验改进

8.9.1 C++ 属性语法高亮与快捷键统一

[[nodiscard]]、[[maybe_unused]]、[[likely]]、[[deprecated]] 这类 C++ 属性在 VS 2026 里终于有了独立着色，不再和普通标识符糊在一起，属性密集的现代 C++ 代码可读性提升明显；颜色可在 **工具 > 选项 > 环境 > 字体和颜色**里单独调整。这一代还统一了一批按键约定，让**快捷键向其他编辑器看齐**，从 VS Code 迁过来的人会更顺手（多光标、复制行、行移动这几类尤其明显）。如果反而改不回肌肉记忆，**工具 > 选项 > 环境 > 键盘**里有**键盘方案**下拉框，可以直接切换到其他编辑器的键位方案。

8.9.2 右下角增强控件：查看与修改文件编码

编辑器**右下角**的编码指示器现在可以直接点击操作，这是解决中文乱码最顺手的一条路。把一份 GBK 老文件转成 UTF-8 的完整步骤：

1. 打开乱码文件，先看右下角显示的编码（例如 **GB2312**，或带 BOM 的 **UTF-8 with BOM**）；
2. **单击**这个编码名，在菜单里选**使用编码重新打开**，从列表里挑一个候选（中文 Windows 上通常是 GB2312 或 UTF-8），用**预览**确认注释已能正常显示；
3. 显示正确后再次单击编码名，选**通过编码保存**，把它真正存成 UTF-8；
4. 编译期再补一刀：在 **项目属性 > C/C++ > 命令行**的“其他选项”里加上 `/utf-8`（等价于同时指定源码字符集与执行字符集），让 MSVC 明确按 UTF-8 解读源文件。

你看到的乱码	常见成因
锟斤拷	文件本是 UTF-8 却被按 GBK 打开（或反之）
`????` 或方块 / `C2001: 常量中有换行符`	前者是保存时目标编码无法表示这些字符（信息已丢失）；后者是源文件编码与编译器默认代码页不一致，字符串字面量被截断

实战场景 从同事那里拷来一批 `.cpp`，注释全是 锟斤拷。不要逐个用记事本“另存为”——在 VS 里逐个走“重新打开 → 通过编码保存”两步即可，改完把 `.editorconfig` 的 `charset` 定为 `utf-8`，下次就不会再犯。

8.9.3 查找排除文件、类视图子串搜索

Quick Find（`Ctrl+F`）与 Find in Files（`Ctrl+Shift+F`）都支持**排除**条件：在查找窗口里把不想搜的路径或通配符填进排除项，例如 `third_party/`、`build/`、`out/`、`*.generated.h`。这解决的是大型仓库里很实在的痛点——搜一个 `CreateWindow`，结果前两百条全来自第三方库和生成代码。

视图 > 类视图（`Ctrl+Shift+C`）的搜索框现在支持**子串匹配**：输入 `render` 就能命中 `DeferredRenderer`、`RendererFactory`，不必从单词开头对齐；配合 VS 2026 对大型项目的性能优化，几万个类型的解决方案里过滤与展开也基本不卡。

8.9.4 右键一键生成预处理输出

在**解决方案资源管理器**里对着一个 `.cpp` 文件右键，选择**预处理**，IDE 会立刻生成预处理之后的完整输出（默认落在项目的中间目录下，如 `x64\Debug`）。它对应编译开关 `/P`（保留 `##line` 标记，便于对照原始行号；`/EP` 则去掉行标记，只留纯代码）。三个最有价值的场景：**宏套娃看不懂**（展开后 `MAX(1+2, 3)` 变成什么一目了然）、**查某个头文件是谁包含进来的**（在输出里搜 `##line` 附近的文件名，一路倒推包含链）、**模板 / 宏相关的编译错误定位**（编译器报的位置在

宏里，输出能还原成实际参与编译的那一行）。输出动辄几万到几十万行，用 Ctrl+F 查找，别滚动着看。

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int v = MAX(1 + 2, 3);   // 预处理后：((1 + 2) > (3) ? (1 + 2) : (3))
// 实参被原样替换两次 —— 若实参带副作用（如 i++），就会被执行两遍
```

8.9.5 全代码库语义索引

VS 2026 为 C++ 引入了**全代码库语义索引**：把整个解决方案的符号关系建成索引后，导航（F12、Ctrl+T、Shift+F12）与语法着色的性能相比以前**提升 2 倍以上**，C++ 的 CodeLens 引用计数也依赖它。配置位于 **工具 > 选项 > C/C++ > IntelliSense > 浏览和导航**。

- 首次打开大型解决方案需要一段建立索引的时间，**此期间补全与跳转质量会下降**——这正是 1.2 里“提示发虚”的常见原因；
- 索引结果缓存在解决方案的隐藏目录（.vs）里，**删掉 .vs 会触发全量重建**，非必要不要删；
- 换了工具集、改了包含路径结构之后，**生成 > 重新扫描解决方案**比干等更快。

8.10 GitHub Copilot 在编辑器里的用法

8.10.1 行内补全与 Copilot Chat

行内补全：继续输入时 Copilot 用灰色“幽灵文本”给出整行或整段建议，Tab 接受，Esc 拒绝，继续打字则自动忽略；写一堆相似的 getter、逐字段的序列化代码、样板测试时最有用。VS 2026 中 C++ 的补全质量有提升——Copilot 会把相关文件作为上下文，减少凭空编造 API 的情况。

Copilot Chat（**视图 > GitHub Copilot Chat**）有两种模式：

模式	做什么	适合的问题
Ask	只回答，不动你的代码	“这段代码为什么会有数据竞争?”“这个模板报错是什么意思?”
Agent	直接在解决方案里改代码、建文件、跑命令	“给 Packet 加上序列化与反序列化，并补上单元测试”

用 Agent 模式要记住两点：它会**同时改多个文件**，改完要逐个审阅 diff 再接受；对不熟悉的仓库，先让它“只解释、先别改”。

8.10.2 右键 Copilot Actions 与 Next Edit Suggestions

在编辑器里**选中一段代码 → 右键**，VS 2026 的右键菜单里新增了一组 **Copilot Actions**：**解释代码**（读陌生代码最实用）、**优化**（性能或可读性建议）、**生成注释**（注意审阅，注释错了比没有更糟）、**生成单元测试**（生成测试脚手架，断言还得自己补）。**Next Edit Suggestions** 则

会预判你下一处要改的地方：刚改了某个 API 的调用方式，它提示“接下来这里也要改”，按 `Tab` 跳过去接受即可，做机械式批量修改（加一个参数并同步所有调用点）时效率很高。开关位于 **工具 > 选项 > GitHub > Copilot**。

8.10.3 隐私与关闭方法

- **关闭入口**：**工具 > 选项 > GitHub > Copilot**，在这里可以关掉行内建议、关闭 Copilot Chat。关掉之后 IDE 其它功能完全不受影响——IntelliSense、导航、重构都是本地能力，与 Copilot 无关。
- **临时暂停**：编辑器右下角有 Copilot 状态指示，可以就地暂停建议，不必进选项页。
- **BYOK / BYOM（自带密钥 / 自带模型）**：可在 Copilot Chat 的模型选择器里接入 Microsoft Foundry、OpenAI、Anthropic、Ollama 等提供程序；**接入本地 Ollama 是代码完全不出网的一种折中方案**，但能力与云端模型有差距。
- **内容排除**：仓库级 / 组织级的排除规则由 GitHub 侧配置，需与管理员确认，不要假设“默认不会上传”。

注意 启用 Copilot 意味着**你正在编辑的代码片段会被发送到服务端**以生成补全请求。合规要求“代码不出内网”时，正确做法是整体关掉它，而不是只关 Chat、留着行内补全。

8.11 常见 IntelliSense 问题与快速处置

这里只列现象和一句话方向，完整排查步骤见 **第 13 章**。

现象	一句话处置
波浪线报错，但生成成功	以编译器为准，误报成因与逐项处置见第 13 章
满屏“无法打开源文件 xxx.h”	包含路径与项目属性不一致，核对包含目录并重新扫描，详见第 13 章
补全列表空、`F12` 跳错地方	IntelliSense 数据库与代码不同步，重建浏览数据库，详见第 13 章
打开后长时间“正在分析”、CPU 占用高	全代码库语义索引正在建立，先等它跑完，详见第 13 章
中文注释导致解析异常、报奇怪的错	文件编码不一致，先用 9.2 的编码控件统一为 UTF-8

8.12 编辑效率速查表

提示 Ctrl+K, Ctrl+D 这类写法表示**分两段按**——先按 Ctrl+K，全部松开，再按 Ctrl+D；顺序反了不会生效。

类别	快捷键	命令	备注
导航	F12 / Ctrl+F12 / Alt+F12	转到定义 / 声明 / 速览定义	每天按几百次，加 Ctrl+- 回原位
导航	Shift+F12	查找所有引用	改签名前的必做动作
导航	Ctrl+T 或 Ctrl+,	转到所有	文件 / 类型 / 成员 / 符号
导航	Ctrl+Shift+C	类视图	支持子串搜索（VS 2026）
编辑	Ctrl+J / Ctrl+Space / Ctrl+Shift+Space	列出成员 / 完成单词 / 参数信息	记不住名字时用 Ctrl+J
编辑	Ctrl+D / Alt+↑、Alt+↓	复制行 / 移动行	不用先选中整行
编辑	Ctrl+K, Ctrl+C / Ctrl+K, Ctrl+U	注释 / 取消注释	对选中块生效
编辑	Ctrl+K, Ctrl+D / Ctrl+K, Ctrl+F	格式化文档 / 选区	提交前跑一次
编辑	Ctrl+Alt+ 单击 / Alt+ 拖动 / Alt+F1 / Ctrl+M, Ctrl+O	多光标 / 框选 / 内联提示 / 折叠到定义	批量同改、截图前关闭、长文件导航
重构	Ctrl+R, Ctrl+R	重命名	可带注释与字符串，可同步改文件名
重构	Ctrl+.	快速操作（灯泡 / 螺丝刀）	一切建议的总入口
重构	Ctrl+. > 创建定义 / 创建声明	生成缺失的另一半	导航行为可选速览 / 打开文档 / 不导航
重构	Ctrl+. > 移动定义 / 提取函数 / 转原始字符串 / 增删 ##include	搬移、抽取与依赖增删	未使用的 ##include 会变灰
搜索	Ctrl+F / Ctrl+H / Ctrl+Shift+F / Ctrl+Shift+H	快速查找 / 替换；在文件中查找 / 替换	后两者可排除 third_party/、build/（VS 2026）
搜索	**视图 > 任务列表**	TODO / HACK 清单	令牌在 **工具 > 选项 > 环境 > 任务列表**配置

8.13 小结

- **IntelliSense 与 MSVC 是两套引擎**：波浪线报错不等于编译失败，反过来也一样——**以编译器为准**。
- 补全与提示的四个核心键：`Ctrl+J`（列出成员）、`Ctrl+Shift+Space`（参数信息）、`Alt+F1`（切换内联提示）、`Ctrl+K, Ctrl+I`（快速信息）。
- 导航体系：`F12` / `Ctrl+F12` / `Alt+F12` 三兄弟，加上 `Shift+F12` 找引用、`Ctrl+T` 找符号、`Ctrl+-` 回原位。
- C++ 专属的螺丝刀建议很值钱：**成员函数加 `const`、全局函数加 `static`、未使用的 `#include` 变灰可一键清理、悬停看大小与对齐**。
- 风格统一靠 `.editorconfig`（随仓库走、就近生效）与 `.clang-format`（在选项里启用后接管格式化）。
- VS 2026 的编辑新能力记住五个：**C++ 属性高亮、右下角改编码、查找排除文件、右键生成预处理输出、全代码库语义索引**（导航与着色快 2 倍以上）。
- Copilot 的开关在 **工具 > 选项 > GitHub > Copilot**；合规要求代码不出网时，**整体关掉**才是正确做法。

8.14 练习题

1. 写一个只读不写状态的成员函数，观察螺丝刀建议并接受它加上 `const`；再把这个对象以 `const` 引用传给另一个函数，验证加 `const` 前编译失败、加之后通过。
2. 在一个 `.cpp` 里故意包含三个用不到的头文件，用 `Ctrl+.` 的“移除所有未使用的 `#include`”清掉，然后生成一次，确认没有误删；如果有问题，记录是哪个符号导致的。
3. 写一个字段依次为 `char`、`int`、`double` 的 `struct`，悬停查看它的大小与对齐；再调整字段顺序把大小降到最小，并解释填充字节是怎么来的。
4. 定义一个带参数的宏，分别用不带副作用和带副作用的实参调用它，用**右键 > 预处理**生成预处理输出，找出副作用被执行多次的证据。
5. 在仓库根目录放一份 `.editorconfig`（至少包含 `indent_style`、`indent_size`、`end_of_line`、`charset`、`trim_trailing_whitespace`、`insert_final_newline`），新建 `.cpp` 验证生效；再在子目录放一份不同配置，验证“就近生效”。最后写一个自定义代码片段（例如打印变量名与值的 `cwout`），通过 **工具 > 代码片段管理器**导入并展开一次；如果没生效，检查 XML 结构、`Language` 属性与文件编码。

8.15 相关笔记

- **本教程**：**Visual Studio 2026 使用教程**
- **上一章**：**项目属性与编译配置**
- **下一章**：**调试技术详解**

- **常见问题**：**常见问题与效率技巧**
- **C++ 教程**：**C++ 教程**

第九章　调试技术详解

调试的本质是“在程序运行时观察它”，而 Visual Studio 的调试器把这件苦差事变成了一套可以自由组合的工具：断点决定在哪里停下，单步决定怎么走，数据窗口决定看什么，调用堆栈决定“我是从哪儿来的”。这一章按实际使用顺序把 C++ 调试的功能串成一条线，从最基础的 F5 一路讲到数据断点、转储分析与多线程死锁排查。读完它，你应该能对任何一个崩溃或数值异常说清楚“下一步该打开哪个窗口”。

9.1　调试的基本流程

9.1.1　调试运行与“不调试运行”

F5 是最常用的启动方式：**调试 > 开始调试**。它会先编译，再把调试器附加到新启动的进程上，加载符号，然后停在第一个命中的断点处。此时进程处于**中断模式**（break mode），所有调试窗口才有内容；按 F5 继续后回到**运行模式**，大部分窗口会变灰。会话控制还有三个命令：Shift+F5 停止调试，Ctrl+Shift+F5 重启调试（比“停止再 F5”快，因为不必重新附加），**调试 > 全部中断**用来打断正在死循环或死锁的程序。Ctrl+F5（**调试 > 开始执行（不调试）**）则只是把程序跑起来，不附加调试器，二者的差别远不止“能不能下断点”：

对比项	F5 调试运行	Ctrl+F5 不调试运行
调试器	已附加，可随时中断	未附加
断点 / 单步 / 异常	有效，可在异常抛出时中断	完全无效，只按程序自身逻辑处理
调试堆与符号	启用调试堆，按设置加载.pdb	都不启用
速度与适用场景	明显更慢，用于定位问题	接近真实速度，用于只看结果

提示 判断“程序本身有 bug”还是“只在调试器下才出错”时，就用 Ctrl+F5 跑一遍。调试堆会把已释放的内存填成 0xDD、未初始化的堆内存填成 0xCD，有些野指针在调试器下“幸运地”崩掉，脱离调试器反而活得很好，这类现象本身就很有价值。

9.1.2　调试符号（.pdb）与 Debug 配置

.pdb（Program Database）保存符号名、类型信息以及源文件与行号的映射，它**不包含**源代码，也不包含可执行代码；编译器用 /Zi 生成它，链接器用 /DEBUG 把它与 PE 文件关联。调

试器靠.pdb 里的 GUID + Age 与二进制严格配对，所以“断点变成空心圆圈，提示‘当前不会命中断点，还没有为该文档加载任何符号’”，九成是这四种情况之一：工程没开 /Zi 或.pdb 没被复制到 exe 旁边；exe 与.pdb 来自两次不同的编译；断点所在模块尚未加载，或那段代码根本没被执行到；符号路径没配对。

用 **调试 > 窗口 > 模块**可以查看每个模块的符号加载状态，在模块上右键可以手动指定.pdb。系统 DLL 的符号不必自己找：在 **工具 > 选项 > 调试 > 符号**里打开“Microsoft 符号服务器”（地址形如 https://msdl.microsoft.com/download/symbols）即可，第一次会慢一些，之后有本地缓存；在**即时窗口**里也能用 .sympath、.reload 命令调整路径并强制重载。

Visual Studio 默认的 Debug 配置对 C++ 施加了一组互相配合的开关，它们的组合决定了“调试器能看到多少真相”：

开关	含义	对调试的影响
/Od	禁用优化	变量不会被优化掉，语句顺序与源码一致，单步不乱跳
/Zi	生成完整调试信息到.pdb	断点能绑定到行号，类型信息可展开
/ZI	“用于编辑并继续”的调试信息	要边调试边改代码时必须用它
/RTC1	运行时检查（/RTCs + /RTCu）	栈帧填充为 0xCC，未初始化局部变量一眼可见
/JMC	仅我的代码单步支持	默认开启，决定 F11 会不会进入库函数
/MDd	调试版 CRT	启用调试堆与内存检查

优化是调试最主要的敌人：内联让“逐语句”一下跨过好几个函数，寄存器分配让变量显示“无法获取值”，循环展开让行号来回跳。必须在 Release 下排查时，至少加上 /Zi 并保留链接器的 /DEBUG:FULL，再用 **调试 > 窗口 > 反汇编**辅助理解。几个“填充图案”值得记住：**0xCCCCCCCC** 是 Debug 下未初始化的栈内存（/RTCs 的功劳），**0xCDCDCDCD** 是未初始化的堆内存，**0xDDDDDDDD** 是已被释放的堆内存，**0xFDFDFDFD** 是调试堆块两端的守护字节。

9.1.3 调试器窗口布局与常用快捷键

```
中断模式下的一屏（窗口之间可以自由停靠）
  +-- 左上：调用堆栈
  |     main()
  |     > BumpAll(int*, int)
  |     [外部代码]
  +-- 中间：源代码编辑器
  |     data[i] += 1;          <-- 黄色箭头（即将执行的语句）
  +-- 右上：监视 1 / 监视 2
  |     g_buf.canary
  |     found   0x0000..
  |     i       4
  +-- 下方选项卡：局部变量 / 自动 / 断点 / 异常设置 / 输出 / 即时窗口
```

```
|                内存 1 / 反汇编
+-- 顶部：调试工具栏 [继续] [全部中断] [停止] [重启]
      菜单：调试 > 窗口 > ...
```

黄色箭头表示“即将执行的语句”，它还没有被执行；中断模式下，这一屏里的窗口是你唯一的信息来源。

窗口	快捷键	窗口	快捷键
断点 / 调用堆栈	`Ctrl+Alt+B` / `Ctrl+Alt+C`	异常设置 / 模块	`Ctrl+Alt+E` / `Ctrl+Alt+U`
监视 1–4	`Ctrl+Alt+W`，再按 1–4	局部变量 / 自动	`Ctrl+Alt+V`，再按 `L` / `A`
即时 / 输出	`Ctrl+Alt+I` / `Ctrl+Alt+O`	线程 / 并行堆栈	`Ctrl+Alt+H` / `Ctrl+Shift+D, S`
内存 / 寄存器 / 反汇编	`Ctrl+Alt+M`（再按 1–4）/ `G` / `D`	打开方式都在 **调试 > 窗口** 下	—

9.2 断点体系

9.2.1 普通断点与断点图标

`F9`（**调试 > 切换断点**）在光标所在行插入或删除断点，也可以直接点击代码左侧的边距；`Ctrl+F9` 切换启用 / 禁用，`Ctrl+Shift+F9` 删除全部断点。图标状态本身就在传递信息：**红色实心圆**表示已启用且已绑定到代码；**空心圆**表示已禁用；**空心圆加警告**表示未绑定，悬停可看原因；**带加号的圆**表示还附带条件、命中次数、筛选器或标签。

9.2.2 条件断点与命中次数

右键断点选择“条件”（快捷键 `Alt+F9` 后按 `C`），或悬停断点时点齿轮图标，打开“断点设置”窗口，里面有“条件表达式 / 命中次数 / 筛选器”三项。条件表达式有两种语义：**为 true**，如 `i == 4`、`p != nullptr && p->id == 3`；**更改时**，只要表达式取值发生变化就中断——注意调试器不把第一次求值算作“变化”，所以它不会在第一轮循环就停下。命中次数则跳过前 N 次，可选“等于 / 大于 / 大于等于 / 是 N 的倍数”，查“第 1000 次迭代之后才出错”这类 bug 时它比条件表达式更快，因为不必每次迭代都求值。

> **注意** 条件表达式由调试器在目标进程里求值，每次命中都要算一遍，放在热循环里会显著拖慢程序。尽量只写纯变量比较，避免在条件里调用函数——带有副作用的表达式可能被调试器拒绝求值，或者干脆改变程序行为。

9.2.3 筛选器断点

同一个函数被主线程和 8 个工作线程反复调用时，条件表达式帮不上忙，该筛选器上场了。它支持 `ThreadId`、`ThreadName`、`ProcessId`、`ProcessName`、`MachineName` 五个字段，例如 `ThreadName = "Worker"`、`ProcessId = 6789`；多个子句用 `&`（与）、`||`（或）、`!`（非）和括号组合，字符串值必须加双引号。这是多线程调试里性价比最高的一招：先给线程起个名（见 5.3），再用 `ThreadName` 把断点限定住。

9.2.4 函数断点

调试 > 新建断点 > 函数断点（`Ctrl+K, B`）适合“知道函数名但不想翻文件”或“同名重载函数想全部拦下”的场景。函数名可以写成全限定名 `MyNamespace::MyClass::Method` 来消除歧义，可以带参数类型 `Method(int, char*)` 来区分重载，可以用 `util.dll!Method` 或 `{Method, , util.dll}` 指定模块，也可以用 `{Method, , util.dll}+2` 跳过函数序言直接停在函数体，甚至写成 `((MyClass*)0x0000000123456780)->Method` 只在这个对象实例上中断。调用堆栈窗口里也可以右键某一帧的函数名插入断点——“我只想在这个调用路径上停下来”时非常顺手。

9.2.5 数据断点（监视地址被改写）

数据断点监视的是**某块内存被写入**这件事本身，而不是某一行代码；当变量地址在运行时才确定，或者你怀疑有人偷偷改了一块内存时，它是唯一的选择。创建方式有两种：**调试 > 新建断点 > 数据断点**，在“地址”里填求值结果为地址的表达式（如 `&g_buf.canary`）；或者在自动、监视、局部变量窗口里右键变量选择“值更改时中断”。之后在“字节数”下拉框里选择要监视的宽度。限制条件必须记牢：

限制	说明
只在本机 C++ 生效	原生代码专属；托管代码的数据断点限制更多，静态变量、结构体字段、带 DebuggerTypeProxy 特性的类都不支持
只在写入时中断	**读取不中断**；排查数组越界读必须换手段（条件断点、ASan 之类）
数量上限	x64 / x86 最多 4 个，ARM64 最多 2 个，ARM 最多 1 个（由硬件调试寄存器数量决定）
宽度与对齐	x86 下最宽 4 字节，x64 下最宽 8 字节；地址需按监视宽度对齐——监视 4 字节要求 4 字节对齐，监视 8 字节要求 8 字节对齐
无效场景	别的进程改这块内存、内存被内核态改写（例如 `ReadFile` 填缓冲区）、共享内存，都不会触发

注意 数据断点绑定的是**绝对地址**，而每次重新调试变量地址几乎都会变，所以这类断点在调试会话结束时会被自动禁用，下次必须重新设置。同理，设在局部变量上的数据断点在该函数返回后地址就失效了，应该在返回前删除或禁用它。

9.2.6 临时断点、依赖断点与跟踪点

- **临时断点**：在边距右键选择“插入临时断点”，或在断点设置里勾选“命中一次后删除”，它只拦一次，等价于“把 F5 按到那一次为止”的自动化版本。
- **依赖断点**：在断点设置里勾选“仅当以下断点命中时启用”并选择一个前置断点，解决“这个工具函数被调用几千次，我只关心从某个入口进来的那一次”，比在条件表达式里写复杂的上下文判断干净得多。
- **跟踪点**（tracepoint）：断点不一定非要停下。设置“操作”让它向**输出**窗口打印一条带表达式的消息而程序继续跑，相当于一个随时可开关、不污染代码的日志语句。

9.2.7 断点标签、导入导出与断点窗口

调试 > 窗口 > 断点是所有断点的管理台，可以按名称、条件、命中次数、语言、标签排序和搜索，支持批量启用、禁用、删除。右键断点选择“编辑标签”，一个断点可以有多个标签：把和越界相关的断点都打上 `oob`，排查结束后按标签筛选、一次性清掉。断点窗口工具栏还提供“导出断点”与“导入断点”（XML 文件），把一组精心设计的断点随代码提交到仓库是团队协作里非常实用的技巧。另外记住：断点条件失效时（语法错误或语义不成立）调试器**仍然会中断**，只是给出警告——别指望它“静默跳过”。

9.3 单步执行

9.3.1 单步命令与快捷键

快捷键	命令	行为
F10	逐过程（**调试 > 逐过程**）	执行当前行，不进入被调函数
F11	逐语句（**调试 > 逐语句**）	进入被调函数；无源码时进入反汇编
Shift+F11	跳出（**调试 > 跳出**）	执行到当前函数返回
Ctrl+F10	运行到光标处	在当前行下临时断点并启动调试
Ctrl+Shift+F10	设置下一条语句	改变执行点，不执行中间代码
F5 / Shift+F5	继续 / 停止调试	运行到下一个断点 / 结束会话
Ctrl+Shift+F5	重新启动	重启进程，断点保留

提示 这一对快捷键常被记反：**运行到光标处是 Ctrl+F10**，设置下一条语句才是 Ctrl+Shift+F10。在编辑状态下右键代码行选择“运行到光标处”会同时完成“下临时断点 + 启动调试”两件事，比“先 F9 再 F5”更快；中断模式下把鼠标悬停在某行，行左侧会出现一个绿色的“运行到单击处”按钮，点它等价于运行到该行，这个命令在调用堆栈窗口里同样可用。

9.3.2 设置下一条语句与“往回走”

调试 > 设置下一条语句把执行点（黄色箭头）移到光标所在行，也可以直接用鼠标拖动黄色箭头。两种典型用途：跳过一次已知会失败的调用，看后面的逻辑是否正常；重放刚才那段代码，反复观察同一次执行的细节。但要清楚它的代价——它只改变指令指针，不撤销任何副作用：变量不会还原，已分配的内存不会回收，已析构的对象不会复活。对 C++ 而言，“往回跳”越过构造与析构、RAII 作用域、`std::unique_ptr` 的所有权转移，极易制造假象；它适合观察纯计算代码，不适合跨越资源管理边界。

9.3.3 “仅我的代码”对单步的影响

“仅我的代码”（Just My Code）在 **工具 > 选项 > 调试 > 常规**里开关，默认开启，它对 C++ 的影响是：

- **调用堆栈**：非用户代码被折叠成一个灰色的 `[外部代码]` 帧，右键选择“显示外部代码”可以展开；
- **单步**：在非用户代码上按 `F11` 会被“跨过”，直接跑到下一行用户代码，`Shift+F11` 则直接跑到当前帧之外的下一行用户代码；
- **判定依据**：对调用堆栈而言，符号里没有源码信息、或源码文件不存在的函数算非用户代码；对单步而言判定更严格——对应.pdb 没有加载的函数就算非用户代码。所以“某个第三方库明明有源码却跨不过去”，通常就是它的.pdb 没加载，去 **调试 > 窗口 > 模块**确认一下；
- 单步相关的编译开关是 `/JMC`，C++ 工程默认开启；用较老的编译器产出的二进制只能靠 `.natstepfilter` 文件定制。

想进入 STL 内部看 `std::vector::push_back` 到底做了什么，就得关掉这个选项。调试器还支持用 `.natjmc` / `.natstepfilter` 两个 XML 文件精细指定“哪些模块、源文件、函数算外部代码”“哪些函数一律跨过”：可以放在 Visual Studio 安装目录的 `Common7\Packages\Debugger\Visualizers` 下，也可以放在用户文档目录的 `Visualizers` 文件夹里，较新版本还允许直接放进项目或解决方案。

> **注意** C++ 下即使开启“仅我的代码”，**异常仍然会在抛出的地方停下**，无论在用户代码还是外部代码里；“异常设置”里的“用户未处理”选项对 C++ 不适用。这一条经常让从 C# 转过来的同学困惑。

9.4 查看数据

9.4.1 数据提示、自动窗口与局部变量

中断模式下把鼠标悬停在变量上会弹出数据提示（DataTip），可以展开对象、查看 STL 容器的元素；点图钉可以把它“固定”在编辑器上，之后单步时会跟着更新，右键菜单里还能“添加监视”“编辑值”“复制表达式”。数据提示里的值同样支持格式说明符（见 4.3）。

窗口	打开方式	内容
自动	**调试 > 窗口 > 自动**	当前行及之前若干行用到的变量；C++ 下是**前 3 行**
局部变量	**调试 > 窗口 > 局部变量**	当前作用域内的全部局部变量，含 `this`
监视 1–4	**调试 > 窗口 > 监视 > 监视 1** 等	手动输入的表达式，跨作用域保留

自动窗口的内容随执行点滚动，适合“顺着代码看”；局部变量是当前作用域的全景；监视窗口是唯一能长期盯住某个表达式的窗口——变量离开作用域时它不会消失，只是变灰。

9.4.2 监视窗口与即时窗口

监视窗口接受的不只是变量名，任何调试器能求值的表达式都可以：

```
p / *p / p->next->next      指针本身、解引用、链式追踪
&g_buf.canary               取地址，配合数据断点
(char*)p / sizeof(...)      强制转换、类型大小
arr,[4]                     把数组或指针按 4 个元素展开
```

调试 > 窗口 > 即时窗口（`Ctrl+Alt+I`）更进一步：它除了求值，还能**执行**语句和函数调用，例如 `BumpAll(g_buf.data, 4)`、`printf("%d\n", i)`、`SetLastError(0)`。这让它成为验证假设的利器——与其改代码重编译，不如在即时窗口里直接调用一次可疑函数；代价是它会真的改变进程状态，调用有副作用的函数容易把现场搞乱。

> **实战场景** 怀疑某个对象的成员在某个时刻已经被损坏，就在监视窗口里写 `&obj` 记住地址，再写 `obj` 展开看内容；想看它何时被改写，直接右键该表达式选“值更改时中断”，比在源码里到处下断点快得多。

9.4.3 格式说明符与并行监视

在监视、自动、局部变量窗口里，变量名后面加一个逗号和说明符就能改变显示方式；监视窗口还有个技巧：**输入逗号后会弹出可用的说明符列表**，记不住时直接选。

说明符	效果	示例
`x` / `X` / `xb`	十六进制（带 / 不带 0x 前缀）	`flags,x`
`c` / `b` / `bb` / `o`	字符 / 二进制 / 八进制	`ch,c`、`mask,bb`
`e` / `f` / `g`	科学计数 / 定点 / 自动浮点	`ratio,f`
`s` / `s8` / `su`	窄字符串 / UTF-8 / UTF-16 宽字符串	`wstr,su`
`wc` / `wm` / `hr`	窗口类标志 / Windows 消息号 / HRESULT 与 Win32 错误码	`cls,wc`、`hres,hr`

说明符	效果	示例
en / fe / !	枚举 / 位标志枚举 / 原始格式（忽略定制视图）	state,en
,[N]	把指针当 N 个元素的数组	p,[10]、p,[0x20]、p,[count]

两点提醒：数组长度现在推荐写 ,[N] 形式，老资料里的 p,10 属于旧写法，长度也可以是表达式（如 p,[count]），调试动态缓冲区时非常方便；十六进制可以写成 ,x，有些资料写作 ,hex，具体以逗号下拉列表里列出的为准——官方文档还提示，一部分老式说明符（如 d、h、u 等）依赖旧的原生调试引擎，在新版调试器中不再生效。调试多线程或并行任务时，**调试 > 窗口 > 并行监视 > 并行监视 1** 可以把同一个表达式按线程分行显示，一眼看出“哪个线程的值不对”，它常和并行堆栈配合使用。

9.5 调用堆栈与线程

9.5.1 调用堆栈窗口

调试 > 窗口 > 调用堆栈（Ctrl+Alt+C）显示“我是怎么走到这里的”：最上面一行是当前函数，下面依次是调用者，最下面是 main 或线程入口。双击某一帧会切换调试上下文——局部变量、监视、this 都随之切换，但**程序并不会继续执行**。这一点是理解调用堆栈的关键：切换堆栈帧只是换了一个观察角度。右键菜单里的常用操作有插入断点、运行到光标处、显示外部代码、查看符号加载信息；有符号但找不到源码的帧会显示为“无可用源代码”。

9.5.2 并行堆栈与线程窗口

调试 > 窗口 > 并行堆栈（Ctrl+Shift+D，S）把每个线程的调用栈并排画出，相同调用路径会被合并，于是“8 个线程都堵在同一个锁上”这种事一眼就能看出来；它可以在“线程”和“任务”两种视图间切换，也能显示或隐藏外部代码、按调用路径分组。**调试 > 窗口 > 线程**（Ctrl+Alt+H）则列出进程内所有线程的 ID、类别、名称与当前位置，三个操作最实用：**切换线程**（双击一行，或用调试工具栏右侧的线程下拉框，切换后的调用堆栈与局部变量都属于该线程）、**重命名线程**（右键 > “重命名”，叫 Worker-IO 远比叫 0x2A4C 好记，而且线程名可以直接用在断点筛选器里）、**冻结 / 解冻线程**（暂时不让某个线程被调度，排查“到底哪个线程在破坏数据”时很有用）。

9.5.3 多线程死锁的排查思路

死锁的典型形态是 A 等 B 的锁、B 等 A 的锁，两个线程都“卡住但活着”。步骤是：程序卡住后先 **调试 > 全部中断**打断所有线程；打开 **调试 > 窗口 > 并行堆栈**看有几个线程停在锁操作上；逐个线程查看调用堆栈，分别记录“它正在等待的对象”和“它已经持有的对象”；然后找环——在监视窗口里展开 CRITICAL_SECTION 的 OwningThread、LockCount、LockSemaphore 等字段确认谁在持有（std::mutex 的内部实现随版本而变，不好直接读，这也是“自己包一层互斥量并记录持有者”这种做法的现实理由）。

```
线程 A:  Lock(mutexA)  <- 成功  -->  Lock(mutexB)  <- 阻塞，等 B 释放
线程 B:  Lock(mutexB)  <- 成功  -->  Lock(mutexA)  <- 阻塞，等 A 释放
            ^
            +-- 互相等待，谁都不放手  =>  死锁
```

提示 预防比排查便宜。约定全局统一的加锁顺序、用 `std::scoped_lock` 一次锁住多个互斥量（它内部带死锁避免算法）、避免在持锁期间调用可能回调用户代码的函数，这三条能消灭绝大多数死锁。

9.6 内存与底层调试

9.6.1 内存窗口

调试 > 窗口 > 内存 > 内存 1（Ctrl+Alt+M，1）直接查看进程内存，地址栏可以输入 `&g_buf`、`p`、`0x0000000123456780` 这类表达式，右键或工具栏可以切换每列的字节数（1 / 2 / 4 / 8）与“自动重新计算”。查指针与数组时：想知道 `p` 指向什么，就把 `p` 作为地址填进去，再按 1 字节或 4 字节解释；想确认越界有没有踩坏相邻变量，直接看 `&g_buf.canary` 附近的内存，并对照填充图案（0xCD / 0xDD / 0xFD）判断是“未初始化”还是“已被释放”；单纯想看字符串，在监视窗口里用 `,s` / `,su` 比在内存窗口里数字符省事得多。

9.6.2 反汇编窗口与寄存器窗口

调试 > 窗口 > 反汇编（Ctrl+Alt+D）显示当前堆栈帧对应的汇编，并尽量把源码行交错显示；它能做到源码做不到的事：在优化过的 Release 代码里单步、看清编译器是否把某个分支优化掉了、确认崩溃时到底是哪条指令出了问题，右键菜单可以在“显示字节代码”“显示符号名”“显示行号”“转到源代码”之间切换。**调试 > 窗口 > 寄存器**（Ctrl+Alt+G）显示通用寄存器与标志位，值可以直接编辑，也可以在监视窗口里用 `@rax`、`@rip` 这样的形式读取。x64 调用约定值得顺手记住：前 4 个整型或指针参数放在 RCX、RDX、R8、R9，浮点参数放在 XMM0–XMM3，返回值在 RAX——在反汇编里看到 `mov rax, [rcx]`，就基本能猜到是在解引用第一个参数。

实战场景 一个只在 Release 下出现的崩溃，断点绑不上。做法是：在异常设置里勾上对应的 Win32 异常，让它在崩溃瞬间中断，然后打开反汇编窗口看当前指令，再用寄存器窗口里的地址配合内存窗口确认指针是否为空、是否已被释放。

9.7 异常处理

9.7.1 异常设置窗口与“两类异常”

调试 > 窗口 > 异常设置（Ctrl+Alt+E）决定“哪些异常会让调试器停下来”，原生 C++ 调试下主要看两个节点：**Win32 异常**，即操作系统层面的异常代码，常见的有 0xC0000005（访

问冲突）、0xC0000094（整数除零）、0xC00000FD（栈溢出）、0xC0000374（堆损坏）；**C++ 异常**，即 C++ 的 `throw`，可以按异常类型添加过滤条件。

异常调试最容易混淆的是“首次异常”与“用户未处理异常”：

概念	时机	此时的堆栈	用途
首次异常（第一轮）	异常刚被抛出，处理程序还没运行	完整，现场最新鲜	找“到底是谁抛的”
用户未处理异常（第二轮）	没有任何处理程序接管，即将终止进程	可能已被栈展开破坏	找“为什么没人接住”

对 C++ 来说，栈展开（unwinding）会调用沿途对象的析构函数，等你在“未处理异常”处中断时，局部对象可能已经析构，变量的值已经不可信，所以**排查 C++ 异常要优先在首次异常处中断**。

> **注意** 默认情况下“仅我的代码”会把外部代码抛出的异常隐藏起来，只在输出窗口留一行消息。要让调试器在系统库或第三方库里抛出异常时也停下，需要关闭“仅我的代码”，并在异常设置里勾上对应类型。

9.7.2 C++ 异常、SEH 与“在 throw 处中断”

C++ 异常（`throw std::runtime_error(...)`）是语言层面的，由运行时函数负责抛出与展开；SEH（`__try` / `__except` / `__finally`）是操作系统层面的，访问冲突、除零这类硬件异常天然就是 SEH 异常，`__except` 的过滤表达式可以用 `GetExceptionCode()` 取到异常代码。当两者混用时，`/EHsc` 与 `/EHa` 的行为差别会成为 bug 的来源：`/EHa` 让 `catch(...)` 能接住 SEH 异常，但也可能掩盖真正的访问冲突——调试时如果发现“该崩的地方没崩”，先检查这个开关。

想让调试器在 `throw` 处中断，有三种做法，精确度递增：在异常设置窗口的 **C++ 异常**节点上勾选“抛出时”中断（覆盖面最广，但库内部的正常异常流也会被拦下，比较吵）；在具体的 `throw` 语句所在行下断点（精确，但只覆盖你找得到的那几处）；**在函数断点里输入 `_CxxThrowException`**——MSVC 的所有 C++ `throw` 最终都会走到这个运行时函数，因此它等价于“任何地方 throw 都中断”，还不会漏掉模板和库代码里的抛出，配合调用堆栈就能立刻看出是谁抛的、抛之前发生了什么。

9.8 编辑并继续 / 热重载

9.8.1 前提条件

“编辑并继续”（Edit and Continue）现在统一叫**热重载**（Hot Reload），但对 C++ 而言底层能力仍是编辑并继续。它只在**调试版本**下工作，且需要：编译用 `/ZI`（“用于编辑并继续”的调试信息格式），链接器启用增量链接 `/INCREMENTAL`；不能用 `/Zo`（增强优化调试）编译，那会**关闭**热重载；通常要求从 Visual Studio 用 `F5` 启动（附加到进程的场景一般不支持）。相关选项

位于 **工具 > 选项 > 调试 > .NET/C++ 热重载**：“启用热重载”“继续时自动应用更改（仅限本机）”“对陈旧代码发出警告”“不调试时启用”“保存文件时应用更改”“日志详细程度”，其中“继续时自动应用更改”默认开着。手动应用用 **调试 > 应用代码更改**，它只应用修改、**不继续执行**；应用过程中想反悔，用 **调试 > 停止应用代码更改**（该菜单项只在应用过程中可见），此时所有更改都不会提交。

9.8.2 能改什么，不能改什么

修改类型	能否热应用	说明
函数体内的语句、局部变量初值与表达式	可以	最常用的场景；执行点可能被移动，C++ 会弹窗提示
新增局部变量、修改当前函数的调用者或被调用者	有限制 / 暂缓	给**在调用堆栈上**的函数新增超过 64 字节变量时无法立即应用，会退化为“陈旧代码”；改了调用者则先执行旧代码
修改类、结构体的成员与布局、函数签名、增删成员函数或全局变量	不可以	会让已有对象的内存布局失效或需要重新编译链接，必须重启
改宏、改头文件里的类型定义、换库或改构建选项	不可以	属于构建配置变更

无法应用时，Visual Studio 会明确提示“此更改需要重新生成 / 重新启动调试”。看到这句话就不要再试图绕过，老老实实 `Ctrl+Shift+F5` 重启。

9.8.3 陈旧代码与执行点漂移

两个概念值得单独记住：**陈旧代码**（stale code）指修改无法立即应用时（编辑了当前函数的调用者，或者给栈上的函数加了太多新变量），调试器继续执行旧代码，并把它显示在一个标题类似 `enc25.tmp` 的临时源文件窗口里，而你编辑后的文件仍在原窗口——此时不要编辑那个临时窗口；**执行点漂移**指某些修改会让黄色箭头的位置发生变化，C++ 下调试器会弹窗提示，需要你确认位置是否正确，不对就用“设置下一条语句”手动纠正。

> **提示** 热重载适合改打印、改分支条件、修正一两个表达式这类“微调”。如果一次要动十几行逻辑或者改数据结构，直接重启调试往往比和热重载较劲更快。

9.9 高级调试手段

9.9.1 附加到进程与远程调试

调试 > 附加到进程（`Ctrl+Alt+P`）适用于程序已经在跑、复现步骤又很长的场景：先让程序进入“即将出错”的状态，再附加，然后手动触发问题。“附加到”一栏要选对类型——纯原

生程序选“本机代码”，混了托管代码选“自动”或“混合”，选错的典型症状是断点变成空心圆、调用堆栈里全是地址；64 位的 Visual Studio 可以调试 32 位和 64 位进程，但反向不行。

目标机器不能装 Visual Studio 时用远程调试：先把与目标机架构匹配的远程调试器（`msvsmon.exe`，位于 Visual Studio 安装目录的 `Common7\IDE\Remote Debugger\<arch>` 下）复制到目标机并运行，记下它监听的端口；然后在开发机上用 **调试 > 附加到进程**，连接类型选“远程（Windows 身份验证）”或“远程（无身份验证）”，目标填 机器名：端口；也可以在 **项目属性 > 调试**里把“要启动的调试器”设为“远程 Windows 调试器”、“远程服务器”填目标机名，之后直接 F5 就是远程启动。要点是：目标机上必须有与二进制**完全匹配**的.pdb，源码路径也要能被开发机找到（否则只能看反汇编），防火墙需要放行 msvsmon 的端口。

9.9.2 转储文件调试

崩溃发生在别人的机器上、现场无法复现时，转储文件（.dmp）是唯一的事后证据：

```
# 让 procdump 在进程崩溃(未处理异常)时抓一份完整转储，也可用任务管理器右键"创建转储文件"
.\procdump.exe -accepteula -ma -e -x .\dumps .\demo.exe
# 检查刚才那次运行的退出代码，0xC0000005 表示访问冲突
$p = Start-Process .\demo.exe -PassThru -Wait; "退出代码: {0}" -f $p.ExitCode
```

打开转储文件用 **文件 > 打开 > 文件**，选中的.dmp 会以调试会话的形式打开，调试器直接停在异常发生的那一帧；之后的操作和在线调试几乎一样，调用堆栈、线程窗口、局部变量、并行堆栈都能用。两个前提：符号路径要配好（至少指向自己程序的.pdb），并且转储文件必须“完整”到包含你需要的线程栈与堆内存——小型转储体积小，但可能看不到局部变量的值。

9.9.3 OutputDebugString、TRACE 与 assert

在调试器之外，最轻量的观察手段是让程序自己说话：

```
static void DebugLog(const char* fmt, ...) {          // 记得用 #ifdef _DEBUG 包住
    char buf[512]; va_list a; va_start(a, fmt); _vsnprintf_s(buf, _TRUNCATE, fmt, a); va_end(a);
    ::OutputDebugStringA(buf);                        // <windows.h>; 输出到 "输出" 窗口的 "调试" 页
}
_RPT1(_CRT_WARN, "count = %d\n", count);              // CRT 宏，自动带文件名与行号
TRACE("value = %d\n", value);                         // MFC / ATL 宏，仅在 _DEBUG 下有效
assert(head != nullptr && "head 不应为空");            // 断言表达式里绝不要写有副作用的代码
_ASSERTE(index >= 0 && index < count);                // _ASSERT 只输出信息，_ASSERTE 输出表达式文本
```

输出出现在 **调试 > 窗口 > 输出**的“调试”页，脱离调试器时可以用 Sysinternals 的 DebugView 捕获；这类调用本身有开销，一定要用 `##ifdef _DEBUG` 包住，或者改用 ETW。断言则是“开发期的合同检查”：

Debug 下断言失败会弹出断言对话框（中止 / 重试 / 忽略），点“重试”会**直接进入调试器**并停在失败的那一行，这是从断言跳进调试会话的标准入口；定义 NDEBUG 后 assert 会被完全消除，所以 assert(++i < n) 是经典错误。断言对话框的行为可以用 _CrtSetReportMode / _CrtSetReportHook 改写，把消息重定向到日志文件，适合自动化测试环境；编译期的检查则交给 static_assert。

9.9.4 调试时的环境变量与命令行参数

项目属性 > 调试这一页决定了 F5 到底怎样启动你的程序，却经常被忽略：

属性	作用	注意事项
命令 / 命令参数	要启动的可执行文件，以及传给 main(argc, argv) 的参数	默认 $(TargetPath)；参数含空格要加引号，改动后必须重启调试才生效
工作目录	进程的当前目录	影响相对路径读写；与 exe 所在目录不同会导致 DLL 找不到
环境 / 合并环境	追加或覆盖环境变量，每行一个 名称 = 值；是否与系统环境合并	常用于给 PATH 追加 DLL 目录；合并设为“否”时环境更干净，便于排除干扰
调试器类型 / 附加 / 远程服务器	自动、本机、混合、仅托管；是否附加到已有进程；远程调试的机器名与端口	纯 C++ 选“本机”；“附加”为“是”时 F5 不启动新进程；远程调试见 9.1

环境变量是排查“本机能跑、换台机器就少 DLL”的第一手段。与其把 DLL 拷来拷去，不如在“环境”里加一行 PATH=%PATH%;D:\libs\bin;$(OutDir)。CMake 工程不用这套属性页，而是在 launch.vs.json 里描述同样的东西。命令行构建与调试符号的关系也很直接：

```
rem 在 "x64 Native Tools Command Prompt for VS" 里执行：带符号的调试版本
cl /nologo /Zi /Od /EHsc /W4 /MDd main.cpp
```

9.10 实战示例

下面这段代码故意埋了两个经典 bug：**数组越界写**与**空指针解引用**。把它存成 main.cpp，跟着做一遍，就能把前面几节的内容串起来。

```
1 // cl /nologo /Zi /Od /EHsc /W4 /MDd main.cpp
2 #include <cstdio>
3 struct Buffer { int data[4]; int canary; };      // canary 在布局上紧跟 data[3] 之
      后
4 static Buffer g_buf = { { 10, 20, 30, 40 }, 0x7FFFFFFF };
5 struct Node { int id; Node* next; };
```

```
6  static void BumpAll(int* data, int count) {
7      for (int i = 0; i <= count; ++i) data[i] += 1;   // BUG 1: 应为 i < count, 越
       界踩到 canary
8  }
9  static Node* FindNode(Node* head, int id) {
10     for (Node* p = head; p != nullptr; p = p->next) if (p->id == id) return p;
11     return nullptr;                                  // 找不到就返回空指针
12 }
13 int main() {
14     printf("canary before = 0x%08X\n", g_buf.canary);
15     BumpAll(g_buf.data, 4);
16     printf("canary after  = 0x%08X\n", g_buf.canary);
17     Node a{ 1, nullptr }, b{ 2, nullptr }; a.next = &b;
18     Node* found = FindNode(&a, 3);                   // 3 根本不存在
19     printf("found id = %d\n", found->id);            // BUG 2: 空指针解引用
20 }
```

9.10.1 用数据断点抓数组越界

canary 的初值是 0x7FFFFFFF，如果运行后它变了，就说明有人越界写到了 g_buf.data 之外。步骤：

1. 在 BumpAll 的循环体那行按 F9 下断点，F5 启动；第一次命中时在监视窗口加入 i 和 &g_buf.canary，记下地址（例如 0x00007FF6A1B23014）与当前 i；
2. 继续按 F5 观察 i 递增到 3——data[3] 是最后一个合法元素，再往后就出界了；
3. 删掉这个断点，改用**数据断点**：**调试** > **新建断点** > **数据断点**，地址填 &g_buf.canary，字节数选 4；
4. 按 F5 继续，程序会**恰好在越界写发生的那一瞬间**停下来，而不是等到 printf 打印出错误的 canary；
5. 此时看调用堆栈：当前帧是 BumpAll，调用者是 main；监视窗口里 i 是 **4**，而 count 也是 4——data[4] 显然越界，for (int i = 0; i <= count; ++i) 里的 <= 就是元凶；
6. 改成 i < count 后应用代码更改或重启调试，canary 应该稳定保持在 0x7FFFFFFF。

如果没有数据断点，你只能靠“canary 变了”这个结果倒推，中间过程完全看不到；数据断点把这个过程压缩成了一次精确的中断。

9.10.2 用断点 + 监视 + 调用堆栈抓空指针

1. 在 printf("found id = %d\n", found->id); 这行按 F9，F5 运行到断点；
2. 在监视窗口加入 found——值是 0x0000000000000000；展开 found->id 会提示无法读取内存，这就是崩溃的前兆；
3. 打开 **调试** > **窗口** > **调用堆栈**，双击 main 上面那一帧进入 FindNode；或者直接在 return nullptr; 那行下断点重新运行一次，看它是在什么输入下返回了空；

4. 按 F10 逐过程执行到下一行（程序会在 printf 内部触发访问冲突）。这里可以做两件事：在 **调试 > 窗口 > 异常设置**里勾上 Win32 Exceptions 下的访问冲突（0xC0000005），让调试器在异常发生的第一时间中断；打开 **调试 > 窗口 > 反汇编**，会看到类似 mov eax, dword ptr [rax] 的指令，而寄存器窗口里 RAX 是 0——机器层面的原因一目了然；
5. 在即时窗口里验证假设：输入 found != nullptr 得到 false，再输入 FindNode(&a, 3) 确认它确实返回空；
6. 修复思路有两种：在调用处判空后再解引用（最小改动），或者让 FindNode 返回引用、可选值之类的类型，把“可能找不到”编码进接口（更好的设计）。

实战场景 真实项目里的崩溃往往比这复杂得多，但流程完全一样。记住这条“三窗口联动”的套路：**调用堆栈告诉你从哪来，监视窗口告诉你现在是什么状态，数据断点与异常设置告诉你是哪一刻变坏的**。三者配合，绝大多数内存类 bug 都能在一个调试会话里定位。

9.11 小结

- F5 是“带调试器跑”，Ctrl+F5 是“纯跑一遍”；.pdb 靠 GUID + Age 与二进制配对，断点变成空心圆先查符号有没有加载，而 /Zi 与 /Od 是能在 Debug 下看清变量与单步的前提。
- 断点远不止“某一行”：条件、命中次数、筛选器、函数断点、数据断点、临时断点、依赖断点各有明确的适用场景；其中数据断点只在写入时中断、x64 下最宽 8 字节且需按宽度对齐、总数不超过 4 个，而断点窗口负责批量管理、打标签与导入导出。
- 单步用 F10 / F11 / Shift+F11，想跳位置用 Ctrl+F10（运行到光标处）和 Ctrl+Shift+F10（设置下一条语句，它只改指令指针、不撤销副作用）；看数据靠数据提示、自动与局部变量、监视 1–4、即时窗口，以及 ,[N]、,x、,su 这类格式说明符。
- 并发问题用调用堆栈、并行堆栈与线程窗口这三件套，死锁的固定套路是“全部中断 + 逐线程看堆栈 + 找环”；只看机器层面时用内存、反汇编、寄存器窗口。
- 异常处理的关键是分清“首次异常”与“用户未处理异常”，对 C++ 在首次异常或 _CxxThrowException 处中断最有价值；热重载能改函数体、改不了数据布局；附加到进程、远程调试、转储文件、OutputDebugString、assert 与“项目属性 > 调试”构成了真实项目的完整工具箱。

9.12 练习题

1. 写一个把 int 数组元素全部翻倍的函数，故意让循环多走一次；先用条件断点 i == count 找到越界，再改用数据断点监视数组之后的一个变量，比较两种方式各自的优缺点（提示：哪种能看见“写入的瞬间”？）。
2. 用 OutputDebugStringA 加一个 ##ifdef _DEBUG 包装宏，实现一个带文件名与行号的日

志函数，让它的输出出现在**输出**窗口的“调试”页；再故意去掉 `_DEBUG`，确认 Release 下它完全不产生代码。

3. 构造一个两线程死锁：线程 A 先锁 mutexA 再锁 mutexB，线程 B 反过来；程序卡住后用“全部中断” + 并行堆栈定位，记录两个线程各自的调用堆栈，并说明如何用统一加锁顺序修复。
4. 给第 10 节的例子加一个会抛出 `std::runtime_error` 的分支，分别用“异常设置里勾选 C++ 异常”和“函数断点 `_CxxThrowException`”两种方式让调试器在 throw 处中断，比较两种方式拦到的异常数量。
5. 用 **项目属性 > 调试**的命令参数给程序传三个参数（其中一个含空格），并在“环境”里追加一个自定义变量，在代码中用 `getenv` 读出来；说明为什么改了参数之后必须重启调试。
6. 在调试会话中修改一个函数体（比如改打印内容），用“应用代码更改”让它生效；然后试着给 `struct Buffer` 加一个新成员，观察 Visual Studio 的提示，并解释为什么这类修改必须重启。

9.13 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **上一章：代码编辑与 IntelliSense**
- **下一章：性能分析与诊断**
- **C++ 教程：Visual Studio 入门**
- **CMake：CMake 使用指南**

第十章　性能分析与诊断

性能问题不能靠感觉解决，它需要数据：谁慢、慢在哪里、慢了多少。Visual Studio 2026 提供了两条互补的路径——调试期间的**诊断工具**窗口用于快速定位，独立的**性能探查器**用于系统性测量；再叠加编译期的 Build Insights、静态分析与运行时检查，就能把性能与内存问题压缩到可控范围。本章按“先测量、再优化、后验证”的顺序走一遍这条工具链。

10.1　诊断工具窗口（调试时）

10.1.1　打开方式与面板构成

按 F5 开始调试时**诊断工具**窗口默认出现；若被关闭，可在调试期间通过 **调试 > 窗口 > 显示诊断工具**重新打开。窗口分两部分：上半部分是**时间线**（叠加 CPU 曲线、内存曲线与事件标记），下半部分是 **事件**、**内存使用率**、**CPU 使用率**三个标签页。工具栏上的设置按钮可以勾选本次会话启用哪些工具——采集本身有开销，不必全部打开。

> **注意** 诊断工具窗口只在调试会话中存在，调试一停止数据就消失，需要时先导出或记录。

10.1.2　CPU 使用率：调用树与热路径

点击“录制 CPU 分析”后得到的是**采样**数据，有三种查看方式：调用树（按层级展开调用关系）、调用方/被调用方（聚焦某函数的上下游）、函数列表（按自身耗时倒序的平铺列表）。

每个函数有两个关键数字：**总耗时（含子调用）**与**自身耗时（不含子调用）**。优化目标是自身耗时高的函数；总耗时高而自身耗时接近 0 的通常只是框架入口，动它没有意义。

```
CPU 使用率（采样）视图示例：总耗时 / 自身耗时
+-- main                                  100.0%   0.2%
|   +-- LoadData                           92.4%   0.5%   <- 只是上游
|   |   +-- ParseRow                       88.1%  12.0%
|   |   |   +-- std::map::find（红黑树查找）  71.0%  71.0%   <- 真正的热路径
|   |   +-- ReadFile                        4.1%   4.1%
|   +-- Render                              7.2%   6.5%
```

排查顺序：先用函数列表按自身耗时倒序看前三名属于哪个子系统；再用调用树确认它们的**调用次数**——是被百万次调用，还是只调一次但单次极慢？两者优化手段完全不同。窗口顶部的**仅我的代码**开关则决定系统库与第三方库是否被折叠：关掉它才能看到 STL 内部的真实开销，适合判断“是不是容器的锅”。

10.1.3 内存使用率、事件与时间线

内存使用率的核心操作是**拍摄快照**：在关心的时刻拍一次，程序再跑一段后拍第二次，然后点“比较”。比较视图给出对象类型/大小/计数/差异四列、某个类型的**分配调用栈**，以及对持续增长且不回收的**泄漏**标记。

> **注意** 快照只统计**堆**上的分配。栈上数组、静态缓冲区、以及绕过 CRT 直接向系统申请的大块内存都不会出现，排查时要单独考虑。

事件标签页记录断点命中、异常抛出、输出字符串、模块加载/卸载、线程创建与退出等信息，价值在于把代码行为与资源曲线对齐到同一条时间轴上。看到“每次循环都创建线程”或“异常被反复抛出用于控制流”，基本可以直接定位到设计层面的问题。

10.2 性能探查器

10.2.1 启动与 C++ 可用的工具

菜单 **调试 > 性能探查器**（快捷键 `Alt+F2`）打开探查器：选择目标（启动项目、可执行文件或附加到进程）-> 勾选工具 -> 开始采集 -> 停止后得到 `.diagsession` 报告。

工具	采集原理	主要产出	适用场景
CPU 使用率	采样	调用树、火焰图、函数与模块统计	默认首选，先找热点
检测（Instrumentation）	插桩	精确调用次数、含/不含子调用耗时	函数太小采不到、要确认调用次数
内存使用率	堆快照与分配跟踪	快照差异、分配调用栈	内存增长、泄漏、分配过频
文件 I/O	ETW 跟踪	读写字节数、按文件与进程汇总	启动慢、磁盘成为瓶颈
并发可视化	ETW 跟踪	线程时间线、阻塞原因分类	多核跑不满、锁争用

> **注意** 列表中的部分工具（面向.NET 的计数器、数据库工具）不适用于纯 C++ 目标；并发可视化在部分版本中需要额外安装组件或扩展，找不到时先到 Visual Studio 安装程序中确认。较新版本的 CPU 使用率工具还提供火焰图（Flame Graph）视图，用栈帧宽度表示耗时占比。

10.2.2 采样与插桩的区别

维度	采样（CPU 使用率）	插桩（检测）
原理	以固定时间间隔中断程序并记录调用栈	在每个函数入口/出口插入探针
开销	低，通常可以忽略	高，调用密集时可能成倍变慢
数据性质	概率分布，短小或调用极少的函数可能被漏掉	精确的调用次数与累计耗时
对内联的影响	不大	明显，内联会移动探针位置，优化构建下尤其失真
典型用途	找“谁最耗时”	查“到底调了多少次”“谁调用的”

选择原则：**先用采样找方向，再用插桩做定量确认**。若采样报告里某函数一直没出现，但逻辑上应该被调用成千上万次，就是插桩登场的信号。

10.2.3 内存、文件 I/O 与并发

内存使用率的标准流程是“两次快照 + 差异”：操作前拍一次，执行可疑操作后拍一次，切到差异视图，看**计数增长最快**的对象类型并展开它的分配调用栈。

实战场景 一个图形程序每次刷新界面内存都涨几百 KB。快照差异显示增长的是 `std::vector<Pixel>`，调用栈指向每帧重建的缓存对象——问题不在容器，而在于缓存对象的作用域被写在了循环内部。

- **文件 I/O**：按进程汇总读写次数与字节数，可按文件排序并展开查看某次读写的调用栈，快速回答“启动慢是算出来的还是读出来的”。
- **并发可视化**：把每个线程画成一条泳道，区分执行、同步、阻塞、I/O 状态并统计阻塞原因。8 核只跑满 1 核时，它能直接指出瓶颈是锁争用、串行 I/O，还是任务粒度太粗。

10.3 Build Insights：编译期性能分析

10.3.1 它解决什么问题

Build Insights 分析的是**构建过程本身**，回答“为什么编译这么慢”：哪些头文件被反复解析、哪些模板实例化最贵、代码生成与优化各占多少时间、链接是否拖后腿。它基于 MSVC 工具链（`cl.exe` / `link.exe`）产生的事件，是 **MSVC 专属的编译分析能力**，对 clang 或 GCC 构建不适用。使用前需在 Visual Studio 安装程序中确认已安装 C++ Build Insights 相关组件。

10.3.2 两种使用方式

图形界面方式可从生成相关菜单启动一次带跟踪的构建，结束后自动打开分析视图（菜单名称随版本略有差异）。需要脚本化或可对比的结果时，用命令行工具 `vcperf.exe`：

```
:: 在管理员权限的命令提示符中执行（跟踪需要系统级权限）
```

```
:: 1) 开始采集
vcperf /start /out build_session.etl

:: 2) 用 MSBuild 重新构建；必须 Rebuild，结果才有意义
msbuild MyApp.sln /t:Rebuild /p:Configuration=Release /p:Platform=x64

:: 3) 停止采集；/tracing 保留 ETL 文件，去掉它则直接打开分析视图
vcperf /stop /tracing build_session.etl
```

采集时可以指定更高的跟踪级别以获得更细的编译器内部阶段信息（如 `/level3`，级别越高开销越大）。若项目不使用 MSBuild 构建，需要按文档为编译器与链接器手动打开事件输出开关，否则跟踪里只有链接与文件访问信息。

10.3.3 定位编译慢的根因

一次构建会被拆成若干阶段：**编译器前端（解析、语义分析、模板实例化）**、**代码生成**、**优化器**、**链接器**，并按文件、头文件、函数、模板等维度聚合。看报告的顺序：按总耗时排序的文件 -> 被间接包含最多的头文件 -> 实例化次数最多的模板 -> 少数巨型函数的代码生成耗时。

现象	处方
某头文件被大量间接包含	前置声明替代 include、pimpl 惯用法、实现移出头部
稳定不变的头被重复解析	预编译头（PCH），把常用第三方头集中进稳定的头
模板重复实例化	显式实例化并配合 `extern template` 抑制隐式实例化
巨型函数拖慢代码生成	拆分函数、削减过度内联、选择性禁止内联
单线程编译时间过长	打开 `/MP` 多进程编译，检查是否有单一巨型翻译单元拖尾

日常排查包含关系时，可在 **项目属性 > C/C++ > 高级 > 显示包含文件**打开 `/showIncludes`，在输出窗口直接查看包含链。

10.4 静态代码分析

10.4.1 运行代码分析

编译器的警告只看语法层面，`/analyze` 则会在编译期做**基于数据流与控制流的路径分析**，发现缓冲区越界、空指针解引用、资源泄漏、格式化字符串不匹配等跨语句缺陷。

- 图形界面：**分析 > 运行代码分析**（可对当前项目或整个解决方案运行）。
- 命令行：给编译器加 `/analyze`，或在 MSBuild 上打开代码分析开关：

```
msbuild MyApp.sln /t:Rebuild /p:Configuration=Release /p:RunCodeAnalysis=true
```

结果出现在**错误列表**窗口，规则编号形如 `C6xxx`，可按编号筛选、忽略或存档。首次全量分析明显变慢，建议只对自有代码开启，并放进 CI，而不是每次本地增量编译都跑。

10.4.2 警告等级与“警告即错误”

选项	含义	建议
/W0 ~ /W4	警告等级，越高越严格	新项目直接用 /W4
/Wall	打开几乎全部警告	噪声大，需配合白名单使用
/WX	把警告当错误	新项目必开，防止警告被无视
/wd4996 等	关闭指定编号的警告	只在有明确理由时使用
/external:W0	第三方头文件不产生警告	与 /W4 /WX 搭配的关键选项
/analyze	数据流分析	与 /W4 互补，不是替代关系
/sdl	追加安全相关检查	发布配置建议开启

在源码里临时屏蔽警告要用成对指令并标注原因：##pragma warning(push)、##pragma warning(disable : 4996)（附上理由注释）、##pragma warning(pop) 三段一组，不要把禁用指令长期留在文件里。

10.4.3 C++ Core Guidelines 检查器

它建立在 /analyze 之上，把指南条款变成可执行规则，覆盖：**资源管理**（RAII、所有权、智能指针）、**边界安全**（数组退化、越界、gsl::span 与 gsl::at）、**类型安全**（窄化转换、有符号与无符号混用、reinterpret_cast 滥用）、**生命周期**（返回局部对象的引用、悬垂指针）、**性能**（不必要的按值传参、可移动却发生拷贝）。

在 **项目属性 > 代码分析 > 常规 > 规则集**中选择相应规则集即可启用，较新版本对新建项目会默认启用一部分规则；也可以在项目文件中用 <CodeAnalysisRuleSet> 属性显式指定规则集路径，便于团队统一。

10.4.4 SAL 注解：把契约写进声明

SAL（Source Annotation Language）用注解描述参数契约，/analyze 据此做跨函数检查：_In_ / _Out_ / _Inout_ 表示输入、输出、输入输出；加 _opt_ 后缀（如 _In_opt_）表示允许空指针；_Outptr_ 表示输出一个指针；_Ret_maybenull_ 表示返回值可能为空；_Null_terminated_ 表示字符串以 \0 结尾；_Success_(expr) 限定注解成立的条件；_Check_return_ 表示返回值不应被忽略。

```
#include <sal.h>

// 契约：buf 至少有 size 字节可写，函数成功时返回写入长度
_Success_(return > 0)
int FillBuffer(_Out_writes_(size) char* buf, _In_ size_t size);
```

写好注解的收益是双向的：调用方传错参数会被立刻指出，函数内部也能得到更精确的推理结果。

10.5 clang-tidy 集成

10.5.1 在 Visual Studio 中启用

Visual Studio 可以把 clang-tidy 挂到 MSVC 的编译流程上，无需切换到 clang 工具集。步骤：

先在安装程序中确认装了“适用于 Windows 的 C++ Clang 工具”组件，然后打开 **项目属性 > 代码分析 > Clang-Tidy**：把“启用 Clang-Tidy 代码分析”设为**是**，在“Clang-Tidy 检查项”里填写要启用的检查（留空表示使用默认检查集），在“其他 Clang-Tidy 参数”里补充命令行参数。

对应的 MSBuild 属性可以直接写进项目文件，便于版本管理：

```
<PropertyGroup>
  <EnableClangTidyCodeAnalysis>true</EnableClangTidyCodeAnalysis>
  <ClangTidyChecks>-*,bugprone-*,performance-*,clang-analyzer-*</ClangTidyChecks>
</PropertyGroup>
```

注意 clang-tidy 在每次编译时被调用，会显著拉长编译时间。日常开发建议只开 `bugprone-*` 与 `clang-analyzer-*`；`modernize-*`、`readability-*` 这类风格检查放到 CI 的单独任务里跑。

10.5.2 .clang-tidy 配置文件

在解决方案根目录放一个 `.clang-tidy`，团队即可共享同一套规则：

```
Checks: >
  bugprone-*,
  performance-*,
  modernize-*,
  cppcoreguidelines-*,
  clang-analyzer-*,
  -modernize-use-trailing-return-type,
  -cppcoreguidelines-avoid-magic-numbers
WarningsAsErrors: 'bugprone-use-after-move,performance-unnecessary-copy-initialization'
HeaderFilterRegex: 'src/.*'
CheckOptions:
  - key: readability-identifier-naming.ClassCase
    value: CamelCase
FormatStyle: file
```

实践建议：`Checks` 里 `-` 前缀表示关闭，采用“先全开再逐条关闭”比反过来更好维护；`HeaderFilterRegex` 决定哪些头文件里的问题被报告，不设置的话第三方头文件的噪声会淹没结果；若项目同时有 `.clang-format`，用 `FormatStyle: file` 让检查与格式化保持一致。

10.5.3 常用检查项

检查项	捕获的问题
bugprone-use-after-move	对象被 std::move 之后又被使用
performance-unnecessary-value-param	本可传引用却按值传参
performance-for-range-copy	范围 for 里发生了拷贝
modernize-use-nullptr	用 0 或 NULL 表示空指针
modernize-use-override	虚函数重写未加 override
modernize-make-unique	可以改用 std::make_unique
readability-container-size-empty	用 size() == 0 判断空容器
clang-analyzer-*	空指针、内存泄漏、不可达代码等路径问题
cppcoreguidelines-owning-memory	裸 new / delete 的所有权管理

部分检查支持自动修复（命令行 --fix），但批量修改后必须重新跑一遍测试。

10.6 运行时检查

10.6.1 AddressSanitizer（/fsanitize=address）

ASan 通过编译期插入检查代码并替换内存分配器，在**运行时**捕获堆、栈、全局缓冲区的越界读写，释放后使用（use-after-free）与重复释放，并可配合 ASAN_OPTIONS=detect_leaks=1 打开泄漏检测。

启用方式：**项目属性 > C/C++ > 常规 > 启用 AddressSanitizer** 设为“是”（MSBuild 属性 EnableASAN），命令行等价于给编译与链接同时加 /fsanitize=address：

```
cl /nologo /Zi /O1 /EHsc /fsanitize=address demo.cpp /link /fsanitize=address
set ASAN_OPTIONS=detect_leaks=1:halt_on_error=1
demo.exe
```

注意 /fsanitize=address 与 /RTC1 不能同时使用，而 Debug 配置默认带 /RTC1，所以启用 ASan 时通常新建一个“Release + /Zi”的配置。ASan 版本会明显变慢并占用更多内存，只用于验证，不要拿它做性能测量；要得到可读堆栈必须保留调试信息。

10.6.2 /RTC 运行时检查

开关	检查内容
/RTCs	栈帧检查，捕获局部数组越界导致的栈破坏
/RTCu	使用未初始化的局部变量
/RTCc	隐式窄化转换（大类型赋给小类型）
/RTC1	等于 /RTCs /RTCu，Debug 配置的默认组合

/RTC 系列必须与 /Od 搭配使用，在开启优化的配置中会被忽略；/RTCc 还会与标准库的部分代码冲突，一般只在排查特定问题时临时打开。

10.6.3 CRT 内存泄漏检测

调试版 CRT（/MDd 或 /MTd，即定义了 _DEBUG）内置分配跟踪能力。完整示例：

```
// LeakDemo.cpp —— CRT 内存泄漏检测完整示例（Debug 配置下运行）
#define _CRTDBG_MAP_ALLOC      // 必须放在最前面，让分配记录带上文件与行号
#include <cstdlib>
#include <crtdbg.h>
#include <iostream>

#ifdef _DEBUG
// 让 new 表达式也记录文件与行号；它与 placement new 形式冲突，谨慎引入
#define DBG_NEW new (_NORMAL_BLOCK, __FILE__, __LINE__)
#define new DBG_NEW
#endif

struct Node { int value; Node* next; };

void LeakyFunction() {
    int* buffer = new int[100];                 // 故意泄漏(1)
    Node* node = (Node*)malloc(sizeof(Node));   // 故意泄漏(2)
    (void)buffer;
    (void)node;
}

int main() {
    // 打开分配跟踪，并让程序退出时自动做一次泄漏检查
    _CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);
    // 警告信息送到调试输出（在调试器中运行时显示于“输出”窗口）
    _CrtSetReportMode(_CRT_WARN, _CRTDBG_MODE_DEBUG);

    LeakyFunction();

    _CrtDumpMemoryLeaks();   // 也可在任意位置手工立即转储

    std::cout << "程序结束" << std::endl;
    return 0;
}
```

在调试器中按 F5 运行，**输出**窗口会出现 Detected memory leaks! 报告，每条记录形如：

```
d:\code\leakdemo.cpp(16) : {123} normal block at 0x000001F4A2C0B0A0, 400 bytes long.
```

包含分配位置、分配序号与字节数。

关键技巧：

- 花括号里的数字是**分配序号**。调用一次 `_CrtSetBreakAlloc(123);`，程序会在该次分配时中断，此时看调用栈即可精确定位。
- 若程序不是从调试器启动（例如被脚本直接运行），`_CRTDBG_MODE_DEBUG` 的输出无人接收，可改为写标准错误：`_CrtSetReportMode(_CRT_WARN, _CRTDBG_MODE_FILE)` 搭配 `_CrtSetReportFile(_CRT_WARN, _CRTDBG_FILE_STDERR)`。
- 想测量“某段代码前后堆增长了多少”，用检查点对比比读泄漏报告更直接：

```
_CrtMemState before, after, diff;
_CrtMemCheckpoint(&before);  DoWork();  _CrtMemCheckpoint(&after);
if (_CrtMemDifference(&diff, &before, &after)) _CrtMemDumpStatistics(&diff);
```

注意 手工调用 `_CrtDumpMemoryLeaks()` 时，若还有静态对象或全局容器未析构，它们持有的内存会被误报为泄漏；改用 `_CRTDBG_LEAK_CHECK_DF` 让 CRT 在退出阶段统一检查通常更准。

10.7 代码度量

菜单 **分析 > 计算代码度量值**会为每个项目、命名空间、类与成员算出一组数值，可导出 Excel 做趋势跟踪。

指标	含义	参考区间	偏高时的手段
可维护性指数	综合圈复杂度、代码行数与 Halstead 体积的 0–100 分	越高越好，低于 20 难以维护	拆分函数、消除重复、简化分支
圈复杂度	独立执行路径数，每个判定分支 +1	1–10 良好，超过 20 考虑重构	提前返回、提取策略、表驱动替代分支
继承深度	继承链上的类层数	越浅越好	用组合替代继承
类耦合	一个类引用的其他类数量	越少越好	缩小接口、引入抽象层
源代码行数	物理行数	—	结合可维护性指数一起看
可执行代码行数	可执行语句行数	—	关注单函数级别的异常值

提示 代码度量对 C++ 的支持程度随版本与项目类型有所不同，部分指标在个别项目上可能不可用。数值适合观察**趋势**与定位异常文件，不适合当作个人考核指标。

10.8 实战：一次完整的性能优化

任务：读取约 200 MB 的文本文件（每行一个 0–9999999 的整数），统计每个值出现的次数并输出前 10 名。

基线实现 v1：用 `std::map<int, int>` 计数，逐行 `std::getline` 加 `std::stoi` 解析——直观、正确，但每次自增都要做一次红黑树查找，每个新键都要分配一个节点。

先固定测量条件：Release 配置加 `/Zi`（保留调试信息以获得符号化调用栈）、关闭实时病毒扫描、重复运行三次取中位数。基线结果约 **3.8 s**。

按 `Alt+F2` 打开性能探查器，选“CPU 使用率”，采集后看函数列表：

排名	函数	自身耗时占比	说明
1	`std::_Tree` 的查找例程	约 42%	`std::map` 的红黑树查找
2	解析相关（`std::stoi` 路径）	约 21%	每行一次的字符串解析
3	`operator new` / 分配例程	约 13%	每个新键都要分配一个树节点
4	`std::getline`	约 8%	逐行读取

结论清晰：大部分时间花在容器与逐行 I/O 上，而不是统计逻辑本身。

第一轮（数据结构）：值域是 0–9999999，可以直接用数组做计数桶，把树查找换成一次随机访问：`std::vector<uint32_t> counts(10'000'000, 0);`（约 40 MB，一次分配）。重测约 **1.1 s**，热点转移到字符串解析。

第二轮（I/O 与编译选项）：整块读入文件后手写解析，避免 `std::stoi` 的临时字符串与异常框架；编译选项加入 `/GL` 并配合链接器 `/LTCG`，需要时再加 `/arch:AVX2`：

```
cl /nologo /O2 /GL /Zi /EHsc /std:c++20 /arch:AVX2 v3.cpp /link /LTCG
```

版本	关键改动	耗时	峰值内存	主要瓶颈
v1	`std::map` + `std::getline` + `std::stoi`	3.8 s	约 120 MB	树查找、节点分配
v2	`std::vector` 直接索引	1.1 s	约 60 MB	字符串解析
v3	整块读取 + 手写解析 + `/GL` `/LTCG` `/arch:AVX2`	0.4 s	约 45 MB	已接近磁盘读取下限

（以上为示意数据，你的机器上会不同。）三点经验：**算法与数据结构的收益远大于编译开关**；**每次只改一件事并重新测量**，否则无法判断哪一步有效；**性能配置要保留 `/Zi`**，纯为“更快”而关掉调试信息只会让下一次排查更困难。较新版本支持把两次采集的报告放在一起比较，可以直接看到每个函数的耗时变化。

10.9 小结

- 调试期间的**诊断工具**窗口（**调试 > 窗口 > 显示诊断工具**）提供 CPU、内存、事件与时间线，适合快速定位。
- **性能探查器**（**调试 > 性能探查器**，`Alt+F2`）提供采样、插桩、内存、文件 I/O、并发可视化等工具，适合系统性测量。
- 采样开销低但可能漏掉小函数；插桩精确但开销大且受内联影响。先采样找方向，再插桩做定量确认。
- **Build Insights** 是 MSVC 专属的编译期分析能力，用于查找编译慢的头文件与模板实例化，`vcperf` 可脚本化采集。
- 静态分析（`/analyze`、C++ Core Guidelines、SAL）在编译期发现问题，clang-tidy 补充风格与现代化建议。
- 运行时检查三件套：`/fsanitize=address` 抓越界与释放后使用，`/RTC1` 抓栈破坏与未初始化变量，`_CrtDumpMemoryLeaks` 与 `_CrtSetBreakAlloc` 抓内存泄漏。
- 优化流程固定为：建立基线 -> 测量定位热点 -> 只改一处 -> 重新测量 -> 记录对比。

10.10 练习题

1. 用调试期间的诊断工具窗口分析自己写的一个程序，找出自身耗时最高的三个函数，并说明各自的调用次数与平均单次耗时。
2. 对同一段代码分别用“CPU 使用率”与“检测”两种工具采集，比较函数耗时数据的差异，并解释为什么插桩结果在开启内联后会失真。
3. 为一个小型项目生成 Build Insights 跟踪，找出被间接包含次数最多的头文件，用前置声明或 pimpl 降低包含成本，并对比重新构建的时间。
4. 在一个故意包含空指针解引用与缓冲区越界的程序上，分别用 `/analyze`、`/fsanitize=address` 与 `/RTC1` 运行，记录各自能捕获哪些问题、漏掉哪些问题。
5. 写一个含三处内存泄漏的程序（`new[]`、`malloc`、容器内指针），用 `_CrtSetDbgFlag` 与 `_CrtSetBreakAlloc` 定位到具体行号。
6. 用 `.clang-tidy` 为本项目启用 `bugprone-*` 与 `performance-*`，统计修复前后错误列表的条目数量，并说明你关闭了哪些检查项及理由。

10.11 相关笔记

- **本教程**：**Visual Studio 2026 使用教程**
- **上一章**：**调试技术详解**
- **下一章**：**CMake 与 vcpkg 集成**
- **C++ 教程**：**Visual Studio 入门**

- **Eigen 库：Eigen 库使用指南**

第十一章　CMake 与 vcpkg 集成

在 Visual Studio 2026 里写 C++，项目系统有两条路：MSBuild 的 `.vcxproj` 与 CMake。前者是 Windows 的原生项目系统，后者是跨平台的构建描述标准。本章不教 CMake 语法——那是 **CMake 使用指南**的活——只讲两件事：**VS 2026 怎么消费 CMake 项目**，以及**怎么让 vcpkg 自动把第三方库接进这套流程**。

11.1　为什么在 VS 里用 CMake

11.1.1　`.vcxproj` 的边界与 CMake 的一等公民地位

MSBuild 项目把源文件清单、编译链接选项、以及 Windows 专属属性（平台工具集、SDK 版本、配置与平台映射）都写死在 XML 里。它在 Windows 上很成熟（见**第 06 章**），但**只在 MSBuild 里成立**：一旦需要 Linux/macOS 构建——哪怕只是让 CI 跑一遍 Linux 单测——就得另维护一份 Makefile 或 CMake，两份描述迟早漂移。相比之下，CMake 在 VS 2026 里是**一等公民**，不是插件：

- **不需要任何扩展**：**文件 > 打开 > 文件夹**就能直接开工。
- **CMake 目标视图**与解决方案资源管理器同级，目标直接列出，右键即可生成、调试。
- 调试、IntelliSense、测试资源管理器、性能探查器、clang-tidy、AddressSanitizer 对 CMake 项目**全部可用**。
- **自带 CMake 4.1.1**，也允许改用你自己安装的版本，并直接读取 `CMakePresets.json` 这一跨编辑器标准格式。

> **提示** 只要项目要在 Windows 之外构建，或团队里有人用 VS Code/CLion，第一天就选 CMake。MSBuild 只适合“确定一辈子只在 Windows 上”的场景。

11.1.2　两条路的能力对比

能力维度	MSBuild 项目（`.vcxproj`）	CMake 项目
项目文件	`.sln` + `.vcxproj` + `.filters`，XML	`CMakeLists.txt`，纯文本

能力维度	MSBuild 项目（.vcxproj）	CMake 项目
跨平台构建	[X] 只能由 MSBuild 在 Windows 上构建	[OK] Windows / Linux / macOS 通吃
VS 中的界面	解决方案资源管理器、属性页、属性表	文件夹视图 + CMake 目标视图，无属性页
选项配置入口	图形化属性页（字段极多）	`target_*` 命令 + `CMakePresets.json`
多配置支持	原生（Debug/Release 各有一套属性）	取决于生成器：VS 生成器支持，Ninja 单配置
依赖管理	NuGet、手工配置	`find_package`、FetchContent、**vcpkg**

注意 两条路**不是互斥的**。CMake 能反向生成 `.sln`/`.vcxproj`（见第 5.2 节），“CMake 做主干 + 生成项目文件交给只有 MSBuild 的 CI”完全成立。

11.2 在 VS 中打开 CMake 项目

11.2.1 用“打开文件夹”，不是“打开项目”

CMake 项目**没有 `.sln` 可打开**，这是新手第一个卡点。正确入口是 **文件 > 打开 > 文件夹**，选中含根 `CMakeLists.txt` 的目录。打开后 VS 会：

1. 把根 `CMakeLists.txt` 当作构建描述入口；
2. 找 **`CMakePresets.json`**（首选）或 **`CMakeSettings.json`**（旧格式）生成配置列表；
3. 找 **`vcpkg.json`**，若存在就准备激活 vcpkg 环境（见第 8 节）；
4. 读取 `.editorconfig`、`.gitignore` 与 `.vs/` 里的状态。

提示 一个工作区只应有一个构建入口。多个互不相关的项目塞进同一文件夹会让配置下拉变乱，多项目请用 `add_subdirectory()` 组织。

11.2.2 文件夹视图与 CMake 目标视图

解决方案资源管理器左上角有**视图切换下拉**，在两套视角之间切换：

```
解决方案资源管理器（CMake 项目的两种视角）

[文件夹视图] 编辑源码、增删文件的视角
MyApp/
+-- src/main.cpp
+-- include/math.h
+-- CMakeLists.txt
+-- CMakePresets.json
+-- vcpkg.json
```

```
[CMake 目标视图] 构建、调试、选启动项的视角
CMake Targets
+-- MyApp
|   +-- myapp.exe   <- 启动项
+-- libs
    +-- geometry_lib (STATIC)
```

两套视角**可以不一致**：磁盘上有文件但没被任何目标引用，就只在文件夹视图里出现；目标生成的中间产物则只在目标视图里有意义。

11.2.3 首次配置与工具包（Kit）选择

工具栏的**配置下拉**列出可用配置；第一次配置时 VS 会给出模板供选择（本机 MSVC、本机 Clang、WSL、远程 Linux 等），选定后自动跑一次 **configure**，输出在 **输出 > 显示输出自 > CMake**，成功的标志是构建目录里出现 `CMakeCache.txt`。

菜单命令	等价命令	用途
项目 > 配置缓存	`cmake -S . -B <dir>`	重新 configure，改 `CMakeLists.txt` 会自动触发
项目 > 生成 / 全部生成	`cmake --build <dir>`	增量构建启动目标 / 构建所有目标
项目 > 删除缓存并重新配置	删 `CMakeCache.txt` 后重配	换生成器、工具链、工具集后必做

注意 `CMakeCache.txt` 存的是**第一次 configure 时确定的值**（生成器、编译器路径、工具链文件、各种 `-D` 变量）。只点“配置”不会刷新它们——这是第 11 节头号坑的来源，也是下一节推荐 Presets 的原因。

11.3 CMakePresets.json：团队协作的推荐方式

11.3.1 为什么它取代了 CMakeSettings.json

`CMakeSettings.json` 是 **VS 私有**格式；`CMakePresets.json` 是 **CMake 官方标准**，`cmake --preset <名字>`、VS Code、CLion、CI 都认。

维度	`CMakeSettings.json`（旧）	`CMakePresets.json`（推荐）
归属	Visual Studio 私有	CMake 官方标准格式
命令行 / 其它编辑器	[X] 都不可用	[OK] `cmake --preset x64-debug`，VS Code、CLion 通用

维度	CMakeSettings.json（旧）	CMakePresets.json（推荐）
继承与 CI	复用能力有限，CI 得另写命令	[OK] inherits 多层继承，CI 复用同一个 preset

注意 两者同时存在时 VS 以 CMakePresets.json 为准，CMakeSettings.json 被忽略。迁移时先把旧文件删掉或改名，免得改了不生效还找不到原因。

11.3.2 一份可用的 CMakePresets.json

放在工作区根目录，与 CMakeLists.txt 同级：

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "windows-base",
      "hidden": true,
      "generator": "Visual Studio 18 2026",
      "binaryDir": "${sourceDir}/out/build/${presetName}",
      "cacheVariables": {
        "CMAKE_GENERATOR_PLATFORM": "x64",
        "CMAKE_GENERATOR_TOOLSET": "version=14.5x",
        "VCPKG_PLATFORM_TOOLSET_VERSION": "14.5x",
        "CMAKE_TOOLCHAIN_FILE": "$env{VCPKG_ROOT}/scripts/buildsystems/vcpkg.cmake
    "
      }
    },
    { "name": "x64-debug", "displayName": "x64 Debug (VS 2026) ",
      "inherits": "windows-base", "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug"
    } },
    { "name": "x64-release", "inherits": "windows-base",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "RelWithDebInfo" } }
  ],
  "buildPresets": [
    { "name": "x64-debug", "configurePreset": "x64-debug", "configuration": "Debug
    " },
    { "name": "x64-release", "configurePreset": "x64-release", "configuration": "
    RelWithDebInfo" }
  ]
}
```

字段	必需	作用
`version`	[OK]	**预设 schema 的版本**，不是文档版本。数值越大解锁字段越多，对 CMake 最低版本要求也越高——团队里取“所有人都能满足的最低值”
`configurePresets`	[OK]	配置预设数组，每项描述“怎么跑一次 configure”
`name` / `hidden`	[OK] / [X]	预设的唯一标识（供 `inherits` 与 `cmake --preset` 引用）；`hidden` 为 `true` 时不出现在下拉里，只作基类
`generator` / `binaryDir`	[X]	生成器名；构建目录，支持 `${sourceDir}`、`${presetName}` 宏
`cacheVariables` / `inherits`	[X]	以 `-D` 传给 CMake 的缓存变量；继承另一预设，子项覆盖父项
`buildPresets` / `testPresets`	[X]	构建预设(用 `configurePreset` 回指,`configuration` 选配置)、测试预设（见**第 11 章**）

提示 `CMAKE_GENERATOR_PLATFORM` 与 `CMAKE_GENERATOR_TOOLSET` 分别对应命令行的 `-A x64` 与 `-T`，比用某些 schema 版本才有的图形字段更稳；`displayName` 是下拉里给人看的名字，VS 优先显示它。

保存 Presets 后，VS 会立刻在工具栏**配置下拉**里列出所有非隐藏预设，同时 **项目 > 配置 < 项目名 >** 也列出同一批；文件被监视，一保存就自动重新 configure。整条链路上，**Presets 是配置的唯一真源**，VS 只负责把它翻译成一次 configure 调用。

11.4 CMake 设置编辑器

11.4.1 界面构成与 Presets 的取舍

项目 > CMake 设置打开 **CMake 设置编辑器**——一个为 `CMakeSettings.json` 设计的图形界面：左栏是配置列表（可增删）；中栏是选中配置的字段（名称、生成器、平台、工具集、构建根目录、CMake 与构建命令参数）；下栏是**缓存变量表**，逐行填“名字 / 类型 / 值”（`BOOL/STRING/PATH/FILEPATH`），与 `cacheVariables` 一一对应。它的价值在于**不用背 JSON 字段名**，但**只覆盖一部分字段**：存在 `CMakePresets.json` 时，`inherits`、`hidden`、`buildPresets`、条件表达式这类结构性内容无法在图形界面里表达，只能直接编 JSON。

你的处境	建议
新建项目、团队协作、要接 CI	直接用 `CMakePresets.json`，编辑器只当“看一眼缓存变量”的工具
已有 `CMakeSettings.json` 且在跑	先别动；等要接 CI 或 VS Code 时一次性迁移到 Presets

提示 只要是**缓存变量**，改完都要重新配置。VS 一般会提示“CMake 缓存已过期，是否重新配置”——选是，别选忽略。

11.5 生成器与工具集

11.5.1 Visual Studio 18 2026 生成器、-T 与自带的 CMake

CMake 为 VS 2026 提供的生成器名是 `Visual Studio 18 2026`（与 VS 2022 的 `Visual Studio 17 2022` 同一命名规律）。在 VS 里通常不必敲命令行，把 `generator` 写进 Presets 即可，效果一致：

```
:: 用 VS 2026 生成器配置；-A 指定平台，-T 指定工具集版本
cmake -S . -B out\build\vs -G "Visual Studio 18 2026" -A x64 -T "version=14.5x"
cmake --build out\build\vs --config Release      :: 多配置生成器必须带 --config
```

- `-T "version=14.5x"` 里的 `14.5x` 代表 **14.50 / 14.51 这一档**；想精确锁死就写完整版本号（如 `version=14.50.35717`），构建才可复现。**指定的工具集必须已安装**，否则配置阶段直接失败——用 VS 安装程序补装“MSVC v145 生成工具”；在 VS 的 CMake 项目里它对应缓存变量 `CMAKE_GENERATOR_TOOLSET`，最终同样变成 `-T`。
- 别把两件事搞混：`v145` **是平台工具集家族**，`14.50/14.51` 是家族内的具体版本（见**第 1 章**）。

VS 2026 还在安装目录的 `Common7\IDE\CommonExtensions\Microsoft\CMake` 子树里自带一份 **CMake 4.1.1**，所以**不装 CMake 也能直接开工**；想改用自己安装的版本，在 VS 的选项里搜索 `CMake` 指定可执行文件路径。

注意 CMake 4.x 移除了对 `cmake_minimum_required(VERSION < 3.5)` 的兼容。老项目里写着 `cmake_minimum_required(VERSION 3.0)` 的，用自带的 CMake 配置会直接报错——把版本号提到 3.5 以上即可。这是升级到 VS 2026 后 CMake 项目最常见的第一个报错。

11.5.2 反向：从 CMake 生成并构建 Visual Studio 项目

链路是双向的。用 VS 生成器 configure 之后，构建目录里就是货真价实的 `.sln` + `.vcxproj`，可以用 MSBuild 构建，也可以直接双击打开；VS 2026 支持新的 **SLNX** 解决方案格式，这套“CMake 生成 VS 解决方案 -> 在 VS 里构建”的流程在这一代更顺畅。需要它的场合：CI 上只有 MSBuild、想用 MSBuild 侧的完整项目系统工具、要给不用 CMake 的同事交付。

注意 生成出来的 `.sln` / `.vcxproj` 是**产物而非源码**，手工改了下次 configure 就被覆盖。要长期生效的修改一律写回 `CMakeLists.txt`。

11.6 构建与调试 CMake 目标

11.6.1 选择启动项、断点与 F5

VS 需要知道 F5 启动哪个 exe：打开 **CMake 目标视图**，对目标右键 **设为启动项**（工具栏绿色三角旁的下拉也能切换），启动配置会写进 `.vs` 目录下的 `launch.vs.json`——**这是本机状态，应加入 `.gitignore`**。之后调试体验与 `.vcxproj` 项目完全一致：F9 断点、F5 启动、F10/F11 单步，**监视/调用堆栈/即时窗口**全部可用，工具栏还能直接填**命令行参数与工作目录**。唯一区别是“调试器怎么找到 exe”：VS 从 CMake 目标信息推断输出路径，而不是读 `.vcxproj` 的 `OutDir`——你在 `CMakeLists.txt` 里设了 `RUNTIME_OUTPUT_DIRECTORY`，调试器也会跟着走。

11.6.2 调试 CMakeLists.txt 脚本本身

CMake 脚本可以被调试，很多人不知道：在 `CMakeLists.txt`（或任何被 `include()` 的 `.cmake` 文件）里按 F9 设断点，然后执行 **项目 > 使用 CMake 调试配置缓存**。VS 会以“调试脚本”的方式跑 configure，停在断点时可查看 `${}` 变量的实际值、单步走 `if()` 分支。

症状	在断点处看什么
`find_package` 找不到包	`XXX_FOUND`、`XXX_DIR` 到底是什么
条件分支走错 / 路径拼接错误	参与 `if()` 判断的变量；`${CMAKE_CURRENT_SOURCE_DIR}`、`${CMAKE_BINARY_DIR}`

注意 脚本调试只覆盖 configure 阶段；构建阶段的问题（编译、链接报错）仍按**第 08 章**排查。按 F10 会让根 `CMakeLists.txt` 每一行都停一下，长脚本优先用条件断点。

11.6.3 CMake 模块的 IntelliSense

VS 对 CMake 语言本身提供编辑器支持：`include(...)` 引入的 **CMake 模块有补全**，鼠标悬停在命令、模块、变量上有**快速信息**（签名与说明），`${}` 变量引用有补全与跳转。能做到这些，是因为 VS 在后台持续通过 CMake 的文件 API 解析项目结构。

实战场景 一个项目在 Windows 上 `find_package(Qt6)` 正常、配到 WSL 就失败。与其反复加 `message()` 重配，不如在 `find_package` 后面打个断点，用 **项目 > 使用 CMake 调试配置缓存**跑一次——两分钟就能看清是 `Qt6_DIR` 为空，还是 Linux 侧的 `CMAKE_PREFIX_PATH` 没设。

11.7 CMake 项目的多配置

11.7.1 单配置与多配置生成器的差别

生成器	类型	说明
Ninja / Unix Makefiles	**单配置**	一个构建目录只对应一个配置，编译快，WSL/远程 Linux 的默认选择
Visual Studio 18 2026	**多配置**	一个构建目录装下 Debug/Release/RelWithDebInfo，由 MSBuild 管理
Ninja Multi-Config	**多配置**	Ninja 的多配置版本，跨平台，兼顾速度与多配置

```
单配置（Ninja）：每个配置一个构建目录，各有一份 CMAKE_BUILD_TYPE
out/build/
+-- x64-debug/
|   +-- CMakeCache.txt
|   +-- myapp.exe        <- Debug
+-- x64-release/
    +-- CMakeCache.txt
    +-- myapp.exe        <- Release

多配置（Visual Studio 18 2026）：一个构建目录装下全部配置
out/build/
+-- vs-x64/
    +-- CMakeCache.txt
    +-- MyApp.sln
    +-- Debug/myapp.exe
    +-- RelWithDebInfo/myapp.exe
```

动作	单配置（Ninja）	多配置（Visual Studio 生成器）
选配置的时机	**configure 时**定死	**build 时**决定
用什么指定 CMAKE_BUILD_TYPE 是否有效	-DCMAKE_BUILD_TYPE=Release [OK]	--config Release [X] **被忽略**，不报错也不生效
同时要 Debug 与 Release	开两个构建目录	同一目录切配置（VS 里就是工具栏配置下拉）

注意 **“多配置生成器里写 CMAKE_BUILD_TYPE 不生效”是最常见的沉默失败**——不报错，只是被忽略。用多配置生成器时，配置一律通过 buildPresets 的 configuration 字段或 --config 指定。生成器表达式 $<CONFIG:...> 在两种生成器下都可用。

11.8 vcpkg 集成

11.8.1 清单模式与经典模式

维度	经典模式（Classic）	清单模式（Manifest，推荐）
依赖声明位置	无，靠命令行 `vcpkg install`	项目根目录的 `vcpkg.json`
库安装到哪	vcpkg 安装目录的 `installed/`	**项目构建目录**下的 `vcpkg_installed/`
版本可复现	[X] 换个时间装可能拿到别的版本	[OK] 由 `builtin-baseline` 锁定
机器间一致性	靠文档与口头约定	依赖清单**提交进版本库**即可
多项目共存	全局安装的库互相干扰	各项目独立，互不影响
VS 中的体验	需手工配工具链	**检测到 `vcpkg.json` 后自动生效**

也就是说，只要工作区根目录有 `vcpkg.json`，VS 就会自动把它自带的 vcpkg 接进 CMake 配置——**不需要你手写 `CMAKE_TOOLCHAIN_FILE`**，输出窗口的 CMake 面板里能看到 vcpkg 正在解析清单、安装依赖。想改用你自己克隆的 vcpkg（团队统一版本时常用），设置环境变量 `VCPKG_ROOT` 指向那份克隆即可。

注意 vcpkg 首次配置可能极慢。依赖没有预编译二进制时 vcpkg 会从源码编译，带 GUI/网络依赖的库编译半小时以上很常见，而且**每个配置各编一份**。第一次配完就去做别的事，别中途反复取消重配，那只会更慢。

11.8.2 一份 vcpkg.json 示例

```
{
  "name": "myapp",
  "version": "1.0.0",
  "dependencies": [
    "fmt",
    { "name": "curl", "default-features": false, "features": [ "ssl" ] }
  ],
  "builtin-baseline": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0",
  "overrides": [ { "name": "fmt", "version": "10.2.1" } ],
  "features": {
    "tests": { "description": "构建单元测试", "dependencies": [ "gtest" ] }
  }
}
```

字段	作用
name / version	项目自身标识，清单模式要求
dependencies	依赖列表。可只写包名，也可写成对象以精确控制特性；default-features 设 false 可避免拖进用不到的东西
builtin-baseline	**锁定基线**：vcpkg 仓库的某个提交号，决定所有依赖解析到的版本。**必须换成你自己 vcpkg 仓库的真实提交号**（git -C <vcpkg 目录> rev-parse HEAD）
overrides	强制某依赖用指定版本，优先级高于基线，用于临时绕开有问题的版本
features	顶层 features 是**本项目对外暴露的特性**（消费方可用 "myapp[core,tests]" 引入）；依赖对象里的 features 则是只启用该依赖的指定特性

提示 特性名（如示例里的 ssl）以你所用的基线为准——vcpkg search < 包名 >，或直接查看官方 ports 目录里该包的 vcpkg.json 最可靠。

11.8.3 vcpkg-configuration.json 与注册表

vcpkg.json 管“要什么”，vcpkg-configuration.json 管“从哪儿拿”：

```
{
  "default-registry": { "kind": "builtin", "baseline": "a1b2c3…" },
  "registries": [
    { "kind": "git", "repository": "https://example.com/your-org/vcpkg-registry.
    git",
      "baseline": "f0e1d2c3b4a5968778695a4b3c2d1e0f1a2b3c4d",
      "packages": [ "your-company-lib" ] }
  ]
}
```

概念上分三类：**内置注册表（builtin）**指 vcpkg 自带的官方 ports 集合；**git 注册表**指你自己或公司维护的 ports 仓库，用来分发私有库与打过补丁的第三方库；**baseline** 是注册表的基线提交号，与清单里的 builtin-baseline 同一套机制——**没有基线，版本就不可复现**。packages 声明该注册表负责哪些包，其余仍走默认注册表；同一个包只能由一个注册表提供，重叠声明会报错，私有包记得加组织前缀避免与官方包重名。

11.8.4 工具集对齐与 CMAKE_TOOLCHAIN_FILE

vcpkg 编译依赖时用的编译器必须与主项目一致，否则会出现“依赖用 v143 编、主程序用 v145 链”的诡异错误。**VCPKG_PLATFORM_TOOLSET_VERSION** 就是给 vcpkg 指定 MSVC 平台工具集版本的缓存变量，取值与 -T 的 version= 同一套版本号；把它与 CMAKE_GENERATOR_TOOLSET 写成同一个数并提交进 Presets 最省事：

```
"cacheVariables": {
  "VCPKG_PLATFORM_TOOLSET_VERSION": "14.50",
  "CMAKE_GENERATOR_TOOLSET": "version=14.50",
  "CMAKE_TOOLCHAIN_FILE": "$env{VCPKG_ROOT}/scripts/buildsystems/vcpkg.cmake",
  "VCPKG_TARGET_TRIPLET": "x64-windows"
}
```

在 VS 里用清单模式通常不必写工具链文件，但**命令行构建、CI、以及不用 VS 内置 vcpkg 的场景**必须自己指定：

```
cmake -S . -B out\build\vs -G "Visual Studio 18 2026" -A x64 ^
      -DCMAKE_TOOLCHAIN_FILE=%VCPKG_ROOT%/scripts/buildsystems/vcpkg.cmake ^
      -DVCPKG_TARGET_TRIPLET=x64-windows
```

- `CMAKE_TOOLCHAIN_FILE` 指向 vcpkg 的工具链脚本，它让 `find_package` 自动找到 vcpkg 装的库；`$env{VCPKG_ROOT}` 是预设里的环境变量宏，路径分隔符统一用 `/`。
- `VCPKG_TARGET_TRIPLET` 指定三元组（平台 + 运行时 + 链接方式）：`x64-windows` 是动态链接，静态链接用 `x64-windows-static`。**三元组必须与项目的运行库设置一致**，否则链接期会爆重复符号或找不到符号（见**第 12 章**）。
- 工具链文件是**缓存变量，只在第一次 configure 时生效**，改了它必须 **项目 > 删除缓存并重新配置**。

11.9 手动管理第三方库的替代方案

11.9.1 直接改目录，或者用 NuGet

最原始的办法是把路径手工填进构建系统：MSBuild 项目在 **项目 > 属性 > C/C++ > 常规 > 附加包含目录**填头文件目录、**链接器 > 常规 > 附加库目录**填 `.lib` 目录、**链接器 > 输入 > 附加依赖项**写库名（字段详情见**第 06 章**）；CMake 项目本质相同，只是换成 `target_include_directories()` 与 `target_link_libraries()`。它的问题随规模放大：**路径写死**（换机器就要重配）、**Debug/Release 各配一遍**（`fmtd.lib` 与 `fmt.lib` 容易错配）、**传递依赖要手工补、版本信息不在仓库里**（新人不知道装哪个版本）。NuGet 则是另一条路，但用在 C++ 上限制明显：包必须提供 `.props/.targets` 才能自动接线，C++ 包数量远少于 vcpkg 且大量 Windows-only，**对 CMake 项目的支持也弱**（原生集成主要面向 MSBuild 项目）。

11.9.2 为什么推荐 vcpkg

方案	版本可复现	跨平台	传递依赖	与 CMake 配合	适合场景
手工改包含/库目录	[X]	[X]	[X]	一般	一次性小工具、单个头文件库
NuGet	[OK]	[X]	[OK]	弱	MSBuild 项目 + Windows 专属包
FetchContent 拉源码	[OK]（锁 tag）	[OK]	[OK]	[OK]	依赖少、愿意每次编译依赖
vcpkg 清单模式	[OK]	[OK]	[OK]	[OK]	**正式项目的默认选择**

提示 vcpkg 生态里找不到的库比想象中少，但确实存在（许可证受限、内部私有库）。私有库用第 8.3 节的**自定义 git 注册表**解决，比退回手工配路径划算得多。

11.10 WSL / 容器 / 远程 Linux 开发

11.10.1 WSL

（1）在 VS 安装程序里勾选工作负荷 **“使用 C++ 的 Linux、Mac 和嵌入式开发”**（旧名“C++ Linux”，本代改了名，按新名字搜）；（2）在 WSL 发行版里装工具链 `sudo apt install build-essential gdb cmake ninja-build`；（3）**文件 > 打开 > 文件夹**打开项目，在**配置下拉**里选面向 WSL 的配置（或在 Presets 里加一个使用 WSL 工具包的配置）；（4）构建并 `F5`，调试由 WSL 里的 `gdb` 承担，断点、单步、调用堆栈与本地一致。关键是 **IntelliSense 也在 Linux 侧运行**，Linux 特有的头文件路径与宏才准确，否则会看到一堆“找不到 `<unistd.h>`”的假报错。

提示 项目放 WSL 的**原生文件系统**（`/home/< 用户 >/...`）而不是 `/mnt/c/...`。跨文件系统访问的 I/O 开销很大，大型项目的构建与 IntelliSense 速度差距非常明显。

11.10.2 Dev Containers（Docker 与 Podman）

容器路线把“开发环境”本身纳入版本控制：仓库里放一个 `.devcontainer/devcontainer.json` 描述基础镜像、工具链与挂载；打开该文件夹后 VS 提供**在容器中打开**的入口，之后的编辑、构建、调试都在容器内进行。**Docker 与 Podman 都支持**，不是只有 Docker 一条路；VS 2026 的**附加到进程**从 18.10 起支持 **Podman** 容器，并且**可以调试容器内的 C++ 与.NET 进程**。

注意 容器里必须装有调试工具链（`gdb`、编译器等），否则只能构建不能调试；容器内外的**路径映射**也要对上，`devcontainer.json` 的挂载写错，断点就会变成空心的。

11.10.3 远程 Linux（SSH）与远程单元测试

连一台真正的 Linux 机器（物理机、虚拟机或服务器）：在 **工具 > 选项 > 跨平台 > 连接管理器**里添加 SSH 连接（菜单名在不同版本间略有差异，找不到就在选项里搜“跨平台”）；在配置下拉里选该远程连接对应的配置，或自己写一个指定远程编译器与 Ninja 生成器的预设；configure 与 build 在远端执行，调试通过远端 `gdbserver` 完成。**测试资源管理器还能发现并运行远端机器上的单元测试**（远程单元测试），配合测试预设更好用（见**第 11 章**）。三条路线怎么选：**WSL** 适合个人在 Windows 与 Linux 间来回切，最省事；**Dev Containers** 适合团队要求“环境一致、开箱即用”；**远程 Linux（SSH）**适合目标就是真实服务器、发行版或交叉编译的场景。

实战场景 一个只在 Windows 上维护的项目要接 Linux CI。与其在 CI 里重写构建脚本，不如先在 VS 里连一台 Linux 跑通 configure——**同一份 `CMakeLists.txt` 与 `CMakePresets.json`，Windows 侧用 VS 生成器，Linux 侧加一个 Ninja 预设**，CI 里直接 `cmake --preset linux-release` 即可。这正是第 3 节推荐 Presets 的根本原因。

11.11 常见坑

现象	原因	处置
找不到编译器，或配置过了但构建命令跑不起来	**生成器与 Kit 不匹配**：Ninja 配了 MSVC 工具集，或 VS 生成器却指定 Linux 的 gcc	生成器与编译器必须成对：VS 生成器 <-> MSVC，Ninja <-> 显式指定 `CMAKE_CXX_COMPILER`
改了工具链、工具集或某个 `-D` 变量却完全不生效	**`CMakeCache.txt` 脏了**：缓存变量只写一次	**项目 > 删除缓存并重新配置**，或删掉构建目录里的 `CMakeCache.txt`
路径报错、找不到文件、输出乱码	**路径含中文或空格**：脚本与工具的引号处理不一致	项目路径只用 ASCII 字母、数字、下划线与短横线，避开 `C:\Program Files` 这类目录
报“找不到指定版本的 MSVC 工具集”	**工具集版本对不上**：`-T version=14.5x` 指定的版本没装	用安装程序补装，或改成已安装的版本；同时核对 `VCPKG_PLATFORM_TOOLSET_VERSION`
首次配置卡几十分钟，像死了一样	**vcpkg 首次构建极慢**：依赖要源码编译，且每个配置一份	耐心等完第一次，别中途取消；减少 `dependencies`、关掉用不到的 `features`

现象	原因	处置
Debug 里混进 Release 产物，或产物路径到处散落	**多配置目录混乱**：多配置生成器里写了 CMAKE_BUILD_TYPE，或 binaryDir 没带 ${presetName}	多配置一律用 --config / 配置下拉；binaryDir 统一写 ${sourceDir}/out/build/${presetName}

11.12 端到端实战：从零到 F5

11.12.1 目录结构

做一个用 fmt 打印格式化输出的控制台程序：Windows 上用 VS 2026 构建，依赖交给 vcpkg，配置由 Presets 描述。

```
MyApp/                                  <- 用 VS 打开的就是这个文件夹
+-- CMakeLists.txt                      <- 构建描述
+-- CMakePresets.json                   <- 配置描述（提交）
+-- vcpkg.json                          <- 依赖清单（提交）
+-- src/main.cpp
+-- out/build/x64-debug/                <- 全部产物，不进版本库
    +-- vcpkg_installed/x64-windows/{include,lib}   <- vcpkg 在这里装依赖
    +-- Debug/myapp.exe                             <- F5 启动的就是它
```

提示 .gitignore 里写 /out/、/.vs/、/vcpkg_installed/ 三行就够。构建目录与 vcpkg 安装目录都是**可再生**的，提交它们只会制造冲突。

11.12.2 三个文件

CMakeLists.txt——注意这里只有标准 CMake 写法，**没有任何 vcpkg 专属命令**：

```
cmake_minimum_required(VERSION 3.20)
project(MyApp VERSION 1.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)          # 对应属性页里的 /std:c++20
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(myapp src/main.cpp)
find_package(fmt CONFIG REQUIRED)   # vcpkg 装上 fmt 后就能直接找到
target_link_libraries(myapp PRIVATE fmt::fmt)
```

CMakePresets.json——实战先留一个最小配置（完整字段与解释见第 3.2 节），因为根目录有 vcpkg.json，VS 会自动注入 vcpkg 工具链：

```
{
```

```
  "version": 3,
  "configurePresets": [
    { "name": "x64-debug", "generator": "Visual Studio 18 2026",
      "binaryDir": "${sourceDir}/out/build/${presetName}",
      "cacheVariables": { "CMAKE_GENERATOR_PLATFORM": "x64" } }
  ],
  "buildPresets": [
    { "name": "x64-debug", "configurePreset": "x64-debug", "configuration": "Debug
    " }
  ]
}
```

vcpkg.json——把 builtin-baseline 换成你自己 vcpkg 仓库的真实提交号（git -C %VCPKG_ROOT% rev-parse HEAD）：

```
{ "name": "myapp", "version": "1.0.0", "dependencies": [ "fmt" ],
  "builtin-baseline": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0" }
```

```
#include <fmt/core.h>
#include <string>

int main() {
    std::string name = "VS 2026";
    fmt::print("Hello, {}!\n", name);
}
```

11.12.3 在 VS 中配置、构建、调试

1. **打开项目**：**文件 > 打开 > 文件夹**，选中 MyApp 目录。
2. **选择配置**：工具栏配置下拉里选 x64-debug，VS 自动跑第一次 configure。这一步会**下载并编译 fmt**，可能要等几分钟，进度在 **输出 > 显示输出自 > CMake**。
3. **确认配置成功**：构建目录里出现 CMakeCache.txt 与 vcpkg_installed/x64-windows/include/fmt/。
4. **设启动项并调试**：解决方案资源管理器切到 **CMake 目标视图**，右键 myapp -> **设为启动项**；在 src/main.cpp 的 fmt::print 行按 F9，按 F5，停在断点后在**监视**里看一眼 name，F10 单步，F5 继续。
5. **验证命令行一致**：在开发者命令提示符里跑 cmake --preset x64-debug 与 cmake --build --preset x64-debug，确认同一份 Presets 在 IDE 之外也能跑通。

实战场景 走到第 5 步时你已经具备接入 CI 的全部条件——**依赖声明在 vcpkg.json，配置在 CMakePresets.json，命令是 cmake --preset ... && cmake --build --preset ...**。构建机上只要装 VS Build Tools、设好 VCPKG_ROOT，就能得到与本地一致的结果，

不需要任何人肉步骤。

11.13 小结

- **CMake 在 VS 2026 里是一等公民**：无需扩展，**文件 > 打开 > 文件夹**即可获得目标视图、调试、IntelliSense 与测试支持；跨平台或多人协作优先选它，`.vcxproj` 留给纯 Windows 场景。
- 解决方案资源管理器可在**文件夹视图**（编辑）与 **CMake 目标视图**（构建、调试、选启动项）之间切换。
- **`CMakePresets.json` 是推荐的配置方式**：标准格式、支持 `inherits`、命令行与 CI 都能复用；存在时 VS 以它为准并忽略旧的 `CMakeSettings.json`，图形化的 **CMake 设置编辑器**只适合临时改缓存变量。
- VS 2026 **自带 CMake 4.1.1**，生成器是 `Visual Studio 18 2026`；用 `-T "version=14.5x"`（或缓存变量 `CMAKE_GENERATOR_TOOLSET`）选工具集版本，vcpkg 侧对应 `VCPKG_PLATFORM_TOOLSET_VERSION`。注意 CMake 4.x 不再兼容 `cmake_minimum_required(VERSION < 3.5)`。
- 在 CMakeLists.txt 里设断点 + **项目 > 使用 CMake 调试配置缓存**可以调试 CMake 脚本本身，是排查 `find_package` 失败的利器；CMake 模块（`include(...)`）还有补全与快速信息。
- **多配置生成器（Visual Studio）用 `--config` 选配置，`CMAKE_BUILD_TYPE` 会被静默忽略**；单配置生成器（Ninja）则必须在 configure 时用 `CMAKE_BUILD_TYPE` 定死。
- **vcpkg 清单模式优于经典模式**：依赖装进项目的 `vcpkg_installed/`，版本由 `builtin-baseline` 锁定，`vcpkg-configuration.json` 可接私有 git 注册表；VS 检测到 `vcpkg.json` 会自动激活环境，无需手写工具链文件。
- **WSL、Dev Containers（Docker 与 Podman）、远程 Linux（SSH）**三条路线都能在 Windows 上开发 Linux 目标；工作负荷已改名为 **“使用 C++ 的 Linux、Mac 和嵌入式开发”**，且附加到进程已支持 Podman 容器、可调试 C++ 容器进程。

11.14 练习题

1. 用 **文件 > 打开 > 文件夹**打开一个含 `CMakeLists.txt` 的目录，在**文件夹视图**与 **CMake 目标视图**间来回切换，记录两边“多出来”和“少掉”的条目并解释原因。
2. 手写一份含 `inherits` 的 `CMakePresets.json`：一个 `hidden` 基类预设加两个继承它的子预设，确认 VS 的配置下拉里只出现两个子预设。
3. 用 `Ninja` 与 `Visual Studio 18 2026` 把同一项目配到两个构建目录并各构建一次；在单配置那份里改 `CMAKE_BUILD_TYPE` 观察生效，在多配置那份里做同样的事并观察**不生效**。
4. 在 `CMakeLists.txt` 的 `find_package` 处设断点，用 **项目 > 使用 CMake 调试配置缓存**

启动，查看包目录变量在断点处的实际值，并说明它由谁设置。

5. 给项目加上 `vcpkg.json` 引入一个依赖，把 `builtin-baseline` 改成你本机 vcpkg 的真实提交号；记录首次配置耗时，删掉 `vcpkg_installed/` 再配一次，比较两次差异。
6. 用 `-T "version=14.50"` 与不指定 `-T` 各配置一次，在 `CMakeCache.txt` 或诊断输出里确认实际使用的工具集版本，写出结论。
7. 给含单元测试的项目补上 `testPresets`，并给 Linux 侧加一个 Ninja 预设；在 Windows 与 Linux（WSL 或 SSH 均可）各跑一次测试，确认同一份 Presets 两边都能工作。

11.15 相关笔记

- **本教程**：**Visual Studio 2026 使用教程**
- **上一章**：**性能分析与诊断**
- **下一章**：**单元测试与代码质量**
- **CMake 使用指南**：**CMake 使用指南**
- **VS Code 中 CMake**：**VS Code 中 CMake 使用指南**

第十二章　单元测试与代码质量

C++ 没有托管语言的运行时保护，一个越界写、一次漏掉的拷贝、一个忘记置空的指针，可能要到用户现场才暴露。单元测试是唯一能让你在几分钟内知道“刚才的改动有没有破坏别的东西”的手段，而 Visual Studio 2026 把测试发现、执行、覆盖率与代码质量检查都收进了同一个界面。本章从最省事的内置框架讲到第三方框架接入、覆盖率与 CI，目标是让你在真实项目里能立刻把测试跑起来。

12.1　为什么 C++ 项目需要单元测试

12.1.1　C++ 的沉默错误成本

C++ 的错误往往不在出错的那一行暴露：越界写会破坏相邻对象，悬垂引用会在几千行之外崩溃，未初始化的成员会让结果偶尔不同。这类问题的定位成本与“发现得有多晚”成正比。单元测试把发现时机提前到编译之后几秒内，并且**把每个 bug 都变成一条再也不会复发的用例**。此外，测试还是重构的安全网：没有测试的重构只能叫改写。

12.1.2　测试资源管理器

测试 > 测试资源管理器是测试工作的主界面：左侧是测试树，右侧是详情与结果，顶部工具栏提供全部运行、运行失败项、按结果/类/特征分组、搜索框与播放列表入口。它不关心你用的是哪套框架——只要项目里有框架的适配器，用例就会被发现并显示在同一棵树上。

```
测试资源管理器（示意）
+- 解决方案 MyApp
|  +- 未运行的测试 (12)
|  +- 已通过的测试 (35)
|  |  +- CalcTests
|  |     +- Add_TwoPositiveNumbers_ReturnsSum      通过  0.4 ms
|  |     +- Average_NormalInput_ReturnsMean        通过  0.6 ms
|  +- 失败的测试 (1)
|     +- Divide_ByZero_Throws
|           消息: Expected exception not thrown
|           堆栈: CalculatorTests.cpp 第 27 行
+- 摘要: 总计 48 | 通过 47 | 失败 1 | 跳过 0
```

提示 测试资源管理器是“生成后更新”的。改完代码先按 F6 生成解决方案，再运行测试，否则跑的还是上一次的二进制。

12.2 Microsoft 原生单元测试框架（C++）

12.2.1 创建本机单元测试项目

在 **文件 > 新建 > 项目**中搜索“本机单元测试项目”（Native Unit Test Project）即可创建。该模板属于 C++ 桌面开发工作负载，不需要安装任何第三方库，生成的项目会：

- 使用 `CppUnitTest.h` 提供的 Microsoft Unit Testing Framework for C++；
- 把配置类型设为动态库，产物是一个包含测试代码的 DLL，由 VS 的测试适配器加载执行；
- 自带一个示例测试类，可直接替换。

12.2.2 核心宏与断言

宏 / API	作用
`TEST_CLASS(名称)`	定义一个测试类，类内的 `TEST_METHOD` 才会被发现
`TEST_METHOD(名称)`	定义一个测试方法，方法名即用例名
`TEST_METHOD_INITIALIZE` / `TEST_METHOD_CLEANUP`	每个测试方法前后执行的初始化与清理
`TEST_CLASS_INITIALIZE` / `TEST_CLASS_CLEANUP`	整个测试类前后各执行一次
`Assert::AreEqual(期望, 实际)`	相等断言，类型需可比较与可打印
`Assert::AreNotEqual` / `Assert::IsTrue` / `Assert::IsFalse` / `Assert::IsNull` / `Assert::IsNotNull`	常用的其余断言
`Assert::ExpectException<T>(lambda)`	断言 lambda 抛出类型为 `T` 的异常
`Assert::Fail(消息)`	直接判定失败

12.2.3 完整示例：Calculator 与被测代码

被测代码放在一个静态库项目 `CalcLib` 中，测试项目引用它。

```
1 // CalcLib/include/Calculator.h —— 被测代码
2 #pragma once
3 #include <stdexcept>
4 #include <vector>
5
```

```
class Calculator {
public:
    int Add(int a, int b) const { return a + b; }

    int Divide(int a, int b) const {
        if (b == 0)
            throw std::invalid_argument("division by zero");
        return a / b;
    }

    // 平均值：空输入视为调用方错误
    double Average(const std::vector<int>& values) const {
        if (values.empty())
            throw std::invalid_argument("empty input");
        long long sum = 0;
        for (int v : values)
            sum += v;
        return static_cast<double>(sum) / static_cast<double>(values.size());
    }
};
```

```
// CalculatorTests.cpp —— 本机单元测试项目中的测试文件
#include "CppUnitTest.h"
#include "Calculator.h"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace CalcTests {

TEST_CLASS(CalculatorTests) {
public:
    // 每个测试方法执行前都会调用，避免用例之间共享状态
    TEST_METHOD_INITIALIZE(SetUp) { m_calc = Calculator(); }

    TEST_METHOD(Add_TwoPositiveNumbers_ReturnsSum) {
        Assert::AreEqual(5, m_calc.Add(2, 3));
    }

    TEST_METHOD(Add_NegativeNumbers_ReturnsSum) {
        Assert::AreEqual(-5, m_calc.Add(-2, -3));
    }

    TEST_METHOD(Divide_ByZero_Throws) {
        Assert::ExpectException<std::invalid_argument>(
            [this] { m_calc.Divide(1, 0); });
```

```
25      }
26
27      TEST_METHOD(Average_EmptyVector_Throws) {
28          Assert::ExpectException<std::invalid_argument>(
29              [this] { m_calc.Average({}); });
30      }
31
32      TEST_METHOD(Average_NormalInput_ReturnsMean) {
33          // 浮点比较用带容差的重载；若所用版本不提供，可改用 Assert::IsTrue(std::abs
        (...) < 1e-9)
34          Assert::AreEqual(2.0, m_calc.Average({ 1, 2, 3 }), 1e-9);
35      }
36
37      TEST_METHOD(Average_LargeInput_NoOverflow) {
38          std::vector<int> v(1000, 2000000000);
39          Assert::IsTrue(m_calc.Average(v) > 1999999999.0);
40      }
41
42  private:
43      Calculator m_calc;
44  };
45
46  } // namespace CalcTests
```

12.2.4 引用被测项目与配置要点

1. 把被测代码放进一个**静态库**项目（示例中的 `CalcLib`），头文件放在 `include` 目录。
2. 在测试项目上右键 **添加 > 引用**，勾选 `CalcLib`；再在 **项目属性 > C/C++ > 常规 > 附加包含目录**中加入被测项目的头文件目录。
3. 测试项目与被测项目应使用**一致的运行库设置**（例如都选 `/MDd`），否则会出现 CRT 相关的链接冲突。
4. 按 `F6` 生成，在测试资源管理器中点“全部运行”。

注意 本机单元测试框架要求测试代码与框架库的编译选项匹配；把 Debug 配置的测试 DLL 与 Release 配置的被测静态库混用时，链接错误通常比运行错误更容易发现，但 ABI 不一致导致的诡异崩溃很难查，务必让两者的配置保持一致。

12.3 接入第三方框架

12.3.1 Google Test：vcpkg + CMake

这是目前最主流的组合，跨平台且 CI 友好。

```
# 安装 Google Test（x64-windows 三元组）并让 VS 自动识别 vcpkg
vcpkg install gtest:x64-windows
vcpkg integrate install
```

```
cmake_minimum_required(VERSION 3.20)
project(Calc CXX)

set(CMAKE_CXX_STANDARD 20)
enable_testing()

find_package(GTest CONFIG REQUIRED)

add_library(calc STATIC src/Calculator.cpp)
target_include_directories(calc PUBLIC include)

add_executable(calc_tests tests/CalculatorTests.cpp)
target_link_libraries(calc_tests PRIVATE calc GTest::gtest_main)

# 让 CTest 与测试资源管理器都能发现每个 gtest 用例
include(GoogleTest)
gtest_discover_tests(calc_tests)
```

```
// tests/CalculatorTests.cpp
#include <gtest/gtest.h>
#include "Calculator.h"

TEST(CalculatorTest, AddReturnsSum) {
    Calculator c;
    EXPECT_EQ(5, c.Add(2, 3));            // 失败也继续执行本用例
    ASSERT_EQ(-5, c.Add(-2, -3));          // 失败立即返回本用例
}

TEST(CalculatorTest, DivideByZeroThrows) {
    Calculator c;
    EXPECT_THROW(c.Divide(1, 0), std::invalid_argument);
}

TEST(CalculatorTest, AverageHandlesEmptyInput) {
    Calculator c;
    EXPECT_THROW(c.Average({}), std::invalid_argument);
    EXPECT_DOUBLE_EQ(2.0, c.Average({ 1, 2, 3 }));
}
```

EXPECT_* 与 ASSERT_* 的区别值得记住：前者记录失败后继续执行该用例，后者失败即返

回——需要先确认前置条件（例如指针非空）时用 `ASSERT_*`，纯结果校验用 `EXPECT_*` 能得到更多信息。

12.3.2 Google Test：在 VS / MSBuild 项目中使用

不想引入 CMake 时，有两条路：

- **NuGet 包**：在测试项目上右键 **管理 NuGet 程序包**，搜索 Google Test 包并安装，它会配置好包含目录与库；再手动把测试用例写进项目。
- **测试适配器**：Visual Studio 提供 Google Test 适配器（在安装程序的单个组件中可找到相关条目，名称随版本略有差异）。装了适配器后，测试资源管理器能够发现 CMake 项目或 MSBuild 项目里的 gtest 用例，并支持按用例筛选与单独调试。

实战场景 一个既有 MSBuild 解决方案不想改构建系统，就在专门的 `MyApp.Tests` 项目里用 NuGet 引入 Google Test，把测试项目加进解决方案并让它在 CI 中被 `vstest.console.exe` 调用——既保留了 `gtest` 的断言生态，也不用动主工程的工程文件。

12.3.3 Boost.Test 与 Catch2

Boost.Test 适合已经在用 Boost 的项目，支持单头文件模式（无需额外链接）：

```
#define BOOST_TEST_MODULE CalcTests
#include <boost/test/included/unit_test.hpp>
#include "Calculator.h"

BOOST_AUTO_TEST_CASE(add_returns_sum) {
    Calculator c;
    BOOST_CHECK_EQUAL(c.Add(2, 3), 5);
    BOOST_REQUIRE_THROW(c.Divide(1, 0), std::invalid_argument);
}
```

Catch2 以表达式断言的可读性著称（`REQUIRE(c.Average({ 1, 2, 3 }) == 2.0)`），可用 `SECTION` 组织共享前置条件的多个分支），v3 起不再是纯单头文件，需要链接库，适合轻量项目与看重断言可读性的团队。

12.3.4 框架对比与选型

框架	引入方式	断言风格	VS 集成	适用场景
Microsoft 原生	内置模板，零第三方依赖	`Assert::AreEqual` 等	内置适配器	纯 Windows/MSVC 项目、要求零依赖

框架	引入方式	断言风格	VS 集成	适用场景
Google Test	vcpkg + CMake，或 NuGet	`EXPECT_*` / `ASSERT_*`	官方适配器	跨平台、生态最大、CI 友好
Boost.Test	Boost 发行包或单头文件模式	`BOOST_CHECK_*`	通过适配器或 CTest	已经在用 Boost 的项目
Catch2	单头文件（v2）或链接库（v3）	`REQUIRE` / `CHECK`	通过适配器或 CTest	轻量项目、看重断言可读性

提示 框架本身不决定成败，**用例的边界覆盖**才决定成败。选型时优先考虑“团队能否在 CI 上一条命令跑完”，其次才是断言写起来好不好看。

12.4 测试适配器、筛选与并行

12.4.1 适配器与发现测试

测试适配器是 IDE 与测试框架之间的桥：它负责枚举测试、执行测试并把结果映射回测试资源管理器的树形结构。表现为“测试资源管理器里看不到任何用例”时，排查顺序是：项目是否生成成功 -> 适配器是否安装 -> 测试框架是否被正确引用（示例中 gtest 是否真的链接进了可执行文件）。

12.4.2 分组与筛选

测试资源管理器顶部的“分组依据”下拉支持按**项目**、**命名空间**、**类**、**特征（Trait）**、**结果**分组。搜索框支持类似下面的过滤语法：

输入	含义
`FullyQualifiedName~Calculator`	名称包含 Calculator 的用例
`FullyQualifiedName!~Slow`	排除名称含 Slow 的用例
`Outcome:Failed`	只显示失败的用例
`Trait:Category=Unit`	按特征筛选（特征由框架提供）

12.4.3 测试播放列表

播放列表（`.playlist`，本质是记录用例 ID 的 XML 文件）用来把“每次提交前必须跑的冒烟集”和“每天夜里跑的完整集”分开：

1. 在测试资源管理器中选中若干用例。
2. 右键 **添加到播放列表 > 新建播放列表**，保存为 `smoke.playlist` 并加入版本库。

3. 之后任何人只需选择该播放列表并点“运行”，就能得到一致的测试子集。

> **注意** 播放列表记录的是用例标识，重命名测试类或方法后这些条目会失效。把播放列表纳入版本管理后，重命名测试时记得同步刷新它。

12.4.4 并行执行

并行化的收益取决于测试本身的性质：

- **纯逻辑测试**（无共享文件、无固定端口、无全局单例）可以放心并行，优先按测试项目或测试可执行文件拆分，交给构建系统（如 `msbuild /m`）或多个 `vstest.console.exe` 进程并行调度；反之，依赖同一临时文件、数据库或端口的测试并行后会互相干扰，应先为每个用例准备独立的工作目录，或者干脆串行执行。
- 命令行运行的并行度支持与所用适配器有关，可在 CI 上先用保守配置跑通，再逐步提高并行度并观察失败率。

12.5 代码覆盖率

12.5.1 在 IDE 与命令行中收集

在测试资源管理器中运行测试后，通过 **测试 > 分析代码覆盖率**可以看到被测试代码的行覆盖率：绿色表示已执行，红色表示未执行，橙色表示部分分支未覆盖。命令行方式适合放进 CI：

```
:: 1) 生成测试项目 (Release x64)
msbuild MyApp.sln /t:Rebuild /p:Configuration=Release /p:Platform=x64

:: 2) 运行测试并收集覆盖率，同时输出 TRX 报告
vstest.console.exe x64\Release\CalcTests.dll ^
    /Platform:x64 ^
    --collect:"Code Coverage" ^
    /Logger:trx /ResultsDirectory:TestResults
```

只想跑某一类用例时，用 `/TestCaseFilter:"FullyQualifiedName~Calculator"` 过滤即可。

> **注意** 对 C++ 本机测试，覆盖率插桩需要测试二进制导出覆盖率符号，通常在链接器选项中加上 `/EXPORT:__coverage`；缺少它时收集到的覆盖率可能为空。不同工具链与项目类型的具体要求请以当前版本的文档为准。

12.5.2 版本能力差异

能力	说明
运行与调试单元测试、测试资源管理器、播放列表	各版本均提供
Microsoft 原生单元测试框架与本机单元测试项目模板	各版本均提供
Google Test / CTest 等适配器	各版本均可安装使用
代码覆盖率（IDE 内可视化）	**部分功能需要更高版本**（Community 版通常不包含，需 Professional 或 Enterprise）
测试影响分析等高级分析能力	**部分功能需要更高版本**（通常属于 Enterprise 能力）
命令行覆盖率收集（`vstest.console.exe --collect`）	可用性同样受版本与许可限制

提示 拿不准时不要猜，直接查当前版本的版本对比说明；同一台机器上不同版本的能力差异主要体现在覆盖率与高级分析上，而“写测试、跑测试、看结果”这条主链路在任何版本上都是通的。

12.6 TDD 在 C++ 中的落地

TDD 的循环只有三步，但每一步都有明确的产出：

```
写一个失败的测试  ->  用最少的代码让它通过  ->  在测试保护下重构
     ^                                          |
     +------------------------------------------+
(1) 红：只写接口签名与断言期望，编译不过也算红
(2) 绿：不求优雅只求正确，不写测试没要求的功能
(3) 重构：提取函数、消除重复、修正命名，每轮循环控制在几分钟内
```

在 C++ 中落地的几个现实约束：

1. **编译速度就是循环速度**。把被测逻辑拆成小而独立的静态库，测试项目只链接它；尽量用前置声明与 pimpl 减少重编译范围。一次循环超过五分钟，TDD 就很难坚持。
2. **先设计接口，再写实现**。测试写不下去通常意味着接口设计有问题（依赖太深、职责太杂），这本身就是 TDD 的收益。
3. **外部依赖要能被替换**。文件、网络、时间、随机数都应通过接口注入；测试中使用伪实现或固定种子，避免结果不可复现。
4. **不要为了覆盖率写测试**。断言为空的“调用一遍”用例只增加维护成本，不增加信心。

实战场景 给一个解析器写新功能时，先写下 `TEST_METHOD(Parse_EmptyInput_ReturnsNull)` 与 `TEST_METHOD(Parse_ValidInput_FillsFields)` 两条用例，此时编译不过；补上最小实现让它们通过；再补边界用例（超长输入、非法字符），最后在绿色的状态下重命名与拆分函数。

12.7 持续集成

CI 上的目标是“一条命令构建、一条命令跑测试、失败时能拿到可读报告”。

```
:: ci-build-and-test.bat —— 构建 + 测试 + TRX 报告
setlocal
set CONFIG=Release
set PLATFORM=x64

:: 1) 构建（/m 允许并行编译）
msbuild MyApp.sln /t:Rebuild /m ^
    /p:Configuration=%CONFIG% /p:Platform=%PLATFORM% || exit /b 1

:: 2) 运行测试，失败即返回非零退出码，让 CI 判定失败
vstest.console.exe %PLATFORM%\%CONFIG%\CalcTests.dll ^
    /Platform:%PLATFORM% ^
    /Logger:trx /ResultsDirectory:TestResults || exit /b 1

endlocal
```

几个实践要点：

- **退出码就是结论**。`vstest.console.exe` 在有用例失败时返回非零，CI 脚本必须把它透传出去（上面的 `|| exit /b 1`），否则流水线会“绿着失败”。
- **TRX 报告一定要上传为构建产物**。失败时能直接看到用例名、消息与堆栈，比翻控制台日志快得多。
- **区分冒烟集与完整集**。提交触发的流水线跑播放列表里的冒烟集并启用并行；夜间流水线跑全部用例，再加上 `/analyze`、clang-tidy 与覆盖率收集。同时用 `/W4 /WX` 把编译警告当失败处理，让代码质量问题在合并前就被拦住。

12.8 小结

- **测试 > 测试资源管理器**是统一的测试入口，支持发现、运行、调试、分组、筛选与播放列表。
- 本机单元测试项目基于 `CppUnitTest.h`，用 `TEST_CLASS` / `TEST_METHOD` 组织用例，用 `Assert::AreEqual`、`Assert::ExpectException` 等断言；被测代码建议放进静态库并被测试项目引用。

- Google Test 适合跨平台与 CI（vcpkg + CMake 是首选组合），Boost.Test 适合已用 Boost 的项目，Catch2 适合轻量项目；选型的关键是 CI 上能否一条命令跑完。
- 出现“看不到用例”时按“生成是否成功 -> 适配器是否安装 -> 框架是否被链接”的顺序排查。
- 播放列表用于分离冒烟集与完整集；依赖共享资源的测试不要盲目并行。
- 代码覆盖率可在 IDE 或命令行收集，C++ 项目可能需要 `/EXPORT:__coverage`；覆盖率与高级测试分析**部分功能需要更高版本**。
- TDD 的循环是红 -> 绿 -> 重构，在 C++ 中它的速度上限由编译速度决定。
- CI 中让 `vstest.console.exe` 的退出码决定流水线成败，并把 TRX 报告留作构建产物。

12.9 练习题

1. 创建一个本机单元测试项目，为 `Calculator` 补上 `Subtract` 与 `Multiply`，写出覆盖正数、负数与零的用例，并让它们全部通过。
2. 用 vcpkg 安装 Google Test，通过 CMake 把上一题的测试改写成 gtest 版本，并让测试资源管理器同时发现两套用例。
3. 为一段会修改文件的自有代码写测试，要求每个用例使用独立的临时目录，并说明为什么这样的测试才能安全并行。
4. 创建一个播放列表，只包含名称中含 `Fast` 的用例，并说明在 CI 上如何把冒烟集与完整集分成两条流水线。
5. 用 `/EXPORT:__coverage` 收集一次代码覆盖率，找出项目中覆盖率最低的一个源文件，并解释它是“缺少测试”还是“不可测设计”。
6. 按 TDD 流程实现一个小功能（例如把字符串按分隔符切分并去掉空项），记录每一步的用时，并分析哪一步最慢、为什么。

12.10 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **上一章：CMake 与 vcpkg 集成**
- **下一章：发布与部署**
- **C++ 教程：Visual Studio 入门**
- **CMake：CMake 使用指南**

第十三章　发布与部署

代码能在自己的机器上按 F5 跑起来，和它能被装到另一台机器上跑起来，是两件完全不同的事。发布阶段的事故几乎都不发生在源码里，而是藏在配置切换、运行库选择、依赖收集和安装包这几个环节。这一章按“从 Release 配置到交付安装包”的顺序把这些环节逐个拆开，重点放在每一步为什么会导致崩溃、以及怎么提前验证。

13.1　发布前的配置切换

13.1.1　Debug 与 Release 的本质差异

切换配置不是换个名字，而是同时改掉了一整组编译器和链接器选项。默认两个配置的差异大致如下：

维度	Debug 配置	Release 配置
优化级别	`/Od` 关闭优化，并叠加 `/RTC1` 基本运行时检查	`/O2` 最大化速度
预处理宏	定义 `_DEBUG`，不定义 `NDEBUG`	定义 `NDEBUG`，不定义 `_DEBUG`
`assert`	生效	被 `NDEBUG` 整体去掉
运行库	`/MDd` 或 `/MTd`	`/MD` 或 `/MT`
迭代器调试级别	`_ITERATOR_DEBUG_LEVEL=2`	`_ITERATOR_DEBUG_LEVEL=0`
调试信息	`/Zi`，链接器默认生成 PDB	默认不生成 PDB
增量链接	打开（`/INCREMENTAL`）	关闭，并启用 `/OPT:REF`、`/OPT:ICF`
全程序优化	关闭	项目模板常在 Release 下打开（`/GL` + `/LTCG`）
未初始化内存	调试堆会填充特征字节，容易暴露	内容随机，可能出现“在我机器上是好的”
速度与体积	慢、大	快、小

注意 `assert` 在 Release 下是**完全不执行**的，所以不要把带副作用的表达式写进 `assert`（例如 `assert(p = CreateBuffer())`）。Debug 下它正常工作，Release 下 `p` 会永远是空指针，而且不会有任何提示。

提示 想保留 Release 的性能又想能调试，可以克隆一个中间配置：`/O2` + `/Zi`，并在 **项目 > 属性 > 链接器 > 调试 > 生成调试信息**选择 **是 (/DEBUG)**。如果问题只在优化后出现，就单独建一个把 **项目 > 属性 > C/C++ > 优化 > 优化**改成 **已禁用 (/Od)** 的配置，用二分法缩小范围。

13.1.2 常用优化开关的含义与风险

开关	含义	需要注意的风险
`/O2`	最大化速度，等于打开一揽子子优化	行号会错位、局部变量可能被优化掉，单步调试体验骤降
`/Od`	关闭所有优化	性能损失明显，只用于排查阶段
`/O1`	最小体积，与 `/O2` 互斥	体积优先时速度可能反而下降
`/Ob2`	内联展开（`/Ob0` 禁止、`/Ob1` 仅 `inline`、`/Ob2` 全部）	断点可能进不去被内联的函数
`/Oi`	生成内建函数（intrinsic）	部分 CRT 调用不再可断点
`/Oy`	省略帧指针	调用栈回溯可能不完整，是“看不到局部变量”的主因
`/GF`	字符串字面量池化	相同字面量共享同一地址，用地址比较字符串会出错
`/Gy`	函数级链接	配合 `/OPT:REF` 剔除未引用函数，是减体积的关键
`/GL`	全程序优化，编译期只产出中间表示	链接明显变慢、内存占用高，必须由 `/LTCG` 完成最终代码生成
`/LTCG`	链接时代码生成	与增量链接冲突，链接器会提示忽略 `/INCREMENTAL`（`LNK4075`）
`/arch:AVX2`	允许编译器生成 AVX2（256 位）指令	在不支持 AVX2 的 CPU 上会触发非法指令异常（`0xC000001D`）
`/arch:AVX512`	允许生成 AVX-512 指令	支持该指令集的消费级 CPU 更少，风险比 `/arch:AVX2` 更大
`/fp:fast`	浮点快速模式，允许重排与近似	结果与调试版不一致，可能出现 `a + b != b + a`，故障无法复现
`/GS`	缓冲区安全检查，默认开启	关掉会放大栈溢出风险，不要关
`/guard:cf`	控制流保护	与某些加壳、注入类工具冲突
`/W4`、`/permissive-`	打开第 4 级警告、关闭宽松模式以贴近标准	会让老代码冒出一批新警告，建议在发布前逐个处理

关于 `/arch` 的正确用法：混用不同 `/arch` 编译的静态库不会链接失败，但最终程序会要求运行机器支持其中最高的那套指令集。如果必须兼顾老机器，做法是用 `/arch:SSE2` 或 `/arch:AVX2` 出通用版本，再在运行时用 `__cpuid` / `__cpuidex` 读取 CPUID 特性位做派发。

实战场景 开发机是最新一代 CPU，用 `/arch:AVX2` 编译后自测一切正常。程序发到一台老笔记本上双击就退出，事件查看器里是异常代码 `0xC000001D`（STATUS_ILLEGAL_INSTRUCTION）。这就是把编译期指令集当成运行期保证的后果。

13.1.3 切换到 Release 后先做的三件事

1. 打开 **生成 > 配置管理器**，确认“活动解决方案配置”是 Release，并且列表里**每一个项目**都是 Release——依赖库项目漏切一个，整个发布就是 Debug 版 CRT 混进 Release 二进制。
2. 打开 **项目 > 属性 > C/C++ > 命令行**（右上角可以切换配置与平台），肉眼扫一遍真实命令行，确认没有从 Debug 继承下来的 `/Od`、`/RTC1`、`/MDd`。
3. 执行 **生成 > 重新生成解决方案**，而不是“生成”。增量生成会把 Debug 的 `.obj` 留在中间目录里，改配置后必须全量重来。

13.2 运行库选择：/MD 还是 /MT

13.2.1 四个取值

位置在 **项目 > 属性 > C/C++ > 代码生成 > 运行库**：

选项	开关	说明
多线程调试 DLL	`/MDd`	Debug 版动态 CRT，带调试堆
多线程调试	`/MTd`	Debug 版静态 CRT
多线程 DLL	`/MD`	Release 版动态 CRT，依赖可再发行程序包
多线程	`/MT`	Release 版静态 CRT，不依赖外部 DLL

选 `/MD` 时，程序在运行期需要下列文件：

文件	作用	备注
`MSVCP140.dll`	C++ 标准库实现	由可再发行程序包安装
`VCRUNTIME140.dll`	编译器支持例程（异常、RTTI 等）	同上
`VCRUNTIME140_1.dll`	较新工具集补充的异常处理支持	只装过旧的运行库就会缺这个文件
`ucrtbase.dll`、`api-ms-win-crt-*.dll`	通用 C 运行时（UCRT）	Windows 10 起属于系统组件，一般无需分发
`concrt140.dll`	并发运行时	只在用到并发容器等设施时才会加载

13.2.2 取舍表

维度	/MD（动态 CRT）	/MT（静态 CRT）
产物体积	exe 小，逻辑集中在系统与运行库目录	exe 明显变大，通常增加数百 KB
部署成本	必须确保目标机装有对应位数的可再发行程序包	免依赖，拷贝即用
跨模块传 C++ 对象	全进程共用一份 CRT，跨 DLL 传递 std::string 等相对安全	每个模块各有一份 CRT 状态，跨边界释放内存极易堆损坏
安全更新	随运行库更新一起被修补	必须自己重新编译并重新发布
第三方库匹配	绝大多数官方预编译包默认是 /MD	需要第三方库也提供 /MT 版本
典型场景	常规桌面程序、插件式架构、多模块工程	绿色单文件工具、无管理员权限的受限环境

13.2.3 混用 /MT 与 /MD 的后果

同一个进程里同时出现静态 CRT 和动态 CRT，等于有两套完全独立的运行时状态：两个堆、两份 errno、两份 std::string 的实现、两份异常处理机制。在 A 模块里 new、在 B 模块里 delete（或 malloc / free 交叉），就会踩坏堆。

- 静态库混用时，链接器通常能用 LNK2038 拦住你，提示“检测到 RuntimeLibrary 的不匹配项：值 MT_StaticRelease 不匹配值 MD_DynamicRelease”。
- _ITERATOR_DEBUG_LEVEL 不一致（Debug 2 与 Release 0）也会报 LNK2038。
- **但如果配置不一致的是一份 DLL，链接期根本发现不了**，只能在运行期炸成堆损坏（0xC0000374）或随机崩溃。

注意 第三方库的 CRT 必须与本项目一致。拿到一份 .lib 或 .dll 时，第一件事是确认它是 /MD 还是 /MT、是 Debug 还是 Release、是 x86 还是 x64。四个维度都对上，才能避免两套运行时的灾难。

统一的工程约定：

1. 一个解决方案内的所有项目统一使用同一种运行库，写进项目模板或 .props 属性表。
2. 先看第三方库给的是什么：Qt、OpenCV、Boost 的官方二进制包基本都是 /MD，那就全工程 /MD。
3. 如果第三方只提供 /MT 版本，要么全工程改 /MT，要么自己用 /MD 重新编译那个库。
4. 用 /MT 时不要跨 DLL 边界传递 STL 容器与字符串，改用 C 风格接口或纯虚接口。
5. 需要同时支持多模块时，优先 /MD，把“体积小、可被系统更新”的优势留给运行库。

13.3 静态链接第三方库与动态链接 DLL

13.3.1 两种方式的部署代价

对比项	静态库（.lib，静态链接）	动态库（.lib 导入库 + .dll）
需要拿到	.lib 与头文件	导入库 .lib、头文件、以及 .dll
部署时要带什么	什么都不用带，代码已经进 exe	必须把 .dll 一起发布
体积	exe 变大，多个程序各自持有一份	多个程序共享一份 .dll
更新	改库就要重新编译发布主程序	替换 .dll 即可（ABI 不变的前提下）
排查难度	符号直接进 exe，问题相对好定位	多一层加载过程，缺文件、位数不符都会在启动时失败
许可	GPL 类库静态链接可能带来传染性义务	通常只要求随程序分发 .dll

13.3.2 用 dumpbin 查依赖

dumpbin 不在普通命令行里，需要用 **工具 > 命令行 > 开发者命令提示符**打开，或从开始菜单启动 Developer Command Prompt。

```
:: 列出直接依赖的 DLL（不递归）
dumpbin /dependents MyApp.exe

:: 查看 PE 头，确认位数：x64 显示 8664 machine (x64)，x86 显示 14C machine (x86)
dumpbin /headers MyApp.exe | findstr machine

:: 查看导入表，能看到每个 DLL 被用到的具体函数
dumpbin /imports MyApp.exe

:: 检查某个第三方 DLL 的位数
dumpbin /headers helper.dll | findstr machine
```

提示 /dependents 只列**直接**依赖，不会展开到第二层。要递归看整棵依赖树，用开源的 Dependencies 工具（Dependency Walker 的现代替代品），或者干脆在每个目录上手工跑一遍 dumpbin。

13.3.3 定位缺失 DLL 的排查路径

1. 先记下报错对话框里的 DLL 文件名，这是最直接的线索。
2. dumpbin /dependents 列出直接依赖，逐个确认它们在目标机器上的位置。

3. 用 Dependencies 递归展开，缺失的那一级会被标出来；它的 API Set 解析比老工具准确得多。
4. 用 Process Monitor 过滤 `Process Name is MyApp.exe` 与 `Result is NAME NOT FOUND`，可以精确看到加载器依次找过哪些目录。
5. 在 Visual Studio 里用 **调试 > 窗口 > 模块**查看实际加载到的模块全路径；**工具 > 选项 > 调试 > 输出窗口**里打开模块加载消息，也能在输出窗口看到加载过程。
6. 注意最常见的陷阱：应用目录里同时存在 32 位与 64 位的同名 DLL，加载器挑中了错的那个，于是报 `0xc000007b`。

13.4 收集运行时依赖

13.4.1 Visual C++ 可再发行程序包

可再发行程序包（Redistributable）就是把 CRT 装进系统目录的官方安装器，文件名按位数区分：

- `vc_redist.x64.exe`——64 位程序用。
- `vc_redist.x86.exe`——32 位程序用（即使跑在 64 位 Windows 上也是它）。
- `vc_redist.arm64.exe`——ARM64 程序用。

要点：

- **位数必须与你的程序一致**。64 位程序装 x86 版运行库没有任何作用，报错依旧。
- 安装包本体位于 VS 安装目录下的 `VC\Redist\MSVC\<工具集版本>`，同级的 `x64\Microsoft.VC<编号>.CRT` 目录里放的是供 app-local 部署的 CRT DLL 文件。
- 自 Visual Studio 2015 起，各代运行库保持二进制兼容并可以共存，装一个新的通常就能满足老程序；但老程序若需要 `VCRUNTIME140_1.dll`，就必须装较新的运行库。
- 通用 CRT（UCRT）从 Windows 10 起由系统提供；更早的系统需要额外安装系统更新（例如 KB2999226 一类），这也是“在 Win10 上好好的，到老系统就缺 `ucrtbase.dll`”的原因。

13.4.2 随安装包分发与静默安装

```
:: 静默安装运行库：/install 安装、/quiet 无界面、/norestart 不自动重启
vc_redist.x64.exe /install /quiet /norestart

:: 取返回值判断结果
echo %ERRORLEVEL%
```

常见返回值：

返回值	含义	处理方式
0	安装成功	继续安装主程序
1638	已存在其他版本，未安装本次版本	视为成功，继续
3010	安装成功但需要重启	提示用户重启后生效
1603	安装失败	检查权限、是否有更高版本阻止降级、日志中的具体错误

其他可用参数：`/passive` 显示进度条但不需要交互、`/repair` 修复、`/uninstall` 卸载、`/log < 文件 >` 输出日志。在安装包项目里，把这个 exe 作为普通文件一起打进去，再在安装流程末尾用批处理或自定义操作静默执行，是最省事的做法。

13.4.3 app-local 部署的边界

“把 CRT 的 DLL 直接拷到 exe 同目录”叫 app-local 部署，它能免掉安装步骤，但有几个前提：

- 只有官方《可再发行代码》清单里列出的文件才允许随程序分发，不同 Visual Studio 版本与不同版本（Community、Professional、Enterprise）的授权范围并不相同，务必以随 VS 安装的 `Redist.txt` 和 Microsoft Learn 上的“可再发行代码”页面为准。
- app-local 的 CRT 不会随 Windows Update 被修补，安全更新要自己重新发布一次。
- 系统目录里若已有另一份 CRT 被其他进程加载，问题会更难排查。

结论：**对外正式发布优先用 `vc_redist` 安装器；绿色小工具或内网工具可以用 app-local**。

13.5 生成安装包

13.5.1 三种打包方式对比

方案	产物	优点	局限
Visual Studio Installer Projects 扩展	`.msi` 与 `setup.exe`	直接集成在 VS 里，图形化添加快捷方式、注册表、文件类型关联	需要单独安装扩展；自定义能力有限；多语言与复杂升级场景支持一般
MSIX 打包项目	`.msix`	安装卸载干净、支持增量更新与回滚、可上 Microsoft Store	安装到受保护目录；文件与注册表被虚拟化；不能装服务与驱动；必须代码签名

WiX Toolset	`.msi`（由 XML 描述）	完全可脚本化，适合放进 CI，能表达复杂安装逻辑	学习曲线陡，要手写 XML

13.5.2 Installer Projects 最小制作流程

1. 打开 **扩展 > 管理扩展**，搜索并安装 `Microsoft Visual Studio Installer Projects`，安装完成后重启 VS。
2. **文件 > 新建 > 项目**，搜索"Setup Project"，选择安装项目模板。
3. 视图切换到 **文件系统 (File System)**，右键 **应用程序文件夹**节点，选择 **添加 > 项目输出**，选中主项目；第三方 DLL 用 **添加 > 文件**补进去。
4. 选中安装项目，在属性窗口里把 **TargetPlatform** 设为 `x64`，必须与 exe 的位数一致，否则会装进 `Program Files (x86)`。
5. 右键 **应用程序文件夹**里的主输出，选择 **创建主输出的快捷方式**，把生成的快捷方式拖到 **用户桌面**和 **用户的"程序"菜单**。
6. 在安装项目的属性里设置 **Manufacturer**、**Version**、**ProductName**；升级安装时要保持 **UpgradeCode** 不变，提升 **Version** 并把 **RemovePreviousVersions** 设为 `True`，否则会出现"已安装另一个版本"而无法覆盖。
7. 执行 **生成 > 生成解决方案**，在安装项目的 Release 输出目录里得到 `.msi` 与 `setup.exe`。
8. 在**一台干净的虚拟机**上完整走一遍安装、运行、卸载。这是唯一真正有效的验证方式。

13.5.3 WiX Toolset 简介

WiX 用 XML 描述安装内容：产品信息、目录树、组件与 GUID、快捷方式、注册表项、服务、自定义操作，然后由命令行工具编译成 MSI。它不做图形化设计器，换来的是可版本管理、可在 CI 里一条命令出包。

适合用 WiX 的信号：安装逻辑复杂（多组件可选安装、按条件写注册表）、需要无人值守构建、需要被内部部署系统调用。如果只是"把 exe 和几个 DLL 装进 Program Files 并建快捷方式"，Installer Projects 更快。

13.6 其他发布形式

13.6.1 绿色免安装版

做法很简单：用 `/MT` 加静态第三方库，或者把所需 DLL 与 exe 放进同一层目录，然后压缩成 zip。之所以"放同目录"最稳，是因为 Windows 加载器的搜索顺序是固定的（默认开启安全搜索模式）：

```
DLL 搜索顺序（安全搜索模式，默认开启）
  1. 加载它的那个 exe/dll 所在目录  <- 唯一完全由你掌控的位置
  2. System32（32 位进程在 64 位系统上是 SysWOW64）
```

```
3. Windows 目录
4. 当前工作目录  <- 双击启动和命令行启动可能不同，别依赖
5. PATH 环境变量中的各目录
```

进一步收紧可以用 SetDefaultDllDirectories 配合 AddDllDirectory / LoadLibraryEx 的搜索标志，只信任自己的目录，避免被同名 DLL 劫持。

13.6.2 单文件与自包含

“单文件发布” 是.NET 的概念（PublishSingleFile、SelfContained），C++ 没有对应的官方开关。C++ 里等价的做法是：静态 CRT + 静态第三方库 + 不依赖外部数据文件，产物天然就是一个 exe。

需要清楚的两点：/MT 只能消掉 CRT 依赖，消不掉对系统 DLL 的依赖；图标、字体、数据库之类的资源仍需外置，或者塞进资源段（在 .rc 里用 RCDATA 声明，运行时用 FindResource / LoadResource 读出来）。

13.6.3 分发渠道一句话说明

- **Steam**：把 exe 与 steam_api64.dll 一起提交到 depot，并可用 Steamworks 的安装脚本（installscript.vdf）声明在首次运行前执行 VC++ 运行库安装程序。
- **内网服务器**：直接放 zip 加校验和，用 certutil -hashfile MyApp.exe SHA256 生成 SHA-256 供同事核对；也可以接内部的包管理源。
- **企业批量部署**：优先出 MSI，交给域策略或软件分发系统；绿色版作为兜底。

13.7 版本信息与资源

13.7.1 添加 .rc 资源文件

1. 打开 **项目** > **添加** > **资源**，选择 **Version**，新建后 VS 会生成 < 项目名 >.rc 与 resource.h。
2. 再次打开 **项目** > **添加** > **资源**，选择 **Icon**，导入 .ico（建议一个文件里包含 16、32、48、256 多档尺寸）。
3. .rc 由资源编译器编译成 .res，再由链接器并入 exe，不需要你手动调用任何工具。

13.7.2 版本号字段

以下是一个最小可用的版本资源写法：

```
#include "resource.h"

VS_VERSION_INFO VERSIONINFO
 FILEVERSION    1,2,3,0
 PRODUCTVERSION 1,2,3,0
 FILEFLAGSMASK  0x3fL
```

```
 FILEFLAGS       0x0L
 FILEOS          0x40004L
 FILETYPE        0x1L
 FILESUBTYPE     0x0L
BEGIN
    BLOCK "StringFileInfo"
    BEGIN
        BLOCK "080404b0"
        BEGIN
            VALUE "CompanyName",      "示例公司\0"
            VALUE "FileDescription",  "示例工具\0"
            VALUE "FileVersion",      "1.2.3.0\0"
            VALUE "InternalName",     "MyApp\0"
            VALUE "LegalCopyright",   "Copyright (C) 2026\0"
            VALUE "OriginalFilename", "MyApp.exe\0"
            VALUE "ProductName",      "MyApp\0"
            VALUE "ProductVersion",   "1.2.3.0\0"
        END
    END
    BLOCK "VarFileInfo"
    BEGIN
        VALUE "Translation", 0x804, 1200
    END
END
```

字段	显示位置与作用
FILEVERSION / PRODUCTVERSION	二进制形式，四个 16 位整数，用逗号分隔；供程序读取比较
FileVersion / ProductVersion	字符串形式，资源管理器“详细信息”里看到的就是它
CompanyName	公司名，安装包与“详细信息”页都会读
FileDescription	文件说明，任务管理器的进程描述列取这里
LegalCopyright	版权声明
OriginalFilename	原始文件名，改名后仍能追溯
080404b0	语言块：0804 为简体中文，04b0 为 Unicode 代码页 1200

注意 StringFileInfo 的语言代码块必须与 VarFileInfo 里的 Translation 一致（上面是 0x804, 1200），否则部分工具读不到字符串。FILEVERSION 与 FileVersion 也要手工保持一致——它们不会互相推导。

13.7.3 图标与属性页设置

- exe 在资源管理器里显示的图标，取资源中 **ID 最小**的那个图标，所以通常用 `IDI_ICON1`。
- 选中 `.rc` 文件后打开属性页，根节点就是 **资源**：**资源 > 常规 > 区域性**可以设置资源语言（中文简体为 `0x0804`）。不改的话默认是英文，多语言资源会显得混乱。
- 想让版本号跟随构建号自动递增，可以在 **项目 > 属性 > 生成事件 > 生成前事件**里调用脚本改写 `.rc`；也可以在 **资源 > 常规 > 预处理器定义**里注入宏，让 `.rc` 用 `##ifdef` 选择版本字符串。

13.8 发布检查清单

配置与平台

☐ **生成 > 配置管理器**里所有项目都是 Release，平台都是 x64（或与目标一致）。

☐ **项目 > 属性 > C/C++ > 命令行**中不再出现 `/Od`、`/RTC1`、`/MDd`、`/MTd`。

☐ 全解决方案执行过 **重新生成解决方案**，没有沿用 Debug 的中间文件。

☐ **项目 > 属性 > 常规 > 平台工具集**与 **Windows SDK 版本**在开发机与构建机上一致。

依赖

☐ `dumpbin /dependents MyApp.exe` 的输出里**没有** `MSVCP140D.dll`、`VCRUNTIME140D.dll`、`ucrtbased.dll` 之类的调试版运行库。

☐ 所有第三方 DLL 都是 Release 版、位数正确、CRT 设置与本项目一致。

☐ 明确记录：目标机器需要装哪个可再发行程序包（x86 还是 x64）。

☐ 若有数据文件、字体、配置模板，也一并纳入安装包或目录结构。

测试

☐ 在干净虚拟机上完成安装、首次运行、卸载全流程。

☐ 在**没有装过开发工具**的机器上验证一遍，重点看启动阶段是否缺 DLL。

☐ 中文路径、含空格用户名、非管理员账户下各跑一次。

☐ 若是数值计算类程序，比较 Release 与 Debug 的关键输出是否一致（防止 `/fp:fast` 引入偏差）。

版本与资源

☐ exe 的“详细信息”页里能看到正确的版本号、公司名、版权。

☐ 图标在资源管理器、任务栏、任务管理器中显示正常。

☐ 版本号与安装包的 **Version** 属性一致。

签名与体积

☐ 用 `signtool sign /fd SHA256 /tr < 时间戳服务器 > /td SHA256 MyApp.exe` 完成签名，并用 `signtool verify /pa /v MyApp.exe` 验证。

☐ 签名带时间戳，这样证书到期后旧版本的签名依然有效。

☐ 体积已用 /Gy + /OPT:REF + /OPT:ICF 优化，且链接进来的库确实都被用到。

☐ 压缩包与安装包文件名里带版本号，便于回滚。

13.9 发布后常见故障

13.9.1 缺少 VCRUNTIME140.dll 一类

表现：弹窗“由于找不到 VCRUNTIME140.dll，无法继续执行代码。重新安装程序可能会解决此问题。”，常见变体还有 MSVCP140.dll、VCRUNTIME140_1.dll、ucrtbase.dll。

原因：程序用 /MD 编译，但目标机器没装对应位数的可再发行程序包；或者装的是 x86 版而程序是 x64；或者运行库太旧，缺 VCRUNTIME140_1.dll。

处理：

1. 立即解决：在目标机运行 vc_redist.x64.exe /install /quiet /norestart。
2. 根治：把运行库作为安装包的先决条件自动安装。
3. 另一种根治：改用 /MT 重新发布，代价是体积变大、安全更新要自己跟。

13.9.2 应用程序无法正常启动 0xc000007b

0xC000007B 是 STATUS_INVALID_IMAGE_FORMAT，意思是“这个映像文件的格式不对”。最常见的原因是**位数不匹配**——64 位进程加载了 32 位 DLL，或反过来；其次是 DLL 文件被截断或损坏、同名文件被不同位数的版本互相覆盖。

```
x64 的 MyApp.exe --> 加载 helper.dll
                     |
                     +-- helper.dll 是 32 位     --> 0xC000007B (映像格式无效)
                     +-- helper.dll 不存在       --> 0xC0000135 (找不到 DLL)
                     +-- helper.dll 初始化失败   --> 0xC0000142 (DLL 初始化失败)
```

排查方法：

```
:: 先确认主程序位数，再逐个确认可疑 DLL 的位数
dumpbin /headers MyApp.exe  | findstr machine
dumpbin /headers helper.dll | findstr machine
```

两者的 machine 值必须一致：x64 显示 8664 machine (x64)，x86 显示 14C machine (x86)。顺带记住另两个高频代码：0xC0000135 是找不到 DLL，0xC0000142 是 DLL 初始化失败（常见于被杀软拦截或它的依赖没满足）。

13.9.3 DLL 地狱的标准排查步骤

1. 抄下完整报错文案，记录故障机器的 Windows 版本与位数。
2. dumpbin /dependents MyApp.exe 列出直接依赖，逐个确认在目标机器上的存在性与位数。

3. 用 Dependencies 递归展开依赖树，找出断链的那一级。
4. 用 Process Monitor 过滤进程名与 `NAME NOT FOUND`，看加载器实际搜索过哪些路径。
5. 检查 PATH、当前工作目录，以及 System32 里是否存在同名 DLL。
6. 在“能跑起来”的机器上用 **调试 > 窗口 > 模块**导出已加载模块清单，与故障机逐条对比。
7. 确认三件事：位数一致、CRT 配置一致（同为 `/MD` 或同为 `/MT`）、没有 Debug 版 CRT 混入。
8. 修完后在干净虚拟机上复测一遍，不要只在开发机上确认。

13.10 小结

- Debug 与 Release 的差异是一整组开关的差异：优化、宏、运行库、调试信息、增量链接，切换时必须全解决方案一致。
- `/O2`、`/GL` + `/LTCG`、`/arch:AVX2` 能显著提速，但分别带来调试困难、构建变慢、以及在不支持该指令集的 CPU 上崩溃的风险。
- `/MD` 依赖可再发行程序包但体积小、跨模块安全；`/MT` 免依赖但体积大且跨模块传 STL 危险；同一程序内混用会导致堆损坏，DLL 层面的不一致链接期发现不了。
- 静态库无部署成本但更新要重编；动态库必须随程序带上 `.dll`，用 `dumpbin /dependents` 与 Dependencies 工具确认依赖完整。
- 运行库按位数分发，`vc_redist.x64.exe /install /quiet /norestart` 可静默安装，返回值 0、1638、3010、1603 要分别处理。
- 安装包三条路线：Installer Projects 最快、MSIX 最干净但限制多、WiX 最适合 CI。
- 版本信息写进 `.rc`，exe 图标取资源中 ID 最小的图标，属性页里的 **资源 > 常规 > 区域性**决定资源语言。
- 发布后两类高频故障是缺运行库和 `0xc000007b`（位数不匹配），排查顺序都是先确认位数、再确认依赖树。

13.11 练习题

1. 把一个控制台项目从 `/MD` 改成 `/MT`，用 `dumpbin /dependents` 对比改动前后的依赖列表，并解释每一行的来源。
2. 用 `/arch:AVX2` 编译一个程序，分别在支持与不支持 AVX2 的机器（或限制 CPU 特性的虚拟机）上运行，记录现象与事件查看器中的异常代码。
3. 给一个项目添加版本资源，使 exe 的“详细信息”页显示公司名与版本号 `1.0.0.0`，然后通过代码读取并打印 `FILEVERSION` 与 `FileVersion`，说明二者的区别。
4. 制作一个最小 MSI：包含主程序 exe 与一个第三方 DLL，安装后在开始菜单创建快捷方式，并在干净虚拟机上完成安装与卸载测试。

5. 故意把 32 位的 DLL 放进 64 位程序的目录，复现 `0xc000007b`，写出从报错到定位的完整排查过程。
6. 写一个批处理，在安装主程序前静默安装 `vc_redist.x64.exe`，并正确处理返回值 0、1638、3010、1603。

13.12 相关笔记

- **本教程**：**Visual Studio 2026 使用教程**
- **上一章**：**单元测试与代码质量**
- **下一章**：**常见问题与效率技巧**
- **CMake**：**CMake 使用指南**
- **Git 命令**：**常用 Git 命令**

第十四章　常见问题与效率技巧

前面各章把 Visual Studio 的功能按模块讲了一遍，这一章换成问题导向：先把实际开发中最常撞上的报错做成速查表，再集中处理中文乱码、IntelliSense 与实际编译不一致、断点失效这类“配置型疑难”，最后收一整套效率技巧与快捷键总表。本章偏工具书性质，遇到问题直接翻到对应小节即可，不必从头读。

14.1　编译与链接错误速查

14.1.1　错误号前缀对应哪个阶段

```
        源码与项目文件
             |
             v
[1] MSBuild (MSBxxxx)       解析 .vcxproj、工具集与 SDK、执行生成事件
             |
             v
[2] 编译器 cl.exe (Cxxxx)   预处理 -> 语法 -> 语义 -> 代码生成 -> .obj
             |
             v
[3] 链接器 link.exe (LNKxxxx)   合并 .obj 与 .lib -> .exe / .dll
             |
             v
       可执行文件

旁路：IntelliSense (Exxxx) 只在编辑器里做实时分析，不参与真实生成
```

前缀	来源	判断要点
MSB	MSBuild	项目配置、工具集、SDK、生成事件的问题，还没轮到编译器
C	编译器 cl.exe	语法、语义、头文件、预处理、代码生成
LNK	链接器 link.exe	符号找不到或重复、库、入口点、位数冲突
E	IntelliSense	只影响编辑器的波浪线，**不一定**影响编译

14.1.2　编译器高频错误（C 系列）

错误号	提示信息要点	常见原因	解决方案
C2065	未声明的标识符	拼写错误、忘了 ##include、少了 std:: 限定、语言标准不够新	按提示里的标识符核对拼写与头文件；检查 **项目 > 属性 > C/C++ > 语言 > C++ 语言标准**
C3861	找不到标识符	函数没声明、模板依赖名称少了 this-> 或 typename	补头文件或前置声明；模板中显式写 this->member
C2143	语法错误：缺少 ；	上一行少分号、类定义后漏分号、宏展开后语法破损	看**上一行**；用 **C/C++ > 预处理器 > 预处理到文件**观察宏展开结果
C1083	无法打开源文件 / 包含文件	头文件路径没配、大小写不符、文件不在磁盘上、SDK 未安装	在 **C/C++ > 常规 > 附加包含目录**补齐路径，并确认配在“所有配置、所有平台”
C1010	在查找预编译头时遇到意外的文件结尾	新增的 .cpp 忘了首行 ##include "pch.h"	补上包含，或对该文件设置 **C/C++ > 预编译头 > 预编译头 = 不使用预编译头**
C4819	该文件包含不能在当前代码页（936）中表示的字符	源码为 UTF-8 无 BOM，编译器按系统 ANSI 代码页解析	另存为 UTF-8 with BOM，或加 /utf-8（见第 2 节）
C4996	该函数或变量可能不安全，请考虑改用 xxx_s	使用了被标记弃用的 CRT 函数（strcpy、sprintf、localtime 等）	换用安全版本，或在 **C/C++ > 预处理器 > 预处理器定义**加 _CRT_SECURE_NO_WARNINGS
C2440	无法从 X 转换为 Y	指针与整数混用、丢失 const、少了显式转换	补 static_cast 等显式转换，并回头检查类型设计
C2664	无法将参数 N 从 X 转换为 Y	实参类型不匹配、char* 与 wchar_t* 混用、字符集设置不对	检查 **项目 > 属性 > 高级 > 字符集**是否与代码一致
C2039	xxx 不是 yyy 的成员	头文件版本不匹配、命名空间或类名写错	用 F12 转到定义核对真实成员列表
C2084	函数已有主体	同一函数被定义两次，通常是头文件里直接写了定义	改为 inline，或把定义移到 .cpp

C2220	警告被视为错误——没有生成对象文件	项目开了 /WX，某些警告被升级为错误	先修触发它的警告（常见搭档是 C4819）

14.1.3 链接器高频错误（LNK 系列）

错误号	提示信息要点	常见原因	解决方案
LNK2019	无法解析的外部符号 xxx，该符号在函数 yyy 中被引用	只声明未定义、.cpp 未加入项目、.lib 未加进附加依赖项、C/C++ 混用少了 extern "C"、模板实现放在了 .cpp	按符号名反查定义；把 .lib 加到 **链接器 > 输入 > 附加依赖项**；确认库的位数与 CRT 设置一致
LNK2001	无法解析的外部符号（同族提示）	同上，多见于静态成员变量未定义、虚函数表缺失	为静态成员补类外定义
LNK1120	N 个无法解析的外部命令	只是上面两类错误的统计行	忽略它，去修具体的 LNK2019 / LNK2001
LNK2005	xxx 已经在 yyy.obj 中定义	头文件里定义了非 inline 的全局函数或变量、用 ##include 引入了 .cpp、两个库提供了同一符号	加 inline 或 static、把定义移到 .cpp、去掉重复包含；不到万不得已不要用 /FORCE:MULTIPLE
LNK1169	找到一个或多个多重定义的符号	与 LNK2005 成对出现	处理掉 LNK2005 即可
LNK1104	无法打开文件 xxx	库路径不对或库名拼错；**上一次的程序还在运行，exe 被占用**；文件被安全软件锁住	先结束残留进程再重新生成；核对 **链接器 > 输入 > 附加依赖项**与库目录
LNK1561	必须定义入口点	没有 main/wmain/WinMain，或子系统类型与控制台/窗口程序不符	补入口函数，或改 **链接器 > 系统 > 子系统**
LNK1112	模块计算机类型 x64 与目标计算机类型 x86 冲突	混入了位数不对的 .lib 或 .obj	统一平台为 x64 或 x86，检查第三方库位数

错误号	提示信息要点	常见原因	解决方案
LNK2038	检测到 RuntimeLibrary 的不匹配项：值 MT_StaticRelease 不匹配值 MD_DynamicRelease	同一程序内混用 /MT 与 /MD，或 Debug 与 Release 混用	全工程与所有第三方库统一运行库设置（见第 12 章）
LNK4098	defaultlib MSVCRT 与其他库的使用冲突	显式链接了不同版本的 CRT 库	先统一运行库设置；仍冲突时再用 **链接器 > 输入 > 忽略特定默认库**

14.1.4 MSBuild 与 IntelliSense 错误

错误号	提示信息要点	常见原因	解决方案
MSB8020	找不到 vXXX 的生成工具（平台工具集 = vXXX）	项目指定的平台工具集本机没装（打开的是别人机器上的工程，或安装时没勾 MSVC 生成工具）	用 **项目 > 重定解决方案目标**切到已安装的工具集，或在 Visual Studio 安装程序里补装
MSB8036	找不到 Windows SDK 版本 10.0.xxxxx.0	项目指定的 SDK 本机没有	在 **项目 > 属性 > 常规 > Windows SDK 版本**改成本机已装的版本，或补装该 SDK
MSB4019	找不到导入的项目 xxx.props / xxx.targets	属性表被移动或删除、扩展未安装	检查 .vcxproj 里的 <Import> 路径，移除失效引用
MSB3073	命令 xxx 已退出，代码为 N	**生成前/后事件**里的命令失败	看输出窗口里该命令自身的报错；路径含空格要加引号，用 $(ProjectDir) 等宏替代相对路径
MSB6006	xxx.exe 已退出，代码为 N	自定义生成步骤或工具进程返回非零	单独在命令行里跑一遍该命令，看真实输出
E0020	标识符 xxx 未定义	IntelliSense 没看到声明：路径没配到当前配置、宏或语言标准不一致	见第 3 节的处理顺序

错误号	提示信息要点	常见原因	解决方案
E1696	无法打开源文件 xxx	包含路径只配在某个配置或平台，其他配置下 IntelliSense 找不到	改到“所有配置、所有平台”，再 **重新扫描解决方案**
E0135	命名空间 std 没有成员 xxx	IntelliSense 使用的语言标准低于代码所需	统一 **C/C++ > 语言 > C++ 语言标准**，并重新扫描

14.1.5 排错的通用流程

1. **永远从第一条错误开始修**，之后的报错大多是它的连锁反应。
2. 分清两个窗口：**视图 > 错误列表**（Ctrl+\，Ctrl+E）只给摘要；完整命令行与上下文在**输出**窗口的“生成”页里。
3. 需要更多细节时，把 **工具 > 选项 > 项目和解决方案 > 生成并运行 > MSBuild 项目生成输出详细程度**调成 **详细**或 **诊断**，再重新生成。
4. 打开 **项目 > 属性 > C/C++ > 命令行**与 **链接器 > 命令行**，把 IDE 里的配置与真实命令行对照，“我明明配了”的问题多半出在这里。
5. 改完配置仍报同样的错，执行 **生成 > 重新生成解决方案**；再不行就关掉 VS，删掉 bin、obj 与 .vs 后重来。

提示 LNK2019 给出的“出错位置”往往是**引用处**而不是**定义缺失处**。真正要找的是提示里的符号名，用 Ctrl+T 全局搜索这个符号最快。

14.2 中文乱码专题

14.2.1 四层编码，先分清错在哪一层

```
源码文件（磁盘字节）
    [1] 文件编码：UTF-8 / UTF-8 with BOM / GBK
        |
        v
编译器读取
    [2] 源字符集：由 BOM 或 /source-charset 决定
        |
        v
字符串字面量（.obj 字节）
    [3] 执行字符集：由 /execution-charset 或 /utf-8 决定
        |
        v
运行时输出到控制台
    [4] 控制台按"当前输出代码页"解释这些字节
```

```
        |
        v
屏幕上的汉字
```

四层里任何一层不一致都会看到乱码。最常见的组合是：源码为 UTF-8 无 BOM（第 1 层），编译器按系统 ANSI 代码页 936 解析（第 2 层），字符串里的 UTF-8 字节又被送到 936 的控制台（第 4 层），两层皆错。

14.2.2 推荐做法与各开关的含义

做法	效果	适用场景
源码保存为 **UTF-8 with BOM**	MSVC 依据 BOM 自动识别为 UTF-8，不依赖开关	想少配选项、以 Windows 为主的项目
源码 UTF-8 无 BOM + `/utf-8`	源字符集与执行字符集都按 UTF-8 处理	团队协作、跨平台、Git 友好，**推荐**
`/source-charset:utf-8`	只指定源字符集	源码是 UTF-8，但希望执行字符集保持 GBK
`/execution-charset:gbk`	让窄字符串按 GBK 编码	需要与只认 GBK 的老代码或老接口对接
源码 GBK（936）+ 不加开关	靠系统 ANSI 代码页“碰巧能用”	仅老项目临时兼容，新项目不要这么做

必须记住的四点：

- **BOM 优先于 `/source-charset`**：文件带 BOM 时编译器按 BOM 判断编码，该开关对它不起作用。
- **`/utf-8` 是组合开关**：等价于同时指定 `/source-charset:utf-8` 与 `/execution-charset:utf-8`。
- 代码页写法上，`/source-charset:.936` 以点号开头的数字表示代码页，`utf-8` 这类名字直接书写。
- `/execution-charset` 只影响**窄字符串与字符字面量**；`u8"..."` 在 C++17 是 `const char[]`，C++20 起是 `const char8_t[]`。想只改单个文件的执行字符集，可在文件开头写 `##pragma execution_character_set("utf-8")`，但它修不了第 1 层的文件编码问题。

14.2.3 控制台输出中文的三种方案

方案 A：全程 UTF-8（推荐）。编译时加 `/utf-8`，程序启动时把控制台代码页切成 UTF-8：`SetConsoleOutputCP(CP_UTF8)` 管输出，`SetConsoleCP(CP_UTF8)` 管输入。批处理里先执行 `chcp 65001` 效果类似，但那是用户侧动作，程序自己设置才可靠。

方案 B：保持 GBK（936）。源码按 GBK 保存，或用 `/source-charset:utf-8 /execution-charset:gbk`，窄字符串里就是 GBK 字节，而中文 Windows 控制台默认输出代码页正是 936，不写任何代码即可正常显示。缺点是跨平台差、换区域设置就可能翻车。

方案 C：宽字符 + 区域设置。输出宽字符前调用 `setlocale(LC_ALL, "")`（跟随系统区域）或 `setlocale(LC_ALL, "zh-CN")`，再用 `std::wcout`。两个坑：其一，`setlocale` 不会改变控制台代码页，若同时又调了 `SetConsoleOutputCP(CP_UTF8)`，宽字符流仍按区域设置转换，可能出现双重转换的乱码；其二，不要让 `std::cout` 与 `std::wcout` 交替混用，窄宽流方向混用会导致输出错序甚至丢失。要在 UTF-8 控制台输出宽字符串，最稳的做法是先用 `WideCharToMultiByte(CP_UTF8, ...)` 转成 UTF-8 的 `std::string`，再交给 `std::cout`。

> **注意** UCRT 也支持 UTF-8 形式的区域名，但它依赖较新的 Windows 系统组件。目标机版本偏老时不要依赖这一条，直接用方案 A。

14.2.4 一个“中文输出正确”的完整示例

```cpp
// main.cpp —— 保存为 UTF-8 with BOM，或使用 /utf-8 编译
#include <windows.h>
#include <iostream>

int main() {
    // (1) 让控制台按 UTF-8 解释输出字节与输入字节
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    // (2) 执行字符集为 UTF-8 时，窄字符串里存的就是 UTF-8 字节
    std::cout << "你好，世界！\n";
    std::cout << "中文注释与中文输出都不会乱码。\n";

    // (3) 打印当前输出代码页，正常应为 65001
    std::cout << "当前输出代码页：" << GetConsoleOutputCP() << "\n";
    return 0;
}
```

```
:: 手动编译时的等价命令行
cl /utf-8 /EHsc /W4 /MD main.cpp /link /SUBSYSTEM:CONSOLE
```

工程级的做法是在 `.vcxproj` 或属性表里给 **C/C++ > 命令行 > 其他选项**统一加上 `/utf-8`，再在仓库根放一个 `.editorconfig` 并把 `charset` 设为 `utf-8-bom`，编辑器与 IDE 保存新文件时就会带上 BOM，从源头少踩一次编码的坑。

14.3 IntelliSense 与实际编译不一致

14.3.1 典型症状与原因

症状	可能原因
满屏红色波浪线，但生成成功	IntelliSense 数据库过期，或它的语言标准、宏定义与真实编译不一致
编辑器不报错，一生成就 C1083/C2065	包含路径或宏只配在某一个配置下，IntelliSense 恰好用的是能通过的那个
头文件上有绿色波浪线并报 E1696	附加包含目录里的目录不存在，或没配到“所有配置、所有平台”
成员列表缺项、std:: 补全不全	IntelliSense 的头文件搜索路径不完整，或语言标准偏低
明明存在的文件搜不到	文件只在磁盘上，没被加入项目，而 IntelliSense 按项目文件扫描

14.3.2 按顺序修复

1. **先确认真实编译结果**：按 Ctrl+Shift+B 生成一次，看 **输出**窗口。编译器的 C 系列错误才是唯一真相，E 系列仅供参考。
2. **把包含路径配到“所有配置、所有平台”**：在 **项目 > 属性 > C/C++ > 常规 > 附加包含目录**中，把左上角配置下拉切到 **所有配置**、平台切到 **所有平台**，再编辑。
3. **重新扫描**：在解决方案资源管理器中右键项目，选择 **重新扫描解决方案**。
4. **清掉缓存**：关闭 Visual Studio，删除解决方案目录下的隐藏文件夹 .vs，必要时一并删除 ipch、*.VC.db 等浏览数据库，再重新打开。
5. **对齐语言标准、宏与文件归属**：确认 **C/C++ > 语言 > C++ 语言标准**与 **C/C++ > 预处理器 > 预处理器定义**和命令行一致，并确认相关文件确实挂在项目节点下（从别的目录拖进来的文件常常只存在于磁盘）。
6. **CMake 工程**：用 **项目 > 删除缓存并重新配置**，必要时删掉 out\build 后重新打开。

实战场景 同事把 附加包含目录只加在了 Debug|x64 下，Release 一生成就 C1083，而他在编辑器里怎么看都正常——因为 IntelliSense 用的正是 Debug 配置。把下拉切成“所有配置、所有平台”重新添加后问题消失。

14.4 路径与工程习惯

14.4.1 中文与空格路径引发的怪问题

- **空格**：生成事件、自定义生成步骤、外部工具的命令行若未给路径加引号，会被拆成多个参数，报出 MSB3073 这类看不出原因的错误。

- **中文**：部分老工具与第三方脚本按 ANSI 代码页处理路径，遇到中文目录会找不到文件，或打出乱码路径让人误判。
- **长路径**：路径总长超过 260 字符时部分工具会失败。缩短目录层级，或在系统上启用长路径支持（Windows 10 1607 起有对应的组策略与 `LongPathsEnabled` 注册表项）。
- **中文用户名**：`%USERPROFILE%` 含中文时，某些工具生成的临时路径同样会出问题，属同一类故障。

14.4.2 建议的目录规范

```
D:\dev\                          <- 盘符浅、全英文、无空格
+-- MyApp\                       <- 仓库根目录
|   +-- MyApp.sln
|   +-- src\                     <- 源码
|   +-- include\                 <- 对外头文件
|   +-- third_party\             <- 第三方库（头文件 + lib + dll）
|   +-- resources\               <- 图标、图片、配置模板
|   +-- build\                   <- 生成目录（x64\Debug、x64\Release 都在这里）
```

- 在 **项目** > **属性** > **常规**里把 **输出目录**与 **中间目录**集中到 `build\$(Platform)\$(Configuration)`，源码目录保持干净。
- 把共享设置（包含目录、警告等级、`/utf-8` 等）做成 `.props` 属性表，通过 **视图** > **属性管理器**添加，避免逐个项目手工配置。
- `.vcxproj.user` 属于个人设置，不要提交。

14.4.3 一份可用的 .gitignore

```
# Visual Studio 的隐藏目录与个人设置
.vs/
*.suo
*.user
*.aps
*.VC.db

# 生成输出与中间文件
[Bb]in/
[Oo]bj/
x64/
x86/
ARM64/
Debug/
Release/
*.ilk
*.exp
*.exe
```

```
# 若仓库需要提交第三方 .lib / .dll，注释掉下面两行，或用 !third_party/** 豁免
*.lib
*.dll
```

提示 `.vs` 里存着 IntelliSense 数据库、打开过的文档、断点与窗口布局，体积可能上百 MB 且随时可重建。把它加进 `.gitignore`，是第 3 节“删掉 `.vs` 重新扫描”这招能放心使用的前提。

14.5 调试符号与断点失效

现象	原因
断点是**空心圆圈**，悬停提示“当前不会命中断点。尚未为该文档加载任何符号”	PDB 未加载，或 PDB 与二进制不是同一次生成的结果
提示“没有可执行代码与此行关联”	该行被优化掉、被内联，或本身就是声明、注释、空行
断点能命中，但行号对不上、变量显示“已优化掉”	`/O2`、`/Oy` 等优化导致代码与源码行号错位
进不了系统库或第三方库源码	“仅我的代码”跳过了非用户代码
附加到进程后断点全部失效，或换台机器打开就失效	附加的进程不是当前生成的 exe（旧进程仍在运行）；或源码路径与 PDB 中记录的不一致

按顺序修复：

1. 确认当前是 Debug 配置；若必须在 Release 下调，至少加 `/Zi`，并在 **链接器 > 调试 > 生成调试信息**选择 **是 (/DEBUG)**。
2. 执行 **生成 > 重新生成解决方案**，保证 PDB 与 exe 同源——手工拷贝过 exe 却没带 PDB 是最常见的原因。
3. 打开 **调试 > 窗口 > 模块**（`Ctrl+Alt+U`）查看目标模块的符号状态，显示“未加载”时右键加载符号，并检查 **工具 > 选项 > 调试 > 符号**的符号位置。
4. 取消 **工具 > 选项 > 调试 > 常规 > 启用“仅我的代码”**，即可单步进入系统或库代码；同时确认 **项目 > 属性 > 调试 > 调试器类型**是 **自动**或 **仅限本机**。
5. 优化引起的跳变无法消除时，在该行临时插入 `__debugbreak()` 强制中断；需要观察被优化掉的变量时，把 **C/C++ > 优化 > 优化**临时设为 **已禁用 (/Od)** 复现问题。

注意 Debug 配置里 **C/C++ > 代码生成 > 基本运行时检查**默认是 **两者 (/RTC1)**，它要求关闭优化，二者配套。不要在同一配置里既开 `/O2` 又指望 `/RTC1` 生效。

14.6 效率技巧集

14.6.1 编辑与代码

- **多光标与框选**：按住 Ctrl+Alt 点击可在多处插入光标，批量改名、批量加前缀特别顺手；按住 Alt 拖动鼠标则是框选（列选择），也可用 Shift+Alt 配方向键。
- **复制与移动行**：Ctrl+E，V 复制当前行（官方文档绑定；Ctrl+D 在不同上下文里可能被“查找”组合框占用）；Alt+↑ / Alt+↓ 上下移动行；Ctrl+L 剪切整行。
- **格式化与注释**：Ctrl+K，Ctrl+D 格式化文档，Ctrl+K，Ctrl+F 格式化选区；Ctrl+K，Ctrl+C 注释、Ctrl+K，Ctrl+U 取消注释，较新版本中 Ctrl+/ 也能切换行注释。
- **剪贴板环**：Ctrl+Shift+V 循环粘贴前几次复制的内容，误覆盖的内容还能捞回来。
- **快速操作与结构化选择**：Ctrl+. 打开灯泡菜单，补头文件、加 override、生成定义都在这里；Shift+Alt+= 与 Shift+Alt+- 可按语法结构扩展与收缩选择。

14.6.2 导航

- **Ctrl+T / Ctrl+,**：转到全部 / 导航到，模糊搜索文件、类型、成员、符号，是最高频的导航方式。
- **F12 / Alt+F12**：转到定义 / 速览定义（不离开当前文件，按 Shift+Esc 切回编辑器）。
- **Ctrl+K，Ctrl+O**：在头文件与实现文件之间来回切换，C++ 开发必备；Ctrl+Alt+↑ 可跳到当前代码块的外层。
- **Ctrl+- / Ctrl+Shift+-**：在浏览历史中后退与前进。
- **书签**：Ctrl+K，Ctrl+K 打书签，Ctrl+K，Ctrl+N 跳到下一个，Ctrl+K，Ctrl+W 打开书签窗口看全貌。
- **Ctrl+;**：把焦点送到解决方案资源管理器的搜索框，找文件不用鼠标。

14.6.3 任务列表、代码片段与环境管理

TODO 注释会自动汇总进任务列表，不容易漏：

```
// TODO: 补充参数校验
// HACK: 绕开上游库的越界问题，等对方修复
```

用 **视图 > 任务列表**（Ctrl+\，Ctrl+T）查看，双击跳到对应代码行；标记关键字可在 **工具 > 选项 > 环境 > 任务列表**里增删。代码片段用 Ctrl+K，Ctrl+X 插入，常用的有 ##if、for、switch、class、ctor、try 等；自定义片段用 **工具 > 代码片段管理器**（Ctrl+K，Ctrl+B）管理，“我的代码片段”目录下的 .snippet 就是普通 XML，改起来很容易。

环境层面的三件事值得花十分钟做一次：

- **自定义快捷键**：**工具 > 选项 > 环境 > 键盘**里按命令名搜索（如 Edit.FormatDocument）后指派组合键。
- **导出与导入设置**：**工具 > 导入和导出设置向导**可只导出“键盘”“字体和颜色”“窗口布局”等某一类设置，换机器几分钟恢复环境。

- **保存窗口布局**：把工具窗口摆顺手后用 **窗口 > 保存窗口布局**存下来，编码布局与调试布局分别保存，需要时一键切换。

14.7 常用快捷键总表

说明： 下表为 Visual Studio 默认“常规”键盘方案。若某个组合键在本机不生效，用 工具 > 选项 > 环境 > 键盘搜索对应命令确认当前绑定——键盘方案、语言设置与已装扩展都可能覆盖默认值。

14.7.1 编辑

快捷键	功能
Ctrl+E, V	复制当前行（官方文档给出的 Duplicate line 绑定）
Ctrl+D	绑定随上下文变化：官方文档将其列为工具栏“查找”组合框；若在你的环境里表现为复制行，以 **工具 > 选项 > 环境 > 键盘**的实际绑定为准
Ctrl+L	剪切整行（Edit.LineCut，文本编辑器上下文）
Alt+↑ / Alt+↓	上移 / 下移当前行或选区
Ctrl+Alt+ 点击	添加插入符（多光标编辑）
Alt+ 拖动鼠标或 Shift+Alt+ 方向键	框选（块 / 列选择）
Ctrl+Shift+V	循环剪贴板（粘贴此前的复制内容）
Ctrl+K, Ctrl+D	设置文档格式
Ctrl+K, Ctrl+F	设置选定内容的格式
Ctrl+K, Ctrl+C	注释选定内容
Ctrl+K, Ctrl+U	取消注释选定内容
Ctrl+K, Ctrl+X	插入代码片段
Ctrl+J	强制显示 IntelliSense 成员列表
Ctrl+.	快速操作（灯泡菜单）
Ctrl+M, Ctrl+O	折叠到定义
Ctrl+M, Ctrl+M	折叠 / 展开当前区域

14.7.2 导航

快捷键	功能
F12	转到定义
Ctrl+F12	转到声明
Alt+F12	速览定义（不离开当前文件）

快捷键	功能
Shift+F12	查找所有引用
Ctrl+T	转到全部（文件、类型、成员、符号）
Ctrl+,	导航到（Navigate To，行为接近“转到全部”）
Ctrl+G	转到行
Ctrl+K, Ctrl+O	在头文件与代码文件间切换
Ctrl+K, Ctrl+T 或 Ctrl+Alt+K	调用层次结构
Ctrl+- / Ctrl+Shift+-	向后 / 向前导航
F8 / Shift+F8	下一个 / 上一个位置（错误列表或输出窗口）
Alt+PgDn / Alt+PgUp	下一个 / 上一个问题
Ctrl+K, Ctrl+K	切换书签
Ctrl+K, Ctrl+N / Ctrl+K, Ctrl+P	下一个 / 上一个书签
Ctrl+K, Ctrl+W	书签窗口
Ctrl+F / Ctrl+Shift+F	快速查找 / 在文件中查找
Ctrl+Q	搜索 Visual Studio（快速启动）

14.7.3 调试

快捷键	功能
F5 / Ctrl+F5	开始调试 / 开始执行（不调试）
Shift+F5	停止调试
Ctrl+Shift+F5	重启调试
F9	在当前行切换断点
Ctrl+Shift+F9	删除所有断点
Ctrl+Alt+B	断点窗口
F10	逐过程
F11	逐语句
Shift+F11	跳出
Ctrl+F10	运行到光标处
Ctrl+Shift+F10	设置下一语句
Ctrl+Alt+P	附加到进程
Shift+F9 或 Ctrl+Alt+Q	快速监视
Ctrl+Alt+C	调用堆栈窗口
Ctrl+Alt+U	模块窗口
Ctrl+Alt+V, L	局部变量窗口
Ctrl+Alt+W, 1	监视 1 窗口

快捷键	功能
Ctrl+Alt+I	即时窗口

14.7.4 构建

快捷键	功能
Ctrl+Shift+B	生成解决方案
Ctrl+F7	编译当前文件
Ctrl+Break	取消生成
Alt+F11	对解决方案运行代码分析
Shift+F4	属性页（配置相关设置的入口）
生成 > 配置管理器	无默认快捷键，切换配置与平台
生成 > 批生成	无默认快捷键，一次生成多个配置
项目 > 重定解决方案目标	无默认快捷键，解决 MSB8020、MSB8036 时使用

14.7.5 窗口

快捷键	功能
Ctrl+Alt+L	解决方案资源管理器
F4	属性窗口
Ctrl+Alt+X	工具箱
Ctrl+Shift+E	资源视图
Ctrl+Alt+A	命令窗口
Ctrl+\, Ctrl+E	错误列表
Ctrl+\, Ctrl+T	任务列表
Ctrl+K, Ctrl+B	代码片段管理器
Ctrl+Tab / Ctrl+Shift+Tab	下一个 / 上一个文档窗口导航
Ctrl+F4 或 Ctrl+W	关闭当前文档窗口
Shift+Esc	关闭工具窗口
Alt+F7 / Shift+Alt+F7	下一个 / 上一个工具窗口导航
Shift+Alt+Enter	全屏
窗口 > 保存窗口布局	无默认快捷键，保存当前布局

14.8 学习资源

官方与权威资料：

- Visual Studio 文档：IDE 功能、调试、测试与部署的官方入口。
- Microsoft C++ 文档：语言、库与生成工具链，其中包含 MSVC 编译器选项 与 链接器选项 的完整参考。
- C++ Core Guidelines：写出少踩坑的现代 C++ 的权威指南。
- cppreference（中文）：标准库与语言特性速查，遇到编译错误时对照最可靠。

本仓库内的相关笔记：

- **Visual Studio 入门**：第一次装好 VS 跑通项目。
- **CMake 使用指南**：用 CMake 管理工程时的替代工作流。
- **VS Code C++ 配置指南**：轻量编辑器的另一种选择。

14.9 小结

- 先看错误号前缀：`MSB` 是项目与工具集、`C` 是编译器、`LNK` 是链接器、`E` 只是 IntelliSense；永远从第一条错误开始修。
- `LNK2019` 找定义与库，`LNK2005` 查头文件里的非 `inline` 定义与重复包含，`LNK1104` 先想到“程序还在运行导致 exe 被占用”。
- 中文乱码要分清源码编码、源字符集、执行字符集、控制台代码页四层，推荐“UTF-8 with BOM 或 `/utf-8`”配合 `SetConsoleOutputCP(CP_UTF8)`。
- IntelliSense 与编译不一致时依次做：确认真实编译结果、把包含路径配到“所有配置、所有平台”、重新扫描解决方案、删除 `.vs`。
- 工程路径坚持全英文、无空格、层级浅；`.vs`、`bin`、`obj` 一律进 `.gitignore`。
- 断点空心先查 PDB 与配置，其次查“仅我的代码”与调试器类型，最后才怀疑优化。
- 效率提升集中在四个动作：`Ctrl+T` 导航、`Ctrl+Alt+` 点击多光标、`Ctrl+K, Ctrl+D` 格式化、用书签与任务列表管住待办。

14.10 练习题

1. 故意制造一次 `LNK2019`（声明了函数却不实现），用错误列表与 `Ctrl+T` 定位并修复，记录提示信息里的关键字段。
2. 在头文件里定义一个非 `inline` 的全局函数，被两个 `.cpp` 包含，复现 `LNK2005` 与 `LNK1169`，然后给出两种不同的修复方式。
3. 把一个含中文注释的 GBK 源码在不加 `/utf-8` 的情况下编译，观察 `C4819`，再分别用“另存为 UTF-8 with BOM”和 `/utf-8` 修好，比较两种做法在代码页处理上的差异。
4. 写一个输出中文的控制台程序，分别在 `chcp 936` 与 `chcp 65001` 下运行，解释结果不同的原因，再改用 `SetConsoleOutputCP(CP_UTF8)` 让程序自己保证正确。
5. 把 *附加包含目录*只配在 Debug 配置下，制造 IntelliSense 与实际编译不一致，然后按“所有配置 + 重新扫描 + 删除 `.vs`”的顺序修好。

6. 用 **工具 > 选项 > 环境 > 键盘**给“设置文档格式”指派一个自定义组合键，再用 **工具 > 导入和导出设置向导**把键盘设置导出为 `.vssettings` 文件。

14.11 相关笔记

- **本教程：Visual Studio 2026 使用教程**
- **上一章：发布与部署**
- **C++ 教程：Visual Studio 入门**
- **VS Code 配置：VS Code C++ 配置指南**
- **CMake：CMake 使用指南**

附录 A　速查与资源

本附录把全书反复用到的版本信息、语言标准档位与官方资源集中列出，便于脱离正文速查。

A.1　版本与工具集速查

A.1.1　IDE 与 MSVC 的对应关系

从 Visual Studio 2026 起，MSVC 编译器的版本号与 IDE 版本号**解耦**：IDE 每月更新，MSVC 约每六个月发布一个新版本。

MSVC	`_MSC_VER`	编译器	首次随附于	支持情况
14.50	`1950`	19.50	VS 2026 18.0（2025 年 11 月）	长期支持，至 2028 年 11 月
14.51	`1951`	19.51	VS 2026 18.6（2026 年 5 月）	标准支持，至 2027 年 2 月
14.52	`1952`	19.52	预览通道	预览

作为对照，Visual Studio 2022 的最后几个版本是 MSVC 14.43（`1943`）与 14.44（`1944`）。

A.1.2　平台工具集

Visual Studio	平台工具集	说明
Visual Studio 2015	`v140`	旧项目可能仍在用
Visual Studio 2017	`v141`	
Visual Studio 2019	`v142`	
Visual Studio 2022	`v143`	迁移期可与 `v145` 并存
Visual Studio 2026	`v145`	本书默认

A.1.3　关键版本节点

版本	时间	要点
Insiders 通道上线	2025 年 9 月 9 日	取代 VS 2022 时代的 Preview 通道
18.0（GA）	2025 年 11 月 11 日	正式发布，随附 MSVC 14.50 与 `v145`
18.6	2026 年 5 月	随附 MSVC 14.51，新增 `<flat_map>` 与 `<flat_set>`
18.10.3	2026 年 9 月 29 日	本书写作时的最新稳定版

A.2 C++ 语言标准档位速查

属性页选项	编译器开关	说明
ISO C++14 标准	`/std:c++14`	编译器在未指定 `/std` 时的默认值
ISO C++17 标准	`/std:c++17`	
ISO C++20 标准	`/std:c++20`	新建 C++ 项目的默认值；隐含 `/permissive-`
预览–ISO C++23 标准	`/std:c++23preview`	只启用 C++23 范围内的特性
预览–最新工作草案	`/std:c++latest`	含进行中的实验特性；隐含 `/permissive-`

注意 **没有 `/std:c++26` 这个开关**。想用前沿特性只能挂 `/std:c++latest`。另外，”新建项目默认 C++20” 来自项目**模板**写入的 `/std:c++20`，与”编译器默认 C++14” 并不矛盾——这是两件事，详见第 1 章。

A.3 官方资源

资源	地址
Visual Studio 2026 发行说明	https://learn.microsoft.com/visualstudio/releases/2026/release-notes
MSVC Build Tools 新增功能	https://learn.microsoft.com/cpp/overview/what-s-new-for-msvc
MSVC 编译器版本对照表	https://learn.microsoft.com/cpp/overview/compiler-versions

资源	地址
/std 语言标准开关	https://learn.microsoft.com/cpp/build/reference/std-specify-language-standard-version
C++ 二进制兼容性	https://learn.microsoft.com/cpp/porting/binary-compat-2015-2017
Visual Studio 2026 系统要求	https://learn.microsoft.com/visualstudio/releases/2026/vs-system-requirements
C++ 团队博客	https://devblogs.microsoft.com/cppblog/
C++ Core Guidelines	https://isocpp.github.io/CppCoreGuidelines/
cppreference	https://en.cppreference.com/

A.4 本仓库内的相关笔记

- **C++ 教程**（21 篇）——C++ 学习，语言基础到学生管理系统实战
- **CMake 使用指南**——CMake 使用指南，构建系统本身
- **Eigen 库使用指南**——Eigen 库使用指南，矩阵与几何运算
- **VS Code C++ 配置指南**——VS code 配置指南，轻量编辑器路线
- **常用 Git 命令**——常用 Git 命令.md，版本管理